Tutoriel Sage

Version 10.10

The Sage Group

28 octobre 2026

Table des matières

1 Introduction **3**
- 1.1 Installation . . . 5
- 1.2 Les différentes manières d'utiliser Sage . . . 5
- 1.3 Objectifs à long terme de Sage . . . 5

2 Visite guidée **7**
- 2.1 Affectation, égalité et arithmétique . . . 7
- 2.2 Obtenir de l'aide . . . 10
- 2.3 Fonctions, indentation et itération . . . 13
- 2.4 Algèbre de base et calcul infinitésimal . . . 19
- 2.5 Graphiques . . . 28
- 2.6 Problèmes fréquents concernant les fonctions . . . 32
- 2.7 Anneaux de base . . . 38
- 2.8 Polynômes . . . 41
- 2.9 Parents, conversions, coercitions . . . 49
- 2.10 Algèbre linéaire . . . 57
- 2.11 Groupes finis, groupes abéliens . . . 63
- 2.12 Théorie des nombres . . . 66
- 2.13 Quelques mathématiques plus avancées . . . 71

3 La ligne de commande interactive **87**
- 3.1 Votre session Sage . . . 88
- 3.2 Journal des entrées-sorties . . . 90
- 3.3 Coller du texte ignore les invites . . . 91
- 3.4 Mesure du temps d'exécution d'une commande . . . 92
- 3.5 Trucs et astuces IPython . . . 95
- 3.6 Erreurs et exceptions . . . 96
- 3.7 Recherche en arrière et complétion de ligne de commande . . . 97
- 3.8 Aide en ligne . . . 98
- 3.9 Enregistrer et charger des objets individuellement . . . 102
- 3.10 Enregister et recharger des sessions entières . . . 105

4 Interfaces **107**
- 4.1 GP/PARI . . . 107
- 4.2 GAP . . . 109
- 4.3 Singular . . . 110
- 4.4 Maxima . . . 112

5 Sage, LaTeX et compagnie **117**
5.1 Vue d'ensemble . . . 117
5.2 Utilisation de base . . . 119
5.3 Personnaliser le code LaTeX produit . . . 120
5.4 Personnaliser le traitement du code par LaTeX . . . 124
5.5 Example : rendu de graphes avec tkz-graph . . . 125
5.6 Une installation TeX pleinement opérationnelle . . . 126
5.7 Programmes externes . . . 126

6 Programmation **129**
6.1 Charger et attacher des fichiers Sage . . . 129
6.2 Écrire des programmes compilés . . . 130
6.3 Scripts Python/Sage autonomes . . . 131
6.4 Types de données . . . 132
6.5 Listes, n-uplets et séquences . . . 134
6.6 Dictionnaires . . . 138
6.7 Ensembles . . . 139
6.8 Itérateurs . . . 140
6.9 Boucles, fonctions, structures de contrôle et comparaisons . . . 141
6.10 Profilage (profiling) . . . 145

7 Utiliser SageTeX **149**

8 Postface **151**
8.1 Pourquoi Python ? . . . 151
8.2 Comment puis-je contribuer ? . . . 153
8.3 Comment citer Sage ? . . . 153

9 Annexe **155**
9.1 Priorité des opérateurs arithmétiques binaires . . . 155

10 Bibliographie **157**

11 Index et tables **159**

Bibliographie **161**

Index **163**

Sage est un logiciel mathématique libre destiné à la recherche et à l'enseignement en algèbre, géométrie, arithmétique, théorie des nombres, cryptographie, calcul scientifique et dans d'autres domaines apparentés. Le modèle de développement de Sage comme ses caractéristiques techniques se distinguent par un souci extrême d'ouverture, de partage, de coopération et de collaboration : notre but est de construire la voiture, non de réinventer la roue. L'objectif général de Sage est de créer une alternative libre viable à Maple, Mathematica, Magma et MATLAB.

Ce tutoriel est la meilleure façon de se familiariser avec Sage en quelques heures. Il est disponible en versions HTML et PDF, ainsi que depuis le notebook Sage (cliquez sur `Help`, puis sur `Tutorial` pour parcourir le tutoriel de façon interactive depuis Sage).

Ce document est distribué sous licence Creative Commons Paternité-Partage des conditions initiales à l'identique 3.0 Unported.

This work is licensed under a Creative Commons Attribution-Share Alike 3.0 License.

CHAPITRE 1

Introduction

Explorer ce tutoriel en entier devrait vous prendre au maximum trois à quatre heures. Vous pouvez le lire en version HTML ou PDF, ou encore l'explorer interactivement à l'intérieur de Sage en cliquant sur `Help` puis sur `Tutorial` depuis le *notebook* (il est possible que vous tombiez sur la version en anglais).

Sage est écrit en grande partie en Python, mais aucune connaissance de Python n'est nécessaire pour lire ce tutoriel. Par la suite, vous souhaiterez sans doute apprendre Python, et il existe pour cela de nombreuses ressources libres d'excellente qualité : le Python Beginner's Guide [PyB] répertorie de nombreuses options. Mais si ce que vous voulez est découvrir rapidement Sage, ce tutoriel est le bon endroit où commencer. Voici quelques exemples :

```
sage: 2 + 2
4
sage: factor(-2007)
-1 * 3^2 * 223

sage: A = matrix(4,4, range(16)); A
[ 0  1  2  3]
[ 4  5  6  7]
[ 8  9 10 11]
[12 13 14 15]

sage: factor(A.charpoly())
x^2 * (x^2 - 30*x - 80)

sage: m = matrix(ZZ,2, range(4))
sage: m[0,0] = m[0,0] - 3
sage: m
[-3  1]
[ 2  3]

sage: E = EllipticCurve([1,2,3,4,5]);
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5
```

(suite sur la page suivante)

(suite de la page précédente)

```
over Rational Field
sage: E.anlist(10)
[0, 1, 1, 0, -1, -3, 0, -1, -3, -3, -3]
sage: E.rank()
1

sage: k = 1/(sqrt(3)*I + 3/4 + sqrt(73)*5/9); k
36/(20*sqrt(73) + 36*I*sqrt(3) + 27)
sage: N(k)
0.165495678130644 - 0.0521492082074256*I
sage: N(k,30)       # 30 "bits"
0.16549568 - 0.052149208*I
sage: latex(k)
\frac{36}{20 \, \sqrt{73} + 36 i \, \sqrt{3} + 27}
```

```
>>> from sage.all import *
>>> Integer(2) + Integer(2)
4
>>> factor(-Integer(2007))
-1 * 3^2 * 223

>>> A = matrix(Integer(4),Integer(4), range(Integer(16))); A
[ 0  1  2  3]
[ 4  5  6  7]
[ 8  9 10 11]
[12 13 14 15]

>>> factor(A.charpoly())
x^2 * (x^2 - 30*x - 80)

>>> m = matrix(ZZ,Integer(2), range(Integer(4)))
>>> m[Integer(0),Integer(0)] = m[Integer(0),Integer(0)] - Integer(3)
>>> m
[-3  1]
[ 2  3]

>>> E = EllipticCurve([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)]);
>>> E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5
over Rational Field
>>> E.anlist(Integer(10))
[0, 1, 1, 0, -1, -3, 0, -1, -3, -3, -3]
>>> E.rank()
1

>>> k = Integer(1)/(sqrt(Integer(3))*I + Integer(3)/Integer(4) +␣
↪sqrt(Integer(73))*Integer(5)/Integer(9)); k
36/(20*sqrt(73) + 36*I*sqrt(3) + 27)
>>> N(k)
0.165495678130644 - 0.0521492082074256*I
>>> N(k,Integer(30))       # 30 "bits"
0.16549568 - 0.052149208*I
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> latex(k)
\frac{36}{20 \, \sqrt{73} + 36 i \, \sqrt{3} + 27}
```

1.1 Installation

Si Sage n'est pas installé sur votre ordinateur, vous pouvez essayer quelques commandes en ligne à l'adresse http://sagecell.sagemath.org.

Des instructions pour installer Sage sur votre ordinateur sont disponibles dans le guide d'installation (*Installation Guide*), dans la section documentation de la page web principale de Sage [SA]. Nous nous limiterons ici à quelques remarques.

1. La version téléchargeable de Sage vient avec ses dépendances. Autrement dit, bien que Sage utilise Python, IPython, PARI, GAP, Singular, Maxima, NTL, GMP, etc., vous n'avez pas besoin de les installer séparément, ils sont fournis dans la distribution Sage. En revanche, pour utiliser certaines des fonctionnalités de Sage, par exemple Macaulay ou KASH, il vous faudra d'abord avoir le logiciel correspondant installé sur votre ordinateur.
2. La version binaire pré-compilée de Sage (disponible sur le site web) est souvent plus facile et plus rapide à installer que la distribution en code source. Pour l'installer, décompressez l'archive et lancez simplement le programme `sage`.
3. Si vous souhaitez utiliser SageTeX (qui permet d'insérer automatiquement dans un document LaTeX les résultats de calculs effectués avec Sage), vous devrez faire en sorte que votre distribution LaTeX le trouve (et, plus précisément, en trouve la version correspondant à la version de Sage que vous utilisez). Pour ce faire, consultez la section « Make SageTeX known to TeX » dans le guide d'installation (Sage installation guide, ce lien devrait pointer vers une copie locale). L'installation est facile : il suffit de copier un fichier dans un répertoire que TeX examine, ou de régler une variable d'environnement.

   La documentation de SageTeX se trouve dans le répertoire `$SAGE_ROOT/venv/share/texmf/tex/latex/sagetex/`, où « `$SAGE_ROOT` » est le répertoire où vous avez installé Sage, par exemple `/opt/sage-9.6`.

1.2 Les différentes manières d'utiliser Sage

Il y a plusieurs façons d'utiliser Sage.

— **Interface graphique (« notebook ») :** démarrer `sage -n jupyter` ; lire Jupyter documentation on-line ;
— **Ligne de commande :** voir *La ligne de commande interactive* ;
— **Programmes :** en écrivant des programmes interprétés ou compilés en Sage (voir *Charger et attacher des fichiers Sage* et *Écrire des programmes compilés*) ;
— **Scripts :** en écrivant des programmes Python indépendants qui font appel à la bibliothèque Sage (voir *Scripts Python/Sage autonomes*).

1.3 Objectifs à long terme de Sage

— **Être utile :** le public visé par Sage comprend les étudiants (du lycée au doctorat), les enseignants et les chercheurs en mathématiques. Le but est de fournir un logiciel qui permette d'explorer toutes sortes de constructions mathématiques et de faire des expériences avec, en algèbre, en géométrie, en arithmétique et théorie des nombres, en analyse, en calcul numérique, etc. Sage facilite l'expérimentation interactive avec des objets mathématiques.
— **Être efficace :** c'est-à-dire rapide. Sage fait appel à des logiciels matures et soigneusement optimisés comme GMP, PARI, GAP et NTL, ce qui le rend très rapide pour certaines opérations.
— **Être libre/open-source :** le code source doit être disponible librement et lisible, de sorte que les utilisateurs puissent comprendre ce que fait le système et l'étendre facilement. Tout comme les mathématiciens acquièrent une compréhension plus profonde d'un théorème en lisant sa preuve soigneusement, ou simplement en la parcourant, les personnes qui font des calculs devraient être en mesure de comprendre comment ceux-ci fonctionnent en lisant un code source documenté. Si vous publiez un article dans lequel vous utilisez Sage pour faire des calculs, vous

avez la garantie que vos lecteurs auront accès librement à Sage et à son code source, et vous pouvez même archiver et redistribuer vous-même la version de Sage que vous utilisez.

— **Être facile à compiler :** le code source de Sage devrait être facile à compiler pour les utilisateurs de Linux, d'OS X et de Windows. Cela rend le système plus flexible pour les utilisateurs qui souhaiteraient le modifier.
— **Favoriser la coopération :** fournir des interfaces robustes à la plupart des autres systèmes de calcul formel, notamment PARI, GAP, Singular, Maxima, KASH, Magma, Maple et Mathematica. Sage cherche à unifier et étendre les logiciels existants.
— **Être bien documenté :** tutoriel, guide du programmeur, manuel de référence, guides pratiques, avec de nombreux exemples et une discussion des concepts mathématiques sous-jacents.
— **Être extensible :** permettre de définir de nouveaux types de données ou des types dérivés de types existants, et d'utiliser du code écrit dans différents langages.
— **Être convivial :** il doit être facile de comprendre quelles fonctionnalités sont disponibles pour travailler avec un objet donné, et de consulter la documentation et le code source. Également, arriver à un bon niveau d'assistance utilisateur.

CHAPITRE 2

Visite guidée

Cette partie présente une sélection des possibilités actuellement offertes par Sage. Pour plus d'exemples, on se reportera à *Sage Constructions* (« Construction d'objets en Sage »), dont le but est de répondre à la question récurrente du type « Comment faire pour construire … ? ».

On consultera aussi le *Sage Reference Manual* (« Manuel de référence pour Sage »), qui contient des milliers d'exemples supplémentaires. Notez également que vous pouvez explorer interactivement ce tutoriel — ou sa version en anglais — en cliquant sur `Help` à partir du notebook de Sage.

(Si vous lisez ce tutoriel à partir du *notebook* de Sage, appuyez sur `maj-enter` pour évaluer le contenu d'une cellule. Vous pouvez même éditer le contenu avant d'appuyer sur `maj-entrée`. Sur certains Macs, il vous faudra peut-être appuyer sur `maj-return` plutôt que `maj-entrée`).

2.1 Affectation, égalité et arithmétique

A quelques exceptions mineures près, Sage utilise le langage de programmation Python, si bien que la plupart des ouvrages d'introduction à Python vous aideront à apprendre Sage.

Sage utilise = pour les affectations. Il utilise ==, <=, >=, < et > pour les comparaisons.

```
sage: a = 5
sage: a
5
sage: 2 == 2
True
sage: 2 == 3
False
sage: 2 < 3
True
sage: a == 5
True
```

```
>>> from sage.all import *
>>> a = Integer(5)
>>> a
5
>>> Integer(2) == Integer(2)
True
>>> Integer(2) == Integer(3)
False
>>> Integer(2) < Integer(3)
True
>>> a == Integer(5)
True
```

Sage fournit toutes les opérations mathématiques de base :

```
sage: 2**3    #  ** désigne l'exponentiation
8
sage: 2^3     #  ^ est un synonyme de ** (contrairement à Python)
8
sage: 10 % 3  #  pour des arguments entiers, % signifie mod, i.e., le reste dans la␣
↪division euclidienne
1
sage: 10/4
5/2
sage: 10//4   #  pour des arguments entiers, // renvoie le quotient dans la division␣
↪euclidienne
2
sage: 4 * (10 // 4) + 10 % 4 == 10
True
sage: 3^2*4 + 2%5
38
```

```
>>> from sage.all import *
>>> Integer(2)**Integer(3)    #  ** désigne l'exponentiation
8
>>> Integer(2)**Integer(3)     #  ^ est un synonyme de ** (contrairement à Python)
8
>>> Integer(10) % Integer(3)  #  pour des arguments entiers, % signifie mod, i.e., le␣
↪reste dans la division euclidienne
1
>>> Integer(10)/Integer(4)
5/2
>>> Integer(10)//Integer(4)   #  pour des arguments entiers, // renvoie le quotient␣
↪dans la division euclidienne
2
>>> Integer(4) * (Integer(10) // Integer(4)) + Integer(10) % Integer(4) == Integer(10)
True
>>> Integer(3)**Integer(2)*Integer(4) + Integer(2)%Integer(5)
38
```

Le calcul d'une expression telle que `3^2*4 + 2%5` dépend de l'ordre dans lequel les opérations sont effectuées ; ceci est expliqué dans l'annexe *Priorité des opérateurs arithmétiques binaires* *Priorité des opérateurs arithmétiques binaires*.

Sage fournit également un grand nombre de fonctions mathématiques usuelles ; en voici quelques exemples choisis :

```
sage: sqrt(3.4)
1.84390889145858
sage: sin(5.135)
-0.912021158525540
sage: sin(pi/3)
1/2*sqrt(3)
```

```
>>> from sage.all import *
>>> sqrt(RealNumber('3.4'))
1.84390889145858
>>> sin(RealNumber('5.135'))
-0.912021158525540
>>> sin(pi/Integer(3))
1/2*sqrt(3)
```

Comme le montre le dernier de ces exemples, certaines expressions mathématiques renvoient des valeurs “exactes” plutôt que des approximations numériques. Pour obtenir une approximation numérique, on utilise au choix la fonction `n` ou la méthode `n` (chacun de ces noms possède le nom plus long `numerical_approx`, la fonction `N` est identique à `n`). Celles-ci acceptent, en argument optionnel, `prec`, qui indique le nombre de bits de précisions requis, et `digits`, qui indique le nombre de décimales demandées ; par défaut, il y a 53 bits de précision.

```
sage: exp(2)
e^2
sage: n(exp(2))
7.38905609893065
sage: sqrt(pi).numerical_approx()
1.77245385090552
sage: sin(10).n(digits=5)
-0.54402
sage: N(sin(10),digits=10)
-0.5440211109
sage: numerical_approx(pi, prec=200)
3.1415926535897932384626433832795028841971693993751058209749
```

```
>>> from sage.all import *
>>> exp(Integer(2))
e^2
>>> n(exp(Integer(2)))
7.38905609893065
>>> sqrt(pi).numerical_approx()
1.77245385090552
>>> sin(Integer(10)).n(digits=Integer(5))
-0.54402
>>> N(sin(Integer(10)),digits=Integer(10))
-0.5440211109
>>> numerical_approx(pi, prec=Integer(200))
3.1415926535897932384626433832795028841971693993751058209749
```

Python est doté d’un typage dynamique. Ainsi la valeur à laquelle fait référence une variable est toujours associée à un type donné, mais une variable donnée peut contenir des valeurs de plusieurs types Python au sein d’une même portée :

```
sage: a = 5   # a est un entier
sage: type(a)
```

(suite sur la page suivante)

(suite de la page précédente)

```
<class 'sage.rings.integer.Integer'>
sage: a = 5/3  # a est maintenant un rationnel...
sage: type(a)
<class 'sage.rings.rational.Rational'>
sage: a = 'hello'  # ...et maintenant une chaîne
sage: type(a)
<... 'str'>
```

```
>>> from sage.all import *
>>> a = Integer(5)    # a est un entier
>>> type(a)
<class 'sage.rings.integer.Integer'>
>>> a = Integer(5)/Integer(3)  # a est maintenant un rationnel...
>>> type(a)
<class 'sage.rings.rational.Rational'>
>>> a = 'hello'  # ...et maintenant une chaîne
>>> type(a)
<... 'str'>
```

Le langage de programmation C, qui est statiquement typé, est bien différent : une fois déclarée de type int, une variable ne peut contenir que des int au sein de sa portée.

2.2 Obtenir de l'aide

Sage est doté d'une importante documentation intégrée, accessible en tapant (par exemple) le nom d'une fonction ou d'une constante suivi d'un point d'interrogation :

```
sage: tan?
Type:        <class 'sage.calculus.calculus.Function_tan'>
Definition:  tan( [noargspec] )
Docstring:

    The tangent function

    EXAMPLES:
        sage: tan(pi)
        0
        sage: tan(3.1415)
        -0.000092653590058191 3
        sage: tan(3.1415/4)
        0.999953674278156
        sage: tan(pi/4)
        1
        sage: tan(1/2)
        tan(1/2)
        sage: RR(tan(1/2))
        0.546302489843790
sage: log2?
Type:        <class 'sage.functions.constants.Log2'>
Definition:  log2( [noargspec] )
Docstring:
```

(suite sur la page suivante)

(suite de la page précédente)

```
    The natural logarithm of the real number 2.

    EXAMPLES:
        sage: log2
        log2
        sage: float(log2)
        0.69314718055994529
        sage: RR(log2)
        0.693147180559945
        sage: R = RealField(200); R
        Real Field with 200 bits of precision
        sage: R(log2)
        0.69314718055994530941723212145817656807550013436025525412068
        sage: l = (1-log2)/(1+log2); l
        (1 - log(2))/(log(2) + 1)
        sage: R(l)
        0.18123221829928249948761381864650311423330609774776013488056
        sage: maxima(log2)
        log(2)
        sage: maxima(log2).float()
        .6931471805599453
        sage: gp(log2)
        0.69314718055994530941723212121...
sage: sudoku?
File:        sage/local/lib/python2.5/site-packages/sage/games/sudoku.py
Type:        <... 'function'>
Definition:  sudoku(A)
Docstring:

    Solve the 9x9 Sudoku puzzle defined by the matrix A.

    EXAMPLE:
        sage: A = matrix(ZZ,9,[5,0,0, 0,8,0, 0,4,9, 0,0,0, 5,0,0,
    0,3,0, 0,6,7, 3,0,0, 0,0,1, 1,5,0, 0,0,0, 0,0,0, 0,0,0, 2,0,8, 0,0,0,
    0,0,0, 0,0,0, 0,1,8, 7,0,0, 0,0,4, 1,5,0,   0,3,0, 0,0,2,
    0,0,0, 4,9,0, 0,5,0, 0,0,3])
        sage: A
        [5 0 0 0 8 0 0 4 9]
        [0 0 0 5 0 0 0 3 0]
        [0 6 7 3 0 0 0 0 1]
        [1 5 0 0 0 0 0 0 0]
        [0 0 0 2 0 8 0 0 0]
        [0 0 0 0 0 0 0 1 8]
        [7 0 0 0 0 4 1 5 0]
        [0 3 0 0 0 2 0 0 0]
        [4 9 0 0 5 0 0 0 3]
        sage: sudoku(A)
        [5 1 3 6 8 7 2 4 9]
        [8 4 9 5 2 1 6 3 7]
        [2 6 7 3 4 9 5 8 1]
        [1 5 8 4 6 3 9 7 2]
```

(suite sur la page suivante)

(suite de la page précédente)

```
        [9 7 4 2 1 8 3 6 5]
        [3 2 6 7 9 5 4 1 8]
        [7 8 2 9 3 4 1 5 6]
        [6 3 5 1 7 2 8 9 4]
        [4 9 1 8 5 6 7 2 3]
```

```
>>> from sage.all import *
>>> tan?
Type:        <class 'sage.calculus.calculus.Function_tan'>
Definition:  tan( [noargspec] )
Docstring:

    The tangent function

    EXAMPLES:
>>> tan(pi)
        0
>>> tan(RealNumber('3.1415'))
        -0.0000926535900581913
>>> tan(RealNumber('3.1415')/Integer(4))
        0.999953674278156
>>> tan(pi/Integer(4))
        1
>>> tan(Integer(1)/Integer(2))
        tan(1/2)
>>> RR(tan(Integer(1)/Integer(2)))
        0.546302489843790
>>> log2?
Type:        <class 'sage.functions.constants.Log2'>
Definition:  log2( [noargspec] )
Docstring:

    The natural logarithm of the real number 2.

    EXAMPLES:
>>> log2
        log2
>>> float(log2)
        0.69314718055994529
>>> RR(log2)
        0.693147180559945
>>> R = RealField(Integer(200)); R
        Real Field with 200 bits of precision
>>> R(log2)
        0.69314718055994530941723212145817656807550013436025525412068
>>> l = (Integer(1)-log2)/(Integer(1)+log2); l
        (1 - log(2))/(log(2) + 1)
>>> R(l)
        0.18123221829928249948761381864650311423330609774776013488056
>>> maxima(log2)
        log(2)
>>> maxima(log2).float()
```

(suite sur la page suivante)

(suite de la page précédente)

```
        .6931471805599453
>>> gp(log2)
        0.693147180559945309417232121...
>>> sudoku?
File:        sage/local/lib/python2.5/site-packages/sage/games/sudoku.py
Type:        <... 'function'>
Definition:  sudoku(A)
Docstring:

    Solve the 9x9 Sudoku puzzle defined by the matrix A.

    EXAMPLE:
>>> A = matrix(ZZ,Integer(9),[Integer(5),Integer(0),Integer(0), Integer(0),Integer(8),
↪Integer(0), Integer(0),Integer(4),Integer(9), Integer(0),Integer(0),Integer(0),␣
↪Integer(5),Integer(0),Integer(0),
    0,3,0, 0,6,7, 3,0,0, 0,0,1, 1,5,0, 0,0,0, 0,0,0, 0,0,0, 2,0,8, 0,0,0,
    0,0,0, 0,0,0, 0,1,8, 7,0,0, 0,0,4, 1,5,0,   0,3,0, 0,0,2,
    0,0,0, 4,9,0, 0,5,0, 0,0,3])
>>> A
        [5 0 0 0 8 0 0 4 9]
        [0 0 0 5 0 0 0 3 0]
        [0 6 7 3 0 0 0 0 1]
        [1 5 0 0 0 0 0 0 0]
        [0 0 0 2 0 8 0 0 0]
        [0 0 0 0 0 0 0 1 8]
        [7 0 0 0 0 4 1 5 0]
        [0 3 0 0 0 2 0 0 0]
        [4 9 0 0 5 0 0 0 3]
>>> sudoku(A)
        [5 1 3 6 8 7 2 4 9]
        [8 4 9 5 2 1 6 3 7]
        [2 6 7 3 4 9 5 8 1]
        [1 5 8 4 6 3 9 7 2]
        [9 7 4 2 1 8 3 6 5]
        [3 2 6 7 9 5 4 1 8]
        [7 8 2 9 3 4 1 5 6]
        [6 3 5 1 7 2 8 9 4]
        [4 9 1 8 5 6 7 2 3]
```

Sage dispose aussi de la complétion de ligne de commande, accessible en tapant les quelques premières lettres du nom d'une fonction puis en appuyant sur la touche tabulation. Ainsi, si vous tapez `ta` suivi de `TAB`, Sage affichera `tachyon, tan, tanh, taylor`. C'est une façon commode de voir quels noms de fonctions et d'autres structures sont disponibles en Sage.

2.3 Fonctions, indentation et itération

Les définitions de fonctions en Sage sont introduites par la commande `def`, et la liste des noms des paramètres est suivie de deux points, comme dans :

```
sage: def is_even(n):
....:     return n%2 == 0
```

(suite sur la page suivante)

(suite de la page précédente)

```
sage: is_even(2)
True
sage: is_even(3)
False
```

```
>>> from sage.all import *
>>> def is_even(n):
...     return n%Integer(2) == Integer(0)
>>> is_even(Integer(2))
True
>>> is_even(Integer(3))
False
```

Remarque : suivant la version du *notebook* que vous utilisez, il est possible que vous voyez trois points `....:` au début de la deuxième ligne de l'exemple. Ne les entrez pas, ils servent uniquement à signaler que le code est indenté.

Les types des paramètres ne sont pas spécifiés dans la définition de la fonction. Il peut y avoir plusieurs paramètres, chacun accompagné optionnellement d'une valeur par défaut. Par exemple, si la valeur de `divisor` n'est pas donnée lors d'un appel à la fonction ci-dessous, la valeur par défaut `divisor=2` est utilisée.

```
sage: def is_divisible_by(number, divisor=2):
....:     return number%divisor == 0
sage: is_divisible_by(6,2)
True
sage: is_divisible_by(6)
True
sage: is_divisible_by(6, 5)
False
```

```
>>> from sage.all import *
>>> def is_divisible_by(number, divisor=Integer(2)):
...     return number%divisor == Integer(0)
>>> is_divisible_by(Integer(6),Integer(2))
True
>>> is_divisible_by(Integer(6))
True
>>> is_divisible_by(Integer(6), Integer(5))
False
```

Il est possible de spécifier un ou plusieurs des paramètres par leur nom lors de l'appel de la fonction ; dans ce cas, les paramètres nommés peuvent apparaître dans n'importe quel ordre :

```
sage: is_divisible_by(6, divisor=5)
False
sage: is_divisible_by(divisor=2, number=6)
True
```

```
>>> from sage.all import *
>>> is_divisible_by(Integer(6), divisor=Integer(5))
False
>>> is_divisible_by(divisor=Integer(2), number=Integer(6))
True
```

En Python, contrairement à de nombreux autres langages, les blocs de code ne sont pas délimités par des accolades ou des mots-clés de début et de fin de bloc. Au lieu de cela, la structure des blocs est donnée par l'indentation, qui doit être la même dans tout le bloc. Par exemple, le code suivant déclenche une erreur de syntaxe parce que l'instruction `return` n'est pas au même niveau d'indentation que les lignes précédentes.

```
sage: def even(n):
....:     v = []
....:     for i in range(3,n):
....:         if i % 2 == 0:
....:             v.append(i)
....:    return v
Syntax Error:
       return v
```

```
>>> from sage.all import *
>>> def even(n):
...     v = []
...     for i in range(Integer(3),n):
...         if i % Integer(2) == Integer(0):
...             v.append(i)
...    return v
Syntax Error:
       return v
```

Une fois l'indentation corrigée, l'exemple fonctionne :

```
sage: def even(n):
....:     v = []
....:     for i in range(3,n):
....:         if i % 2 == 0:
....:             v.append(i)
....:     return v
sage: even(10)
[4, 6, 8]
```

```
>>> from sage.all import *
>>> def even(n):
...     v = []
...     for i in range(Integer(3),n):
...         if i % Integer(2) == Integer(0):
...             v.append(i)
...     return v
>>> even(Integer(10))
[4, 6, 8]
```

Il n'y a pas besoin de placer des points-virgules en fin de ligne ; une instruction est en général terminée par un passage à la ligne. En revanche, il est possible de placer plusieurs instructions sur la même ligne en les séparant par des points-virgules :

```
sage: a = 5; b = a + 3; c = b^2; c
64
```

```
>>> from sage.all import *
>>> a = Integer(5); b = a + Integer(3); c = b**Integer(2); c
```

(suite sur la page suivante)

(suite de la page précédente)

```
64
```

Pour continuer une instruction sur la ligne suivante, placez une barre oblique inverse en fin de ligne :

```
sage: 2 + \
....:     3
5
```

```
>>> from sage.all import *
>>> Integer(2) +     Integer(3)
5
```

Pour compter en Sage, utilisez une boucle dont la variable d'itération parcourt une séquence d'entiers. Par exemple, la première ligne ci-dessous a exactement le même effet que `for(i=0; i<3; i++)` en C++ ou en Java :

```
sage: for i in range(3):
....:     print(i)
0
1
2
```

```
>>> from sage.all import *
>>> for i in range(Integer(3)):
...     print(i)
0
1
2
```

La première ligne ci-dessous correspond à `for(i=2;i<5;i++)`.

```
sage: for i in range(2,5):
....:     print(i)
2
3
4
```

```
>>> from sage.all import *
>>> for i in range(Integer(2),Integer(5)):
...     print(i)
2
3
4
```

Le troisième paramètre contrôle le pas de l'itération. Ainsi, ce qui suit est équivalent à `for(i=1;i<6;i+=2)`.

```
sage: for i in range(1,6,2):
....:     print(i)
1
3
5
```

```
>>> from sage.all import *
>>> for i in range(Integer(1),Integer(6),Integer(2)):
...     print(i)
1
3
5
```

Vous souhaiterez peut-être regrouper dans un joli tableau les résultats numériques que vous aurez calculés avec Sage. Une façon de faire commode utilise les chaînes de format. Ici, nous affichons une table des carrés et des cubes en trois colonnes, chacune d'une largeur de six caractères.

```
sage: for i in range(5):
....:     print('%6s %6s %6s' % (i, i^2, i^3))
     0      0      0
     1      1      1
     2      4      8
     3      9     27
     4     16     64
```

```
>>> from sage.all import *
>>> for i in range(Integer(5)):
...     print('%6s %6s %6s' % (i, i**Integer(2), i**Integer(3)))
     0      0      0
     1      1      1
     2      4      8
     3      9     27
     4     16     64
```

La structure de données de base de Sage est la liste, qui est — comme son nom l'indique — une liste d'objets arbitraires. Voici un exemple de liste :

```
sage: v = [1, "hello", 2/3, sin(x^3)]
sage: v
[1, 'hello', 2/3, sin(x^3)]
```

```
>>> from sage.all import *
>>> v = [Integer(1), "hello", Integer(2)/Integer(3), sin(x**Integer(3))]
>>> v
[1, 'hello', 2/3, sin(x^3)]
```

Comme dans de nombreux langages de programmation, les listes sont indexées à partir de 0.

```
sage: v[0]
1
sage: v[3]
sin(x^3)
```

```
>>> from sage.all import *
>>> v[Integer(0)]
1
>>> v[Integer(3)]
sin(x^3)
```

La fonction `len(v)` donne la longueur de `v`.... :`v.append(obj)` ajoute un nouvel objet à la fin de `v`; et `del v[i]` supprime l'élément d'indice `i` de `v`.

```
sage: len(v)
4
sage: v.append(1.5)
sage: v
[1, 'hello', 2/3, sin(x^3), 1.50000000000000]
sage: del v[1]
sage: v
[1, 2/3, sin(x^3), 1.50000000000000]
```

```
>>> from sage.all import *
>>> len(v)
4
>>> v.append(RealNumber('1.5'))
>>> v
[1, 'hello', 2/3, sin(x^3), 1.50000000000000]
>>> del v[Integer(1)]
>>> v
[1, 2/3, sin(x^3), 1.50000000000000]
```

Une autre structure de données importante est le dictionnaire (ou tableau associatif). Un dictionnaire fonctionne comme une liste, à ceci près que les indices peuvent être presque n'importe quels objets (les objets mutables sont interdits) :

```
sage: d = {'hi':-2,  3/8:pi,   e:pi}
sage: d['hi']
-2
sage: d[e]
pi
```

```
>>> from sage.all import *
>>> d = {'hi':-Integer(2),  Integer(3)/Integer(8):pi,   e:pi}
>>> d['hi']
-2
>>> d[e]
pi
```

Vous pouvez définir de nouveaux types de données en utilisant les classes. Encapsuler les objets mathématiques dans des classes représente une technique puissante qui peut vous aider à simplifier et organiser vos programmes Sage. Dans l'exemple suivant, nous définissons une classe qui représente la liste des entiers impairs strictement positifs jusqu'à n. Cette classe dérive du type interne `list`.

```
sage: class Evens(list):
....:     def __init__(self, n):
....:         self.n = n
....:         list.__init__(self, range(2, n+1, 2))
....:     def __repr__(self):
....:         return "Even positive numbers up to n."
```

```
>>> from sage.all import *
>>> class Evens(list):
...     def __init__(self, n):
```

(suite sur la page suivante)

(suite de la page précédente)

```
...         self.n = n
...         list.__init__(self, range(Integer(2), n+Integer(1), Integer(2)))
...     def __repr__(self):
...         return "Even positive numbers up to n."
```

La méthode `__init__` est appelée à la création de l'objet pour l'initialiser ; la méthode `__repr__` affiche l'objet. À la seconde ligne de la méthode `__init__`, nous appelons le constructeur de la classe `list`. Pour créer un objet de classe `Evens`, nous procédons ensuite comme suit :

```
sage: e = Evens(10)
sage: e
Even positive numbers up to n.
```

```
>>> from sage.all import *
>>> e = Evens(Integer(10))
>>> e
Even positive numbers up to n.
```

Notez que `e` s'affiche en utilisant la méthode `__repr__` que nous avons définie plus haut. Pour voir la liste de nombres sous-jacente, on utilise la fonction `list` :

```
sage: list(e)
[2, 4, 6, 8, 10]
```

```
>>> from sage.all import *
>>> list(e)
[2, 4, 6, 8, 10]
```

Il est aussi possible d'accéder à l'attribut `n`, ou encore d'utiliser `e` en tant que liste.

```
sage: e.n
10
sage: e[2]
6
```

```
>>> from sage.all import *
>>> e.n
10
>>> e[Integer(2)]
6
```

2.4 Algèbre de base et calcul infinitésimal

Sage peut accomplir divers calculs d'algèbre et d'analyse de base : par exemple, trouver les solutions d'équations, dériver, intégrer, calculer des transformées de Laplace. Voir la documentation Sage Constructions pour plus d'exemples.

2.4.1 Résolution d'équations

La fonction `solve` résout des équations. Pour l'utiliser, il convient de spécifier d'abord les variables. Les arguments de `solve` sont alors une équation (ou un système d'équations) suivie des variables à résoudre.

```
sage: x = var('x')
sage: solve(x^2 + 3*x + 2, x)
[x == -2, x == -1]
```

```
>>> from sage.all import *
>>> x = var('x')
>>> solve(x**Integer(2) + Integer(3)*x + Integer(2), x)
[x == -2, x == -1]
```

On peut résoudre une équation en une variable en fonction des autres :

```
sage: x, b, c = var('x b c')
sage: solve([x^2 + b*x + c == 0],x)
[x == -1/2*b - 1/2*sqrt(b^2 - 4*c), x == -1/2*b + 1/2*sqrt(b^2 - 4*c)]
```

```
>>> from sage.all import *
>>> x, b, c = var('x b c')
>>> solve([x**Integer(2) + b*x + c == Integer(0)],x)
[x == -1/2*b - 1/2*sqrt(b^2 - 4*c), x == -1/2*b + 1/2*sqrt(b^2 - 4*c)]
```

On peut également résoudre un système à plusieurs variables :

```
sage: x, y = var('x, y')
sage: solve([x+y==6, x-y==4], x, y)
[[x == 5, y == 1]]
```

```
>>> from sage.all import *
>>> x, y = var('x, y')
>>> solve([x+y==Integer(6), x-y==Integer(4)], x, y)
[[x == 5, y == 1]]
```

L'exemple suivant, qui utilise Sage pour la résolution d'un système d'équations non-linéaires, a été proposé par Jason Grout. D'abord, on résout le système de façon symbolique :

```
sage: var('x y p q')
(x, y, p, q)
sage: eq1 = p+q==9
sage: eq2 = q*y+p*x==-6
sage: eq3 = q*y^2+p*x^2==24
sage: solve([eq1,eq2,eq3,p==1],p,q,x,y)
[[p == 1, q == 8, x == -4/3*sqrt(10) - 2/3, y == 1/6*sqrt(10) - 2/3], [p == 1, q == 8,
↪ x == 4/3*sqrt(10) - 2/3, y == -1/6*sqrt(10) - 2/3]]
```

```
>>> from sage.all import *
>>> var('x y p q')
(x, y, p, q)
>>> eq1 = p+q==Integer(9)
>>> eq2 = q*y+p*x==-Integer(6)
>>> eq3 = q*y**Integer(2)+p*x**Integer(2)==Integer(24)
>>> solve([eq1,eq2,eq3,p==Integer(1)],p,q,x,y)
[[p == 1, q == 8, x == -4/3*sqrt(10) - 2/3, y == 1/6*sqrt(10) - 2/3], [p == 1, q == 8,
↪ x == 4/3*sqrt(10) - 2/3, y == -1/6*sqrt(10) - 2/3]]
```

Pour une résolution numérique, on peut utiliser à la place :

```
sage: solns = solve([eq1,eq2,eq3,p==1],p,q,x,y, solution_dict=True)
sage: [[s[p].n(30), s[q].n(30), s[x].n(30), s[y].n(30)] for s in solns]
[[1.0000000, 8.0000000, -4.8830369, -0.13962039],
 [1.0000000, 8.0000000, 3.5497035, -1.1937129]]
```

```
>>> from sage.all import *
>>> solns = solve([eq1,eq2,eq3,p==Integer(1)],p,q,x,y, solution_dict=True)
>>> [[s[p].n(Integer(30)), s[q].n(Integer(30)), s[x].n(Integer(30)), s[y].
↪n(Integer(30))] for s in solns]
[[1.0000000, 8.0000000, -4.8830369, -0.13962039],
 [1.0000000, 8.0000000, 3.5497035, -1.1937129]]
```

(La fonction `n` affiche une approximation numérique ; son argument indique le nombre de bits de précision.)

2.4.2 Dérivation, intégration, etc.

Sage est capable de dériver et d'intégrer de nombreuses fonctions. Par exemple, pour dériver $\sin(u)$ par rapport à u, on procède comme suit :

```
sage: u = var('u')
sage: diff(sin(u), u)
cos(u)
```

```
>>> from sage.all import *
>>> u = var('u')
>>> diff(sin(u), u)
cos(u)
```

Pour calculer la dérivée quatrième de $\sin(x^2)$:

```
sage: diff(sin(x^2), x, 4)
16*x^4*sin(x^2) - 48*x^2*cos(x^2) - 12*sin(x^2)
```

```
>>> from sage.all import *
>>> diff(sin(x**Integer(2)), x, Integer(4))
16*x^4*sin(x^2) - 48*x^2*cos(x^2) - 12*sin(x^2)
```

Pour calculer la dérivée partielle de $x^2 + 17y^2$ par rapport à x et y respectivement :

```
sage: x, y = var('x,y')
sage: f = x^2 + 17*y^2
sage: f.diff(x)
2*x
sage: f.diff(y)
34*y
```

```
>>> from sage.all import *
>>> x, y = var('x,y')
>>> f = x**Integer(2) + Integer(17)*y**Integer(2)
>>> f.diff(x)
2*x
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> f.diff(y)
34*y
```

Passons aux primitives et intégrales. Pour calculer $\int x \sin(x^2)\, dx$ et $\int_0^1 \frac{x}{x^2+1}\, dx$

```
sage: integral(x*sin(x^2), x)
-1/2*cos(x^2)
sage: integral(x/(x^2+1), x, 0, 1)
1/2*log(2)
```

```
>>> from sage.all import *
>>> integral(x*sin(x**Integer(2)), x)
-1/2*cos(x^2)
>>> integral(x/(x**Integer(2)+Integer(1)), x, Integer(0), Integer(1))
1/2*log(2)
```

Pour calculer la décomposition en éléments simples de $\frac{1}{x^2-1}$:

```
sage: f = 1/((1+x)*(x-1))
sage: f.partial_fraction(x)
-1/2/(x + 1) + 1/2/(x - 1)
```

```
>>> from sage.all import *
>>> f = Integer(1)/((Integer(1)+x)*(x-Integer(1)))
>>> f.partial_fraction(x)
-1/2/(x + 1) + 1/2/(x - 1)
```

2.4.3 Résolution des équations différentielles

On peut utiliser Sage pour étudier les équations différentielles ordinaires. Pour résoudre l'équation $x' + x - 1 = 0$:

```
sage: t = var('t')     # on définit une variable t
sage: function('x')(t)    # on déclare x fonction de cette variable
x(t)
sage: DE = lambda y: diff(y,t) + y - 1
sage: desolve(DE(x(t)), [x(t),t])
(_C + e^t)*e^(-t)
```

```
>>> from sage.all import *
>>> t = var('t')     # on définit une variable t
>>> function('x')(t)    # on déclare x fonction de cette variable
x(t)
>>> DE = lambda y: diff(y,t) + y - Integer(1)
>>> desolve(DE(x(t)), [x(t),t])
(_C + e^t)*e^(-t)
```

Ceci utilise l'interface de Sage vers Maxima [Max], aussi il se peut que la sortie diffère un peu des sorties habituelles de Sage. Dans notre cas, le résultat indique que la solution générale à l'équation différentielle est $x(t) = e^{-t}(e^t + C)$.

Il est aussi possible de calculer des transformées de Laplace. La transformée de Laplace de $t^2 e^t - \sin(t)$ s'obtient comme suit :

```
sage: s = var("s")
sage: t = var("t")
sage: f = t^2*exp(t) - sin(t)
sage: f.laplace(t,s).simplify_rational()
-(s^3 - 5*s^2 + 3*s - 3)/(s^5 - 3*s^4 + 4*s^3 - 4*s^2 + 3*s - 1)
```

```
>>> from sage.all import *
>>> s = var("s")
>>> t = var("t")
>>> f = t**Integer(2)*exp(t) - sin(t)
>>> f.laplace(t,s).simplify_rational()
-(s^3 - 5*s^2 + 3*s - 3)/(s^5 - 3*s^4 + 4*s^3 - 4*s^2 + 3*s - 1)
```

Voici un exemple plus élaboré. L'élongation à partir du point d'équilibre de ressorts couplés attachés à gauche à un mur

```
|------\/\/\/\/\---|masse1|----\/\/\/\/\/----|masse2|
        ressort1                 ressort2
```

est modélisée par le système d'équations différentielles d'ordre 2

$$m_1 x_1'' + (k_1 + k_2)x_1 - k_2 x_2 = 0 m_2 x_2'' + k_2(x_2 - x_1) = 0,$$

où m_i est la masse de l'objet *i*, x_i est l'élongation à partir du point d'équilibre de la masse *i*, et k_i est la constante de raideur du ressort *i*.

Exemple : Utiliser Sage pour résoudre le problème ci-dessus avec $m_1 = 2$, $m_2 = 1$, $k_1 = 4$, $k_2 = 2$, $x_1(0) = 3$, $x_1'(0) = 0$, $x_2(0) = 3$, $x_2'(0) = 0$.

Solution : Considérons la transformée de Laplace de la première équation (avec les notations $x = x_1$, $y = x_2$) :

```
sage: de1 = maxima("2*diff(x(t),t, 2) + 6*x(t) - 2*y(t)")
sage: lde1 = de1.laplace("t","s"); lde1.sage()
2*s^2*laplace(x(t), t, s) - 2*s*x(0) + 6*laplace(x(t), t, s) - 2*laplace(y(t), t, s) -
↪ 2*D[0](x)(0)
```

```
>>> from sage.all import *
>>> de1 = maxima("2*diff(x(t),t, 2) + 6*x(t) - 2*y(t)")
>>> lde1 = de1.laplace("t","s"); lde1.sage()
2*s^2*laplace(x(t), t, s) - 2*s*x(0) + 6*laplace(x(t), t, s) - 2*laplace(y(t), t, s) -
↪ 2*D[0](x)(0)
```

La réponse n'est pas très lisible, mais elle signifie que

$$-2x'(0) + 2s^2 \cdot X(s) - 2sx(0) - 2Y(s) + 6X(s) = 0$$

(où la transformée de Laplace d'une fonction notée par une lettre minuscule telle que $x(t)$ est désignée par la majuscule correspondante $X(s)$). Considérons la transformée de Laplace de la seconde équation :

```
sage: t,s = SR.var('t,s')
sage: x = function('x')
sage: y = function('y')
sage: f = 2*x(t).diff(t,2) + 6*x(t) - 2*y(t)
sage: f.laplace(t,s)
2*s^2*laplace(x(t), t, s) - 2*s*x(0) + 6*laplace(x(t), t, s) - 2*laplace(y(t), t, s) -
↪ 2*D[0](x)(0)
```

```
>>> from sage.all import *
>>> t,s = SR.var('t,s')
>>> x = function('x')
>>> y = function('y')
>>> f = Integer(2)*x(t).diff(t,Integer(2)) + Integer(6)*x(t) - Integer(2)*y(t)
>>> f.laplace(t,s)
2*s^2*laplace(x(t), t, s) - 2*s*x(0) + 6*laplace(x(t), t, s) - 2*laplace(y(t), t, s) -
↪ 2*D[0](x)(0)
```

Ceci signifie

$$-Y'(0) + s^2Y(s) + 2Y(s) - 2X(s) - sy(0) = 0.$$

Injectons les conditions initiales pour $x(0)$, $x'(0)$, $y(0)$ et $y'(0)$ et résolvons les deux équations qui en résultent :

```
sage: var('s X Y')
(s, X, Y)
sage: eqns = [(2*s^2+6)*X-2*Y == 6*s, -2*X +(s^2+2)*Y == 3*s]
sage: solve(eqns, X,Y)
[[X == 3*(s^3 + 3*s)/(s^4 + 5*s^2 + 4),
  Y == 3*(s^3 + 5*s)/(s^4 + 5*s^2 + 4)]]
```

```
>>> from sage.all import *
>>> var('s X Y')
(s, X, Y)
>>> eqns = [(Integer(2)*s**Integer(2)+Integer(6))*X-Integer(2)*Y == Integer(6)*s, -
↪Integer(2)*X +(s**Integer(2)+Integer(2))*Y == Integer(3)*s]
>>> solve(eqns, X,Y)
[[X == 3*(s^3 + 3*s)/(s^4 + 5*s^2 + 4),
  Y == 3*(s^3 + 5*s)/(s^4 + 5*s^2 + 4)]]
```

À présent, prenons la transformée de Laplace inverse pour obtenir la réponse :

```
sage: var('s t')
(s, t)
sage: inverse_laplace((3*s^3 + 9*s)/(s^4 + 5*s^2 + 4),s,t)
cos(2*t) + 2*cos(t)
sage: inverse_laplace((3*s^3 + 15*s)/(s^4 + 5*s^2 + 4),s,t)
-cos(2*t) + 4*cos(t)
```

```
>>> from sage.all import *
>>> var('s t')
(s, t)
>>> inverse_laplace((Integer(3)*s**Integer(3) + Integer(9)*s)/(s**Integer(4) +␣
↪Integer(5)*s**Integer(2) + Integer(4)),s,t)
cos(2*t) + 2*cos(t)
>>> inverse_laplace((Integer(3)*s**Integer(3) + Integer(15)*s)/(s**Integer(4) +␣
↪Integer(5)*s**Integer(2) + Integer(4)),s,t)
-cos(2*t) + 4*cos(t)
```

Par conséquent, la solution est

$$x_1(t) = \cos(2t) + 2\cos(t), \quad x_2(t) = 4\cos(t) - \cos(2t).$$

On peut en tracer le graphe paramétrique en utilisant

```
sage: t = var('t')
sage: P = parametric_plot((cos(2*t) + 2*cos(t), 4*cos(t) - cos(2*t) ),
....:     (t, 0, 2*pi), rgbcolor=hue(0.9))
sage: show(P)
```

```
>>> from sage.all import *
>>> t = var('t')
>>> P = parametric_plot((cos(Integer(2)*t) + Integer(2)*cos(t), Integer(4)*cos(t) -␣
↪cos(Integer(2)*t) ),
...     (t, Integer(0), Integer(2)*pi), rgbcolor=hue(RealNumber('0.9')))
>>> show(P)
```

Les coordonnées individuelles peuvent être tracées en utilisant

```
sage: t = var('t')
sage: p1 = plot(cos(2*t) + 2*cos(t), (t, 0, 2*pi), rgbcolor=hue(0.3))
sage: p2 = plot(4*cos(t) - cos(2*t), (t, 0, 2*pi), rgbcolor=hue(0.6))
sage: show(p1 + p2)
```

```
>>> from sage.all import *
>>> t = var('t')
>>> p1 = plot(cos(Integer(2)*t) + Integer(2)*cos(t), (t, Integer(0), Integer(2)*pi),␣
↪rgbcolor=hue(RealNumber('0.3')))
>>> p2 = plot(Integer(4)*cos(t) - cos(Integer(2)*t), (t, Integer(0), Integer(2)*pi),␣
↪rgbcolor=hue(RealNumber('0.6')))
>>> show(p1 + p2)
```

Les fonctions de tracé de graphes sont décrites dans la section *Graphiques* de ce tutoriel. On pourra aussi consulter [NagleEtAl2004], §5.5 pour plus d'informations sur les équations différentielles.

2.4.4 Méthode d'Euler pour les systèmes d'équations différentielles

Dans l'exemple suivant, nous illustrons la méthode d'Euler pour des équations différentielles ordinaires d'ordre un et deux. Rappelons d'abord le principe de la méthode pour les équations du premier ordre. Etant donné un problème donné avec une valeur initiale sous la forme

$$y' = f(x, y), \quad y(a) = c,$$

nous cherchons une valeur approchée de la solution au point $x = b$ avec $b > a$.

Rappelons que par définition de la dérivée

$$y'(x) \approx \frac{y(x+h) - y(x)}{h},$$

où $h > 0$ est fixé et petit. Ceci, combiné à l'équation différentielle, donne $f(x, y(x)) \approx \frac{y(x+h)-y(x)}{h}$. Aussi $y(x+h)$ s'écrit :

$$y(x+h) \approx y(x) + h \cdot f(x, y(x)).$$

Si nous notons $h \cdot f(x, y(x))$ le « terme de correction » (faute d'un terme plus approprié), et si nous appelons $y(x)$ « l'ancienne valeur de y » et $y(x+h)$ la « nouvelle valeur de y », cette approximation se réécrit

$$y_{nouveau} \approx y_{ancien} + h \cdot f(x, y_{ancien}).$$

Divisions l'intervalle entre a et b en n pas, si bien que $h = \frac{b-a}{n}$. Nous pouvons alors remplir un tableau avec les informations utilisées dans la méthode.

x	y	$h \cdot f(x,y)$
a	c	$h \cdot f(a,c)$
$a+h$	$c + h \cdot f(a,c)$	$\ldots$
$a+2h$	$\ldots$	
$\ldots$		
$b = a+nh$	???	$\ldots$

Le but est est de remplir tous les trous du tableau, ligne après ligne, jusqu'à atteindre le coefficient « ??? », qui est l'approximation de $y(b)$ au sens de la méthode d'Euler.

L'idée est la même pour les systèmes d'équations différentielles.

Exemple : Rechercher une approximation numérique de $z(t)$ en $t = 1$ en utilisant 4 étapes de la méthode d'Euler, où $z'' + tz' + z = 0$, $z(0) = 1$, $z'(0) = 0$.

Il nous faut réduire l'équation différentielle d'ordre 2 à un système de deux équations différentielles d'ordre 1 (en posant $x = z$, $y = z'$) et appliquer la méthode d'Euler :

```
sage: t,x,y = PolynomialRing(RealField(10),3,"txy").gens()
sage: f = y; g = -x - y * t
sage: eulers_method_2x2(f,g, 0, 1, 0, 1/4, 1)
      t                x            h*f(t,x,y)                y        h*g(t,x,y)
      0                1                  0.00                0           -0.25
    1/4              1.0                -0.062            -0.25           -0.23
    1/2             0.94                 -0.12            -0.48           -0.17
    3/4             0.82                 -0.16            -0.66          -0.081
      1             0.65                 -0.18            -0.74           0.022
```

```
>>> from sage.all import *
>>> t,x,y = PolynomialRing(RealField(Integer(10)),Integer(3),"txy").gens()
>>> f = y; g = -x - y * t
>>> eulers_method_2x2(f,g, Integer(0), Integer(1), Integer(0), Integer(1)/Integer(4),␣
↪Integer(1))
      t                x            h*f(t,x,y)                y        h*g(t,x,y)
      0                1                  0.00                0           -0.25
    1/4              1.0                -0.062            -0.25           -0.23
    1/2             0.94                 -0.12            -0.48           -0.17
    3/4             0.82                 -0.16            -0.66          -0.081
      1             0.65                 -0.18            -0.74           0.022
```

On en déduit $z(1) \approx 0.65$.

On peut également tracer le graphe des points (x, y) pour obtenir une image approchée de la courbe. La fonction `eulers_method_2x2_plot` réalise cela ; pour l'utiliser, il faut définir les fonctions f et g qui prennent un argument à trois coordonnées : (t, x, y).

```
sage: f = lambda z: z[2]         # f(t,x,y) = y
sage: g = lambda z: -sin(z[1])   # g(t,x,y) = -sin(x)
sage: P = eulers_method_2x2_plot(f,g, 0.0, 0.75, 0.0, 0.1, 1.0)
```

```
>>> from sage.all import *
>>> f = lambda z: z[Integer(2)]         # f(t,x,y) = y
>>> g = lambda z: -sin(z[Integer(1)])   # g(t,x,y) = -sin(x)
>>> P = eulers_method_2x2_plot(f,g, RealNumber('0.0'), RealNumber('0.75'), RealNumber(
↪'0.0'), RealNumber('0.1'), RealNumber('1.0'))
```

Arrivé à ce point, `P` conserve en mémoire deux graphiques : `P[0]`, le graphe de x en fonction de t, et `P[1]`, le graphique de y par rapport à t. On peut tracer les deux graphiques simultanément par :

```
sage: show(P[0] + P[1])
```

```
>>> from sage.all import *
>>> show(P[Integer(0)] + P[Integer(1)])
```

(Pour plus d'information sur le tracé de graphiques, voir *Graphiques*.)

2.4.5 Fonctions spéciales

Plusieurs familles de polynômes orthogonaux et fonctions spéciales sont implémentées via PARI [GAP] et Maxima [Max]. Ces fonctions sont documentées dans les sections correspondantes (*Orthogonal polynomials* et *Special functions*, respectively) du manuel de référence de Sage (*Sage reference manual*).

```
sage: x = polygen(QQ, 'x')
sage: chebyshev_U(2,x)
4*x^2 - 1
sage: bessel_I(1,1).n(250)
0.56515910399248502720769602760986330732889962162109200948029448947925564096
sage: bessel_I(1,1).n()
0.565159103992485
sage: bessel_I(2,1.1).n()
0.167089499251049
```

```
>>> from sage.all import *
>>> x = polygen(QQ, 'x')
>>> chebyshev_U(Integer(2),x)
4*x^2 - 1
>>> bessel_I(Integer(1),Integer(1)).n(Integer(250))
0.56515910399248502720769602760986330732889962162109200948029448947925564096
>>> bessel_I(Integer(1),Integer(1)).n()
0.565159103992485
>>> bessel_I(Integer(2),RealNumber('1.1')).n()
0.167089499251049
```

Pour l'instant, ces fonctions n'ont été adaptées à Sage que pour une utilisation numérique. Pour faire du calcul formel, il faut utiliser l'interface Maxima directement, comme le présente l'exemple suivant :

```
sage: maxima.eval("f:bessel_y(v, w)")
'bessel_y(v,w)'
sage: maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
```

```
>>> from sage.all import *
>>> maxima.eval("f:bessel_y(v, w)")
```

(suite sur la page suivante)

(suite de la page précédente)

```
'bessel_y(v,w)'
>>> maxima.eval("diff(f,w)")
'(bessel_y(v-1,w)-bessel_y(v+1,w))/2'
```

2.5 Graphiques

Sage peut produire des graphiques en deux ou trois dimensions.

2.5.1 Graphiques en deux dimensions

En deux dimensions, Sage est capable de tracer des cercles, des droites, des polygones, des graphes de fonctions en coordonnées cartésiennes, des graphes en coordonnées polaires, des lignes de niveau et des représentations de champs de vecteurs. Nous présentons quelques exemples de ces objets ici. Pour plus d'exemples de graphiques avec Sage, on consultera *Résolution des équations différentielles*, *Maxima* et aussi la documentation Sage Constructions

La commande suivante produit un cercle jaune de rayon 1 centré à l'origine :

```
sage: circle((0,0), 1, rgbcolor=(1,1,0))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> circle((Integer(0),Integer(0)), Integer(1), rgbcolor=(Integer(1),Integer(1),
↪Integer(0)))
Graphics object consisting of 1 graphics primitive
```

Il est également possible de produire un disque plein :

```
sage: circle((0,0), 1, rgbcolor=(1,1,0), fill=True)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> circle((Integer(0),Integer(0)), Integer(1), rgbcolor=(Integer(1),Integer(1),
↪Integer(0)), fill=True)
Graphics object consisting of 1 graphics primitive
```

Il est aussi possible de créer un cercle en l'affectant à une variable ; ceci ne provoque pas son affichage.

```
sage: c = circle((0,0), 1, rgbcolor=(1,1,0))
```

```
>>> from sage.all import *
>>> c = circle((Integer(0),Integer(0)), Integer(1), rgbcolor=(Integer(1),Integer(1),
↪Integer(0)))
```

Pour l'afficher, on utilise `c.show()` ou `show(c)`, comme suit :

```
sage: c.show()
```

```
>>> from sage.all import *
>>> c.show()
```

Alternativement, l'évaluation de `c.save('filename.png')` enregistre le graphique dans le fichier spécifié.

Toutefois, ces « cercles » ressemblent plus à des ellipses qu'à des cercles, puisque les axes possèdent des échelles différentes. On peut arranger ceci :

```
sage: c.show(aspect_ratio=1)
```

```
>>> from sage.all import *
>>> c.show(aspect_ratio=Integer(1))
```

La commande `show(c, aspect_ratio=1)` produit le même résultat. On peut enregistrer l'image avec cette option par la commande `c.save('filename.png', aspect_ratio=1)`.

Il est très facile de tracer le graphique de fonctions de base :

```
sage: plot(cos, (-5,5))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> plot(cos, (-Integer(5),Integer(5)))
Graphics object consisting of 1 graphics primitive
```

En spécifiant un nom de variable, on peut aussi créer des graphes paramétriques :

```
sage: x = var('x')
sage: parametric_plot((cos(x),sin(x)^3),(x,0,2*pi),rgbcolor=hue(0.6))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> x = var('x')
>>> parametric_plot((cos(x),sin(x)**Integer(3)),(x,Integer(0),Integer(2)*pi),
↪rgbcolor=hue(RealNumber('0.6')))
Graphics object consisting of 1 graphics primitive
```

Différents graphiques peuvent se combiner sur une même image :

```
sage: x = var('x')
sage: p1 = parametric_plot((cos(x),sin(x)),(x,0,2*pi),rgbcolor=hue(0.2))
sage: p2 = parametric_plot((cos(x),sin(x)^2),(x,0,2*pi),rgbcolor=hue(0.4))
sage: p3 = parametric_plot((cos(x),sin(x)^3),(x,0,2*pi),rgbcolor=hue(0.6))
sage: show(p1+p2+p3, axes=false)
```

```
>>> from sage.all import *
>>> x = var('x')
>>> p1 = parametric_plot((cos(x),sin(x)),(x,Integer(0),Integer(2)*pi),
↪rgbcolor=hue(RealNumber('0.2')))
>>> p2 = parametric_plot((cos(x),sin(x)**Integer(2)),(x,Integer(0),Integer(2)*pi),
↪rgbcolor=hue(RealNumber('0.4')))
>>> p3 = parametric_plot((cos(x),sin(x)**Integer(3)),(x,Integer(0),Integer(2)*pi),
↪rgbcolor=hue(RealNumber('0.6')))
>>> show(p1+p2+p3, axes=false)
```

Une manière commode de tracer des formes pleines est de préparer une liste de points (`L` dans l'exemple ci-dessous) puis d'utiliser la commande `polygon` pour tracer la forme pleine dont le bord est formé par ces points. Par example, voici un deltoïde vert :

```
sage: L = [[-1+cos(pi*i/100)*(1+cos(pi*i/100)),
....:      2*sin(pi*i/100)*(1-cos(pi*i/100))] for i in range(200)]
sage: polygon(L, rgbcolor=(1/8,3/4,1/2))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> L = [[-Integer(1)+cos(pi*i/Integer(100))*(Integer(1)+cos(pi*i/Integer(100))),
...      Integer(2)*sin(pi*i/Integer(100))*(Integer(1)-cos(pi*i/Integer(100)))] for i␣
↪in range(Integer(200))]
>>> polygon(L, rgbcolor=(Integer(1)/Integer(8),Integer(3)/Integer(4),Integer(1)/
↪Integer(2)))
Graphics object consisting of 1 graphics primitive
```

Pour visualiser le graphique en masquant les axes, tapez `show(p, axes=false)`.

On peut ajouter un texte à un graphique :

```
sage: L = [[6*cos(pi*i/100)+5*cos((6/2)*pi*i/100),
....:      6*sin(pi*i/100)-5*sin((6/2)*pi*i/100)] for i in range(200)]
sage: p = polygon(L, rgbcolor=(1/8,1/4,1/2))
sage: t = text("hypotrochoid", (5,4), rgbcolor=(1,0,0))
sage: show(p+t)
```

```
>>> from sage.all import *
>>> L = [[Integer(6)*cos(pi*i/Integer(100))+Integer(5)*cos((Integer(6)/
↪Integer(2))*pi*i/Integer(100)),
...      Integer(6)*sin(pi*i/Integer(100))-Integer(5)*sin((Integer(6)/Integer(2))*pi*i/
↪Integer(100))] for i in range(Integer(200))]
>>> p = polygon(L, rgbcolor=(Integer(1)/Integer(8),Integer(1)/Integer(4),Integer(1)/
↪Integer(2)))
>>> t = text("hypotrochoid", (Integer(5),Integer(4)), rgbcolor=(Integer(1),Integer(0),
↪Integer(0)))
>>> show(p+t)
```

En cours d'analyse, les professeurs font souvent le dessin suivant au tableau : non pas une mais plusieurs branches de la fonction arcsin, autrement dit, le graphe d'équation $y = \sin(x)$ pour x entre -2π et 2π, renversé par symétrie par rapport à la première bissectrice des axes. La commande Sage suivante réalise cela :

```
sage: v = [(sin(x),x) for x in srange(-2*float(pi),2*float(pi),0.1)]
sage: line(v)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> v = [(sin(x),x) for x in srange(-Integer(2)*float(pi),Integer(2)*float(pi),
↪RealNumber('0.1'))]
>>> line(v)
Graphics object consisting of 1 graphics primitive
```

Comme les valeurs prises par la fonction tangente ne sont pas bornées, pour utiliser la même astuce pour représenter la fonction arctangente, il faut préciser les bornes de la coordonnée x :

```
sage: v = [(tan(x),x) for x in srange(-2*float(pi),2*float(pi),0.01)]
sage: show(line(v), xmin=-20, xmax=20)
```

```
>>> from sage.all import *
>>> v = [(tan(x),x) for x in srange(-Integer(2)*float(pi),Integer(2)*float(pi),
↪RealNumber('0.01'))]
>>> show(line(v), xmin=-Integer(20), xmax=Integer(20))
```

Sage sait aussi tracer des graphiques en coordonnées polaires, des lignes de niveau et (pour certains types de fonctions) des champs de vecteurs. Voici un exemple de lignes de niveau :

```
sage: f = lambda x,y: cos(x*y)
sage: contour_plot(f, (-4, 4), (-4, 4))
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> f = lambda x,y: cos(x*y)
>>> contour_plot(f, (-Integer(4), Integer(4)), (-Integer(4), Integer(4)))
Graphics object consisting of 1 graphics primitive
```

2.5.2 Graphiques en trois dimensions

Sage produit des graphes en trois dimensions en utilisant le package open source appelé [ThreeJS]. En voici quelques exemples :

Le parapluie de Whitney tracé en jaune http://en.wikipedia.org/wiki/Whitney_umbrella :

```
sage: u, v = var('u,v')
sage: fx = u*v
sage: fy = u
sage: fz = v^2
sage: parametric_plot3d([fx, fy, fz], (u, -1, 1), (v, -1, 1),
....:    frame=False, color="yellow")
Graphics3d Object
```

```
>>> from sage.all import *
>>> u, v = var('u,v')
>>> fx = u*v
>>> fy = u
>>> fz = v**Integer(2)
>>> parametric_plot3d([fx, fy, fz], (u, -Integer(1), Integer(1)), (v, -Integer(1),␣
↪Integer(1)),
...    frame=False, color="yellow")
Graphics3d Object
```

Une fois évaluée la commande `parametric_plot3d`, qui affiche le graphique, il est possible de cliquer et de le tirer pour faire pivoter la figure.

Le bonnet croisé (cf. http://en.wikipedia.org/wiki/Cross-cap ou http://www.mathcurve.com/surfaces/bonnetcroise/bonnetcroise.shtml) :

```
sage: u, v = var('u,v')
sage: fx = (1+cos(v))*cos(u)
sage: fy = (1+cos(v))*sin(u)
sage: fz = -tanh((2/3)*(u-pi))*sin(v)
sage: parametric_plot3d([fx, fy, fz], (u, 0, 2*pi), (v, 0, 2*pi),
```

(suite sur la page suivante)

(suite de la page précédente)

```
....:    frame=False, color="red")
Graphics3d Object
```

```
>>> from sage.all import *
>>> u, v = var('u,v')
>>> fx = (Integer(1)+cos(v))*cos(u)
>>> fy = (Integer(1)+cos(v))*sin(u)
>>> fz = -tanh((Integer(2)/Integer(3))*(u-pi))*sin(v)
>>> parametric_plot3d([fx, fy, fz], (u, Integer(0), Integer(2)*pi), (v, Integer(0),␣
↪Integer(2)*pi),
...    frame=False, color="red")
Graphics3d Object
```

Un tore tordu :

```
sage: u, v = var('u,v')
sage: fx = (3+sin(v)+cos(u))*cos(2*v)
sage: fy = (3+sin(v)+cos(u))*sin(2*v)
sage: fz = sin(u)+2*cos(v)
sage: parametric_plot3d([fx, fy, fz], (u, 0, 2*pi), (v, 0, 2*pi),
....:    frame=False, color="red")
Graphics3d Object
```

```
>>> from sage.all import *
>>> u, v = var('u,v')
>>> fx = (Integer(3)+sin(v)+cos(u))*cos(Integer(2)*v)
>>> fy = (Integer(3)+sin(v)+cos(u))*sin(Integer(2)*v)
>>> fz = sin(u)+Integer(2)*cos(v)
>>> parametric_plot3d([fx, fy, fz], (u, Integer(0), Integer(2)*pi), (v, Integer(0),␣
↪Integer(2)*pi),
...    frame=False, color="red")
Graphics3d Object
```

2.6 Problèmes fréquents concernant les fonctions

La définition de fonctions, par exemple pour calculer leurs dérivées ou tracer leurs courbes représentatives, donne lieu à un certain nombre de confusions. Le but de cette section est de clarifier quelques points à l'origine de ces confusions.

Il y a plusieurs façons de définir un objet que l'on peut légitimement appeler « fonction ».

1. Définir une fonction Python, comme expliqué dans la section *Fonctions, indentation et itération*. Les fonctions Python peuvent être utilisées pour tracer des courbes, mais pas dérivées ou intégrées symboliquement :

```
sage: def f(z): return z^2
sage: type(f)
<... 'function'>
sage: f(3)
9
sage: plot(f, 0, 2)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> def f(z): return z**Integer(2)
>>> type(f)
<... 'function'>
>>> f(Integer(3))
9
>>> plot(f, Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
```

Remarquez la syntaxe de la dernière ligne. Écrire plutôt `plot(f(z), 0, 2)` provoquerait une erreur : en effet, le `z` qui apparaît dans la définition de `f` est une variable muette qui n'a pas de sens en dehors de la définition. Un simple `f(z)` déclenche la même erreur. En l'occurrence, faire de `z` une variable symbolique comme dans l'exemple ci-dessous fonctionne, mais cette façon de faire soulève d'autres problèmes (voir le point 4 ci-dessous), et il vaut mieux s'abstenir de l'utiliser.

```
sage: var('z')   # on définit z comme variable symbolique
z
sage: f(z)
z^2
sage: plot(f(z), 0, 2)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> var('z')   # on définit z comme variable symbolique
z
>>> f(z)
z^2
>>> plot(f(z), Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
```

L'appel de fonction `f(z)` renvoie ici l'expression symbolique `z^2`, qui est alors utilisée par la fonction `plot`.

2. Définir une expression symbolique fonctionnelle (« appelable »). Une telle expression représente une fonction dont on peut tracer le graphe, et que l'on peut aussi dériver ou intégrer symboliquement

```
sage: g(x) = x^2
sage: g        # g envoie x sur x^2
x |--> x^2
sage: g(3)
9
sage: Dg = g.derivative(); Dg
x |--> 2*x
sage: Dg(3)
6
sage: type(g)
<class 'sage.symbolic.expression.Expression'>
sage: plot(g, 0, 2)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); g = symbolic_expression(x**Integer(2)).function(x)
>>> g        # g envoie x sur x^2
x |--> x^2
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> g(Integer(3))
9
>>> Dg = g.derivative(); Dg
x |--> 2*x
>>> Dg(Integer(3))
6
>>> type(g)
<class 'sage.symbolic.expression.Expression'>
>>> plot(g, Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
```

Notez que, si `g` est une expression symbolique fonctionnelle (`x |--> x^2`), l'objet `g(x)` (`x^2`) est d'une nature un peu différente. Les expressions comme `g(x)` peuvent aussi être tracées, dérivées, intégrées, etc., avec cependant quelques difficultés illustrées dans le point 5 ci-dessous.

```
sage: g(x)
x^2
sage: type(g(x))
<class 'sage.symbolic.expression.Expression'>
sage: g(x).derivative()
2*x
sage: plot(g(x), 0, 2)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> g(x)
x^2
>>> type(g(x))
<class 'sage.symbolic.expression.Expression'>
>>> g(x).derivative()
2*x
>>> plot(g(x), Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
```

3. Utiliser une fonction usuelle prédéfinie de Sage. Celles-ci peuvent servir à tracer des courbes, et, indirectement, être dérivées ou intégrées

```
sage: type(sin)
<class 'sage.functions.trig.Function_sin'>
sage: plot(sin, 0, 2)
Graphics object consisting of 1 graphics primitive
sage: type(sin(x))
<class 'sage.symbolic.expression.Expression'>
sage: plot(sin(x), 0, 2)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> type(sin)
<class 'sage.functions.trig.Function_sin'>
>>> plot(sin, Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
>>> type(sin(x))
```

(suite sur la page suivante)

(suite de la page précédente)

```
<class 'sage.symbolic.expression.Expression'>
>>> plot(sin(x), Integer(0), Integer(2))
Graphics object consisting of 1 graphics primitive
```

Il n'est pas possible de dériver la fonction `sin` tout court pour obtenir `cos`

```
sage: f = sin
sage: f.derivative()
Traceback (most recent call last):
...
AttributeError: ...
```

```
>>> from sage.all import *
>>> f = sin
>>> f.derivative()
Traceback (most recent call last):
...
AttributeError: ...
```

Une possibilité est de remplacer `f = sin` par `f = sin(x)`, mais il est généralement préférable de définir une expression symbolique fonctionnelle `f(x) = sin(x)`

```
sage: S(x) = sin(x)
sage: S.derivative()
x |--> cos(x)
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); S = symbolic_expression(sin(x)).function(x)
>>> S.derivative()
x |--> cos(x)
```

Examinons maintenant quelques problèmes fréquents.

4. Évaluation accidentelle

```
sage: def h(x):
....:     if x < 2:
....:         return 0
....:     return x - 2
```

```
>>> from sage.all import *
>>> def h(x):
...     if x < Integer(2):
...         return Integer(0)
...     return x - Integer(2)
```

Problème : `plot(h(x), 0, 4)` trace la droite $y = x - 2$, et non pas la fonction affine par morceaux définie par `h`. Pourquoi ? Lors de l'exécution, `plot(h(x), 0, 4)` évalue d'abord `h(x)` : la fonction Python `h` est appelée avec le paramètre `x`, et la condition `x < 2` est donc évaluée.

```
sage: type(x < 2)
<class 'sage.symbolic.expression.Expression'>
```

```
>>> from sage.all import *
>>> type(x < Integer(2))
<class 'sage.symbolic.expression.Expression'>
```

Or, l'évaluation d'une inégalité symbolique renvoie `False` quand la condition n'est pas clairement vraie. Ainsi, `h(x)` s'évalue en `x - 2`, et c'est cette expression-là qui est finalement tracée.

Solution : Il ne faut pas utiliser `plot(h(x), 0, 4)`, mais plutôt

```
sage: def h(x):
....:     if x < 2:
....:         return 0
....:     return x - 2
sage: plot(h, 0, 4)
Graphics object consisting of 1 graphics primitive
```

```
>>> from sage.all import *
>>> def h(x):
...     if x < Integer(2):
...         return Integer(0)
...     return x - Integer(2)
>>> plot(h, Integer(0), Integer(4))
Graphics object consisting of 1 graphics primitive
```

5. Constante plutôt que fonction

```
sage: f = x
sage: g = f.derivative()
sage: g
1
```

```
>>> from sage.all import *
>>> f = x
>>> g = f.derivative()
>>> g
1
```

Problème : `g(3)` déclenche une erreur avec le message « ValueError : the number of arguments must be less than or equal to 0 ».

```
sage: type(f)
<class 'sage.symbolic.expression.Expression'>
sage: type(g)
<class 'sage.symbolic.expression.Expression'>
```

```
>>> from sage.all import *
>>> type(f)
<class 'sage.symbolic.expression.Expression'>
>>> type(g)
<class 'sage.symbolic.expression.Expression'>
```

En effet, `g` n'est pas une fonction, mais une constante, sans variable en laquelle on peut l'évaluer.

Solution : il y a plusieurs possibilités.

— Définir `f` comme une expression symbolique fonctionnelle

```
sage: f(x) = x         # au lieu de 'f = x'
sage: g = f.derivative()
sage: g
x |--> 1
sage: g(3)
1
sage: type(g)
<class 'sage.symbolic.expression.Expression'>
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f = symbolic_expression(x        ).function(x)# au lieu de
↪'f = x'
>>> g = f.derivative()
>>> g
x |--> 1
>>> g(Integer(3))
1
>>> type(g)
<class 'sage.symbolic.expression.Expression'>
```

— Ou, sans changer la définition de `f`, définir `g` comme une expression symbolique fonctionnelle

```
sage: f = x
sage: g(x) = f.derivative()  # au lieu de 'g = f.derivative()'
sage: g
x |--> 1
sage: g(3)
1
sage: type(g)
<class 'sage.symbolic.expression.Expression'>
```

```
>>> from sage.all import *
>>> f = x
>>> __tmp__=var("x"); g = symbolic_expression(f.derivative()  ).function(x)# au␣
↪lieu de 'g = f.derivative()'
>>> g
x |--> 1
>>> g(Integer(3))
1
>>> type(g)
<class 'sage.symbolic.expression.Expression'>
```

— Ou encore, avec `f` et `g` définies comme dans l'exemple de départ, donner explicitement la variable à remplacer par sa valeur

```
sage: f = x
sage: g = f.derivative()
sage: g
1
sage: g(x=3)    # au lieu de  'g(3)'
1
```

```
>>> from sage.all import *
>>> f = x
>>> g = f.derivative()
>>> g
1
>>> g(x=Integer(3))     # au lieu de  'g(3)'
1
```

Nous terminons en mettant encore une fois en évidence la différence entre les dérivées des expressions `f` définies par `f = x` et par `f(x) = x`

```
sage: f(x) = x
sage: g = f.derivative()
sage: g.variables()  # variables apparaissant dans g
()
sage: g.arguments()  # paramètres auxquels on peut donner une valeur dans g
(x,)
sage: f = x
sage: h = f.derivative()
sage: h.variables()
()
sage: h.arguments()
()
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f = symbolic_expression(x).function(x)
>>> g = f.derivative()
>>> g.variables()  # variables apparaissant dans g
()
>>> g.arguments()  # paramètres auxquels on peut donner une valeur dans g
(x,)
>>> f = x
>>> h = f.derivative()
>>> h.variables()
()
>>> h.arguments()
()
```

Comme l'illustre cet exemple, `h` n'accepte pas de paramètres. C'est pour cela que `h(3)` déclenche une erreur.

2.7 Anneaux de base

Nous illustrons la prise en main de quelques anneaux de base avec Sage. Par exemple, `RationalField()` ou `QQ` désigneront dans ce qui suit au corps des nombres rationnels :

```
sage: RationalField()
Rational Field
sage: QQ
Rational Field
sage: 1/2 in QQ
True
```

```
>>> from sage.all import *
>>> RationalField()
Rational Field
>>> QQ
Rational Field
>>> Integer(1)/Integer(2) in QQ
True
```

Le nombre décimal `1.2` est considéré comme un élément de `QQ`, puisqu'il existe une application de coercition entre les réels et les rationnels :

```
sage: 1.2 in QQ
True
```

```
>>> from sage.all import *
>>> RealNumber('1.2') in QQ
True
```

Néanmoins, il n'y a pas d'application de coercition entre le corps fini à 3 éléments et les rationnels :

```
sage: c = GF(3)(1)    # c est l'élément 1 du corps fini à 3 éléments
sage: c in QQ
False
```

```
>>> from sage.all import *
>>> c = GF(Integer(3))(Integer(1))    # c est l'élément 1 du corps fini à 3 éléments
>>> c in QQ
False
```

De même, bien entendu, la constante symbolique π n'appartient pas aux rationnels :

```
sage: pi in QQ
False
```

```
>>> from sage.all import *
>>> pi in QQ
False
```

Le symbole `I` représente la racine carrée de -1 ; `i` est synonyme de `I`. Bien entendu, `I` n'appartient pas aux rationnels :

```
sage: i  # i^2 = -1
I
sage: i in QQ
False
```

```
>>> from sage.all import *
>>> i  # i^2 = -1
I
>>> i in QQ
False
```

À ce propos, d'autres anneaux sont prédéfinis en Sage : l'anneau des entiers relatifs `ZZ`, celui des nombres réels `RR` et celui des nombres complexes `CC`. Les anneaux de polynômes sont décrits dans *Polynômes*.

Passons maintenant à quelques éléments d'arithmétique.

```
sage: a, b = 4/3, 2/3
sage: a + b
2
sage: 2*b == a
True
sage: parent(2/3)
Rational Field
sage: parent(4/2)
Rational Field
sage: 2/3 + 0.1       # coercition automatique avant addition
0.766666666666667
sage: 0.1 + 2/3       # les règles de coercition sont symétriques en Sage
0.766666666666667
```

```
>>> from sage.all import *
>>> a, b = Integer(4)/Integer(3), Integer(2)/Integer(3)
>>> a + b
2
>>> Integer(2)*b == a
True
>>> parent(Integer(2)/Integer(3))
Rational Field
>>> parent(Integer(4)/Integer(2))
Rational Field
>>> Integer(2)/Integer(3) + RealNumber('0.1')       # coercition automatique avant␣
↪addition
0.766666666666667
>>> RealNumber('0.1') + Integer(2)/Integer(3)       # les règles de coercition sont␣
↪symétriques en Sage
0.766666666666667
```

Il y a une subtilité dans la définition des nombres complexes. Comme mentionné ci-dessus, le symbole `i` représente une racine carrée de -1, mais il s'agit d'une racine carrée *formelle* de -1, comme élément d'un corps de nombres quadratique. L'appel `CC(i)` renvoie la racine carrée de -1 comme nombre complexe en virgule flottante.

```
sage: i = CC(i)       # nombre complexe en virgule flottante
sage: z = a + b*i
sage: z
1.33333333333333 + 0.666666666666667*I
sage: z.imag()        # partie imaginaire
0.666666666666667
sage: z.real() == a   # coercition automatique avant comparaison
True
sage: QQ(11.1)
111/10
```

```
>>> from sage.all import *
>>> i = CC(i)       # nombre complexe en virgule flottante
>>> z = a + b*i
>>> z
1.33333333333333 + 0.666666666666667*I
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> z.imag()         # partie imaginaire
0.666666666666667
>>> z.real() == a    # coercition automatique avant comparaison
True
>>> QQ(RealNumber('11.1'))
111/10
```

2.8 Polynômes

Dans cette partie, nous expliquons comment créer et utiliser des polynômes avec Sage.

2.8.1 Polynômes univariés

Il existe trois façons de créer des anneaux de polynômes.

```
sage: R = PolynomialRing(QQ, 't')
sage: R
Univariate Polynomial Ring in t over Rational Field
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, 't')
>>> R
Univariate Polynomial Ring in t over Rational Field
```

Ceci crée un anneau de polynômes et indique à Sage d'utiliser (la chaîne de caractère) "t" comme indéterminée lors de l'affichage à l'écran. Toutefois, ceci ne définit pas le symbole `t` pour son utilisation dans Sage. Aussi, il n'est pas possible de l'utiliser pour saisir un polynôme (comme $t^2 + 1$) qui appartient à `R`.

Une deuxième manière de procéder est

```
sage: S = QQ['t']
sage: S == R
True
```

```
>>> from sage.all import *
>>> S = QQ['t']
>>> S == R
True
```

Ceci a les mêmes effets en ce qui concerne `t`.

Une troisième manière de procéder, très pratique, consiste à entrer

```
sage: R.<t> = PolynomialRing(QQ)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('t',)); (t,) = R._first_ngens(1)
```

ou

```
sage: R.<t> = QQ['t']
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
```

ou même

```
sage: R.<t> = QQ[]
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
```

L'effet secondaire de ces dernières instructions est de définir la variable `t` comme l'indéterminée de l'anneau de polynômes. Ceci permet de construire très aisément des éléments de `R`, comme décrit ci-après. (Noter que cette troisième manière est très semblable à la notation par constructeur de Magma et que, de même que dans Magma, ceci peut servir pour une très large classe d'objets.)

```
sage: poly = (t+1) * (t+2); poly
t^2 + 3*t + 2
sage: poly in R
True
```

```
>>> from sage.all import *
>>> poly = (t+Integer(1)) * (t+Integer(2)); poly
t^2 + 3*t + 2
>>> poly in R
True
```

Quelle que soit la méthode utilisée pour définir l'anneau de polynômes, on récupère l'indéterminée comme le 0-ième générateur :

```
sage: R = PolynomialRing(QQ, 't')
sage: t = R.0
sage: t in R
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, 't')
>>> t = R.gen(0)
>>> t in R
True
```

Notez que les nombres complexes peuvent être construits de façon similaire : les nombres complexes peuvent être vus comme engendrés sur les réels par le symbole `i`. Aussi, on dispose de :

```
sage: CC
Complex Field with 53 bits of precision
sage: CC.0  # 0ième générateur CC
1.00000000000000*I
```

```
>>> from sage.all import *
>>> CC
Complex Field with 53 bits of precision
>>> CC.gen(0)  # 0ième générateur CC
1.00000000000000*I
```

Pour un anneau de polynômes, on peut obtenir à la fois l'anneau et son générateur ou juste le générateur au moment de la création de l'anneau comme suit :

```
sage: R, t = QQ['t'].objgen()
sage: t    = QQ['t'].gen()
sage: R, t = objgen(QQ['t'])
sage: t    = gen(QQ['t'])
```

```
>>> from sage.all import *
>>> R, t = QQ['t'].objgen()
>>> t    = QQ['t'].gen()
>>> R, t = objgen(QQ['t'])
>>> t    = gen(QQ['t'])
```

Finalement, on peut faire de l'arithmétique dans $\mathbf{Q}[t]$.

```
sage: R, t = QQ['t'].objgen()
sage: f = 2*t^7 + 3*t^2 - 15/19
sage: f^2
4*t^14 + 12*t^9 - 60/19*t^7 + 9*t^4 - 90/19*t^2 + 225/361
sage: cyclo = R.cyclotomic_polynomial(7); cyclo
t^6 + t^5 + t^4 + t^3 + t^2 + t + 1
sage: g = 7 * cyclo * t^5 * (t^5 + 10*t + 2)
sage: g
7*t^16 + 7*t^15 + 7*t^14 + 7*t^13 + 77*t^12 + 91*t^11 + 91*t^10 + 84*t^9
       + 84*t^8 + 84*t^7 + 84*t^6 + 14*t^5
sage: F = factor(g); F
(7) * t^5 * (t^5 + 10*t + 2) * (t^6 + t^5 + t^4 + t^3 + t^2 + t + 1)
sage: F.unit()
7
sage: list(F)
[(t, 5), (t^5 + 10*t + 2, 1), (t^6 + t^5 + t^4 + t^3 + t^2 + t + 1, 1)]
```

```
>>> from sage.all import *
>>> R, t = QQ['t'].objgen()
>>> f = Integer(2)*t**Integer(7) + Integer(3)*t**Integer(2) - Integer(15)/Integer(19)
>>> f**Integer(2)
4*t^14 + 12*t^9 - 60/19*t^7 + 9*t^4 - 90/19*t^2 + 225/361
>>> cyclo = R.cyclotomic_polynomial(Integer(7)); cyclo
t^6 + t^5 + t^4 + t^3 + t^2 + t + 1
>>> g = Integer(7) * cyclo * t**Integer(5) * (t**Integer(5) + Integer(10)*t +␣
↪Integer(2))
>>> g
7*t^16 + 7*t^15 + 7*t^14 + 7*t^13 + 77*t^12 + 91*t^11 + 91*t^10 + 84*t^9
       + 84*t^8 + 84*t^7 + 84*t^6 + 14*t^5
>>> F = factor(g); F
(7) * t^5 * (t^5 + 10*t + 2) * (t^6 + t^5 + t^4 + t^3 + t^2 + t + 1)
>>> F.unit()
7
>>> list(F)
[(t, 5), (t^5 + 10*t + 2, 1), (t^6 + t^5 + t^4 + t^3 + t^2 + t + 1, 1)]
```

On remarquera que la factorisation prend correctement en compte le coefficient dominant, et ne l'oublie pas dans le résultat.

S'il arrive que vous utilisiez intensivement, par exemple, la fonction `R.cyclotomic_polynomial` dans un projet de recherche quelconque, en plus de citer Sage, vous devriez chercher à quel composant Sage fait appel pour calculer en réalité ce polynôme cyclotomique et citer ce composant. Dans ce cas particulier, en tapant `R.cyclotomic_polynomial??` pour voir le code source, vous verriez rapidement une ligne telle que `f = pari.polcyclo(n)` ce qui signifie que PARI est utilisé pour le calcul du polynôme cyclotomique. Pensez à citer PARI dans votre travail.

La division d'un polynôme par un autre produit un élément du corps des fractions, que Sage crée automatiquement.

```
sage: x = QQ['x'].0
sage: f = x^3 + 1; g = x^2 - 17
sage: h = f/g;  h
(x^3 + 1)/(x^2 - 17)
sage: h.parent()
Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> x = QQ['x'].gen(0)
>>> f = x**Integer(3) + Integer(1); g = x**Integer(2) - Integer(17)
>>> h = f/g;  h
(x^3 + 1)/(x^2 - 17)
>>> h.parent()
Fraction Field of Univariate Polynomial Ring in x over Rational Field
```

En utilisant des séries de Laurent, on peut calculer des développements en série dans le corps des fractions de `QQ[x]` :

```
sage: R.<x> = LaurentSeriesRing(QQ); R
Laurent Series Ring in x over Rational Field
sage: 1/(1-x) + O(x^10)
1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^8 + x^9 + O(x^10)
```

```
>>> from sage.all import *
>>> R = LaurentSeriesRing(QQ, names=('x',)); (x,) = R._first_ngens(1); R
Laurent Series Ring in x over Rational Field
>>> Integer(1)/(Integer(1)-x) + O(x**Integer(10))
1 + x + x^2 + x^3 + x^4 + x^5 + x^6 + x^7 + x^8 + x^9 + O(x^10)
```

Si l'on nomme les variables différemment, on obtient un anneau de polynômes univariés différent.

```
sage: R.<x> = PolynomialRing(QQ)
sage: S.<y> = PolynomialRing(QQ)
sage: x == y
False
sage: R == S
False
sage: R(y)
x
sage: R(y^2 - 17)
x^2 - 17
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> S = PolynomialRing(QQ, names=('y',)); (y,) = S._first_ngens(1)
>>> x == y
```

(suite sur la page suivante)

(suite de la page précédente)

```
False
>>> R == S
False
>>> R(y)
x
>>> R(y**Integer(2) - Integer(17))
x^2 - 17
```

L'anneau est déterminé par sa variable. Notez que créer un autre anneau avec la même variable `x` ne renvoie pas de nouvel anneau.

```
sage: R = PolynomialRing(QQ, "x")
sage: T = PolynomialRing(QQ, "x")
sage: R == T
True
sage: R is T
True
sage: R.0 == T.0
True
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, "x")
>>> T = PolynomialRing(QQ, "x")
>>> R == T
True
>>> R is T
True
>>> R.gen(0) == T.gen(0)
True
```

Sage permet aussi de travailler dans des anneaux de séries formelles et de séries de Laurent sur un anneau de base quelconque. Dans l'exemple suivant, nous créons un élément de $\mathbf{F}_7[[T]]$ et effectuons une division pour obtenir un élément de $\mathbf{F}_7((T))$.

```
sage: R.<T> = PowerSeriesRing(GF(7)); R
Power Series Ring in T over Finite Field of size 7
sage: f = T  + 3*T^2 + T^3 + O(T^4)
sage: f^3
T^3 + 2*T^4 + 2*T^5 + O(T^6)
sage: 1/f
T^-1 + 4 + T + O(T^2)
sage: parent(1/f)
Laurent Series Ring in T over Finite Field of size 7
```

```
>>> from sage.all import *
>>> R = PowerSeriesRing(GF(Integer(7)), names=('T',)); (T,) = R._first_ngens(1); R
Power Series Ring in T over Finite Field of size 7
>>> f = T  + Integer(3)*T**Integer(2) + T**Integer(3) + O(T**Integer(4))
>>> f**Integer(3)
T^3 + 2*T^4 + 2*T^5 + O(T^6)
>>> Integer(1)/f
T^-1 + 4 + T + O(T^2)
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> parent(Integer(1)/f)
Laurent Series Ring in T over Finite Field of size 7
```

On peut aussi créer des anneaux de séries formelles en utilisant des doubles crochets :

```
sage: GF(7)[['T']]
Power Series Ring in T over Finite Field of size 7
```

```
>>> from sage.all import *
>>> GF(Integer(7))[['T']]
Power Series Ring in T over Finite Field of size 7
```

2.8.2 Polynômes multivariés

Pour travailler avec des polynômes à plusieurs variables, on commence par déclarer l'anneau des polynômes et les variables, de l'une des deux manières suivantes.

```
sage: R = PolynomialRing(GF(5),3,"z") # here, 3 = number of variables
sage: R
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(5)),Integer(3),"z") # here, 3 = number of variables
>>> R
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
```

De même que pour les polynômes à une seule variable, les variantes suivantes sont autorisées :

```
sage: GF(5)['z0, z1, z2']
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
sage: R.<z0,z1,z2> = GF(5)[]; R
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
```

```
>>> from sage.all import *
>>> GF(Integer(5))['z0, z1, z2']
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
>>> R = GF(Integer(5))['z0, z1, z2']; (z0, z1, z2,) = R._first_ngens(3); R
Multivariate Polynomial Ring in z0, z1, z2 over Finite Field of size 5
```

Si l'on désire de simples lettres comme noms de variables, on peut utiliser les raccourcis suivants :

```
sage: PolynomialRing(GF(5), 3, 'xyz')
Multivariate Polynomial Ring in x, y, z over Finite Field of size 5
```

```
>>> from sage.all import *
>>> PolynomialRing(GF(Integer(5)), Integer(3), 'xyz')
Multivariate Polynomial Ring in x, y, z over Finite Field of size 5
```

A présent, passons aux questions arithmétiques.

```
sage: z = GF(5)['z0, z1, z2'].gens()
sage: z
(z0, z1, z2)
sage: (z[0]+z[1]+z[2])^2
z0^2 + 2*z0*z1 + z1^2 + 2*z0*z2 + 2*z1*z2 + z2^2
```

```
>>> from sage.all import *
>>> z = GF(Integer(5))['z0, z1, z2'].gens()
>>> z
(z0, z1, z2)
>>> (z[Integer(0)]+z[Integer(1)]+z[Integer(2)])**Integer(2)
z0^2 + 2*z0*z1 + z1^2 + 2*z0*z2 + 2*z1*z2 + z2^2
```

On peut aussi utiliser des notations plus mathématiques pour construire un anneau de polynômes.

```
sage: R = GF(5)['x,y,z']
sage: x,y,z = R.gens()
sage: QQ['x']
Univariate Polynomial Ring in x over Rational Field
sage: QQ['x,y'].gens()
(x, y)
sage: QQ['x'].objgens()
(Univariate Polynomial Ring in x over Rational Field, (x,))
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))['x,y,z']
>>> x,y,z = R.gens()
>>> QQ['x']
Univariate Polynomial Ring in x over Rational Field
>>> QQ['x,y'].gens()
(x, y)
>>> QQ['x'].objgens()
(Univariate Polynomial Ring in x over Rational Field, (x,))
```

Sous Sage, les polynômes multivariés sont implémentés en représentation « distributive » (par opposition à récursive), à l'aide de dictionnaires Python. Sage a souvent recours à Singular [Si], par exemple, pour le calcul de pgcd ou de bases de Gröbner d'idéaux.

```
sage: R, (x, y) = PolynomialRing(RationalField(), 2, 'xy').objgens()
sage: f = (x^3 + 2*y^2*x)^2
sage: g = x^2*y^2
sage: f.gcd(g)
x^2
```

```
>>> from sage.all import *
>>> R, (x, y) = PolynomialRing(RationalField(), Integer(2), 'xy').objgens()
>>> f = (x**Integer(3) + Integer(2)*y**Integer(2)*x)**Integer(2)
>>> g = x**Integer(2)*y**Integer(2)
>>> f.gcd(g)
x^2
```

Créons ensuite l'idéal (f, g) engendré par f et g, en multipliant simplement `(f,g)` par `R` (nous pourrions aussi bien écrire `ideal([f,g])` ou `ideal(f,g)`).

```
sage: I = (f, g)*R; I
Ideal (x^6 + 4*x^4*y^2 + 4*x^2*y^4, x^2*y^2) of Multivariate Polynomial
Ring in x, y over Rational Field
sage: B = I.groebner_basis(); B
[x^6, x^2*y^2]
sage: x^2 in I
False
```

```
>>> from sage.all import *
>>> I = (f, g)*R; I
Ideal (x^6 + 4*x^4*y^2 + 4*x^2*y^4, x^2*y^2) of Multivariate Polynomial
Ring in x, y over Rational Field
>>> B = I.groebner_basis(); B
[x^6, x^2*y^2]
>>> x**Integer(2) in I
False
```

En passant, la base de Gröbner ci-dessus n'est pas une liste mais une suite non mutable. Ceci signifie qu'elle possède un univers, un parent, et qu'elle ne peut pas être modifiée (ce qui est une bonne chose puisque changer la base perturberait d'autres routines qui utilisent la base de Gröbner).

```
sage: B.universe()
Multivariate Polynomial Ring in x, y over Rational Field
sage: B[1] = x
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
```

```
>>> from sage.all import *
>>> B.universe()
Multivariate Polynomial Ring in x, y over Rational Field
>>> B[Integer(1)] = x
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
```

Un peu (comprenez : pas assez à notre goût) d'algèbre commutative est disponible en Sage. Ces routines font appel à Singular. Par exemple, il est possible de calculer la décomposition en facteurs premiers et les idéaux premiers associés de I :

```
sage: I.primary_decomposition()
[Ideal (x^2) of Multivariate Polynomial Ring in x, y over Rational Field,
 Ideal (y^2, x^6) of Multivariate Polynomial Ring in x, y over Rational Field]
sage: I.associated_primes()
[Ideal (x) of Multivariate Polynomial Ring in x, y over Rational Field,
 Ideal (y, x) of Multivariate Polynomial Ring in x, y over Rational Field]
```

```
>>> from sage.all import *
>>> I.primary_decomposition()
[Ideal (x^2) of Multivariate Polynomial Ring in x, y over Rational Field,
 Ideal (y^2, x^6) of Multivariate Polynomial Ring in x, y over Rational Field]
>>> I.associated_primes()
```

(suite sur la page suivante)

(suite de la page précédente)

```
[Ideal (x) of Multivariate Polynomial Ring in x, y over Rational Field,
 Ideal (y, x) of Multivariate Polynomial Ring in x, y over Rational Field]
```

2.9 Parents, conversions, coercitions

Cette section peut paraître plus technique que celles qui précèdent, mais nous pensons qu'il est important de comprendre ce que sont les parents et les coercitions pour utiliser comme il faut les structures algébriques fournies par Sage.

Nous allons voir ici ce que ces notions signifient, mais pas comment les mettre en œuvre pour implémenter une nouvelle structure algébrique. Un tutoriel thématique couvrant ce point est disponible ici.

2.9.1 Éléments

Une première approximation en Python de la notion mathématique d'anneau pourrait consister à définir une classe pour les éléments `x` de l'anneau concerné, de fournir les méthodes « double-underscore » nécessaires pour donner un sens aux opérations de l'anneau, par exemple `__add__`, `__sub__` et `__mul__`, et naturellement de s'assurer qu'elles respectent les axiomes de la structure d'anneau.

Python étant un language (dynamiquement) fortement typé, on pourrait s'attendre à devoir implémenter une classe pour chaque anneau. Après tout, Python définit bien un type `<int>` pour les entiers, un type `<float>` pour les réels, et ainsi de suite. Mais cette approche ne peut pas fonctionner : il y a une infinité d'anneaux différents, et l'on ne peut pas implémenter une infinité de classes !

Une autre idée est de créer une hiérarchie de classes destinées à implémenter les éléments des structures algébriques usuelles : éléments de groupes, d'anneaux, d'algèbres à division, d'anneaux commutatifs, de corps, d'algèbres, etc.

Mais cela signifie que des éléments d'anneaux franchement différents peuvent avoir le même type.

```
sage: P.<x,y> = GF(3)[]
sage: Q.<a,b> = GF(4,'z')[]
sage: type(x)==type(a)
True
```

```
>>> from sage.all import *
>>> P = GF(Integer(3))['x, y']; (x, y,) = P._first_ngens(2)
>>> Q = GF(Integer(4),'z')['a, b']; (a, b,) = Q._first_ngens(2)
>>> type(x)==type(a)
True
```

On pourrait aussi vouloir avoir des classes Python différentes pour fournir plusieurs implémentations d'une même structure mathématique (matrices denses contre matrices creuses par exemple).

```
sage: P.<a> = PolynomialRing(ZZ)
sage: Q.<b> = PolynomialRing(ZZ, sparse=True)
sage: R.<c> = PolynomialRing(ZZ, implementation='NTL')
sage: type(a); type(b); type(c)
<class 'sage.rings.polynomial.polynomial_integer_dense_flint.Polynomial_integer_dense_
↪flint'>
<class 'sage.rings.polynomial.polynomial_ring.PolynomialRing_integral_domain_with_
↪category.element_class'>
<class 'sage.rings.polynomial.polynomial_integer_dense_ntl.Polynomial_integer_dense_
↪ntl'>
```

```
>>> from sage.all import *
>>> P = PolynomialRing(ZZ, names=('a',)); (a,) = P._first_ngens(1)
>>> Q = PolynomialRing(ZZ, sparse=True, names=('b',)); (b,) = Q._first_ngens(1)
>>> R = PolynomialRing(ZZ, implementation='NTL', names=('c',)); (c,) = R._first_
↪ngens(1)
>>> type(a); type(b); type(c)
<class 'sage.rings.polynomial.polynomial_integer_dense_flint.Polynomial_integer_dense_
↪flint'>
<class 'sage.rings.polynomial.polynomial_ring.PolynomialRing_integral_domain_with_
↪category.element_class'>
<class 'sage.rings.polynomial.polynomial_integer_dense_ntl.Polynomial_integer_dense_
↪ntl'>
```

Deux problèmes se posent alors. D'une part, si deux éléments sont instances de la même classe, on s'attend à ce que leur méthode `__add__` soit capable de les additionner, alors que ce n'est pas ce que l'on souhaite si les éléments appartiennent en fait à des anneaux différents. D'autre part, si l'on a deux éléments qui appartiennent à des implémentations différentes d'un même anneau, on veut pouvoir les ajouter, et ce n'est pas immédiats s'ils ne sont pas instances de la même classe.

La solution à ces difficultés est fournie par le mécanisme de *coercition* décrit ci-dessous.

Mais avant tout, il est essentiel que chaque élément « sache » de quoi il est élément. Cette information est donnée par la méthode `parent()`.

```
sage: a.parent(); b.parent(); c.parent()
Univariate Polynomial Ring in a over Integer Ring
Sparse Univariate Polynomial Ring in b over Integer Ring
Univariate Polynomial Ring in c over Integer Ring (using NTL)
```

```
>>> from sage.all import *
>>> a.parent(); b.parent(); c.parent()
Univariate Polynomial Ring in a over Integer Ring
Sparse Univariate Polynomial Ring in b over Integer Ring
Univariate Polynomial Ring in c over Integer Ring (using NTL)
```

2.9.2 Parents et catégories

En plus d'une hiérarchie de classes destinée à implémenter les éléments de structures algébriques, Sage fournit une hiérarchie similaire pour les structures elles-mêmes. Ces structures s'appellent en Sage des *parents*, et leurs classes dérivent d'une même classe de base. Celle-ci a des sous-classes « ensemble », « anneau », « corps », et ainsi de suite, dont la hiérarchie correspond à peu près à celle des concepts mathématiques qu'elles décrivent :

```
sage: isinstance(QQ,Field)
True
sage: isinstance(QQ, Ring)
True
sage: isinstance(ZZ,Field)
False
sage: isinstance(ZZ, Ring)
True
```

```
>>> from sage.all import *
>>> isinstance(QQ,Field)
True
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> isinstance(QQ, Ring)
True
>>> isinstance(ZZ,Field)
False
>>> isinstance(ZZ, Ring)
True
```

Or en algèbre, on regroupe les objets qui partagent le même genre de structure algébrique en ce que l'on appelle des *catégories*. Il y a donc un parallèle approximatif entre la hiérarchie des classes de Sage et la hiérarchie des catégories. Mais cette correspondance n'est pas parfaite, et Sage implémente par ailleurs les catégories en tant que telles :

```
sage: Rings()
Category of rings
sage: ZZ.category()
Join of Category of euclidean domains
    and Category of noetherian rings
    and Category of infinite enumerated sets
    and Category of metric spaces
sage: ZZ.category().is_subcategory(Rings())
True
sage: ZZ in Rings()
True
sage: ZZ in Fields()
False
sage: QQ in Fields()
True
```

```
>>> from sage.all import *
>>> Rings()
Category of rings
>>> ZZ.category()
Join of Category of euclidean domains
    and Category of noetherian rings
    and Category of infinite enumerated sets
    and Category of metric spaces
>>> ZZ.category().is_subcategory(Rings())
True
>>> ZZ in Rings()
True
>>> ZZ in Fields()
False
>>> QQ in Fields()
True
```

Tandis que la hiérarchie des classes est déterminée avant tout par des considérations de programmation, l'infrastructure des catégories cherche plutôt à respecter la structure mathématique. Elle permet de munir les objets d'une catégorie de méthodes et de tests génériques, qui ne dépendent pas de l'implémentation particulière d'un objet donné de la catégorie.

Les parents en tant qu'objets Python doivent être uniques. Ainsi, lorsqu'un anneau de polynômes sur un anneau donné et avec une liste donnée de générateurs est construit, il est conservé en cache et réutilisé par la suite :

```
sage: RR['x','y'] is RR['x','y']
True
```

```
>>> from sage.all import *
>>> RR['x','y'] is RR['x','y']
True
```

2.9.3 Types et parents

Le type `RingElement` ne correspond pas parfaitement à la notion mathématique d'élément d'anneau. Par exemple, bien que les matrices carrées appartiennent à un anneau, elles ne sont pas de type `RingElement` :

```
sage: M = Matrix(ZZ,2,2); M
[0 0]
[0 0]
sage: isinstance(M, RingElement)
False
```

```
>>> from sage.all import *
>>> M = Matrix(ZZ,Integer(2),Integer(2)); M
[0 0]
[0 0]
>>> isinstance(M, RingElement)
False
```

Si les *parents* sont censés être uniques, des *éléments* égaux d'un parent ne sont pas nécessairement identiques. Le comportement de Sage diffère ici de celui de Python pour certains entiers (pas tous) :

```
sage: int(1) is int(1) # Python int
True
sage: int(-15) is int(-15)
False
sage: 1 is 1           # Sage Integer
False
```

```
>>> from sage.all import *
>>> int(Integer(1)) is int(Integer(1)) # Python int
True
>>> int(-Integer(15)) is int(-Integer(15))
False
>>> Integer(1) is Integer(1)           # Sage Integer
False
```

Il faut bien comprendre que les éléments d'anneaux différents ne se distinguent généralement pas par leur type, mais par leur parent :

```
sage: a = GF(2)(1)
sage: b = GF(5)(1)
sage: type(a) is type(b)
True
sage: parent(a)
Finite Field of size 2
sage: parent(b)
Finite Field of size 5
```

```
>>> from sage.all import *
>>> a = GF(Integer(2))(Integer(1))
>>> b = GF(Integer(5))(Integer(1))
>>> type(a) is type(b)
True
>>> parent(a)
Finite Field of size 2
>>> parent(b)
Finite Field of size 5
```

Ainsi, **le parent d'un élément est plus important que son type** du point de vue algébrique.

2.9.4 Conversion et coercition

Il est parfois possible de convertir un élément d'un certain parent en élément d'un autre parent. Une telle conversion peut être explicite ou implicite. Les conversions implicites sont appelées *coercitions*.

Le lecteur aura peut-être rencontré les notions de *conversion de type* et de *coercition de type* dans le contexte du langage C par exemple. En Sage, il existe aussi des notions de conversion et de coercition, mais elles s'appliquent aux *parents* et non aux types. Attention donc à ne pas confondre les conversions en Sage avec les conversions de type du C !

Nous nous limitons ici à une brève présentation, et renvoyons le lecteur à la section du manuel de référence consacrée aux coercitions ainsi qu'au tutoriel spécifique pour plus de détails.

On peut adopter deux positions extrêmes sur les opérations arithmétiques entre éléments d'anneaux *différents* :

— les anneaux différents sont des mondes indépendants, et l'addition ou la multiplication entre éléments d'anneaux différents n'ont aucun sens ; même `1 + 1/2` n'a pas de sens puisque le premier terme est un entier et le second un rationnel ;

ou

— si un élément `r1` d'un anneau `R1` peut, d'une manière ou d'une autre, s'interpréter comme élément d'un autre anneau `R2`, alors toutes les opérations arithmétiques entre `r1` et un élément quelconque de `R2` sont permises. En particulier, les éléments neutres de la multiplication dans les corps et anneaux doivent tous être égaux entre eux.

Sage adopte un compromis. Si `P1` et `P2` sont des parents et si `p1` est un élément de `P1`, l'utilisateur peut demander explicitement comment `P1` s'interprète dans `P2`. Cela n'a pas forcément de sens dans tous les cas, et l'interprétation peut n'être définie que pour certains éléments de `P1` ; c'est à l'utilisateur de s'assurer que la conversion a un sens. Cela s'appelle une **conversion** :

```
sage: a = GF(2)(1)
sage: b = GF(5)(1)
sage: GF(5)(a) == b
True
sage: GF(2)(b) == a
True
```

```
>>> from sage.all import *
>>> a = GF(Integer(2))(Integer(1))
>>> b = GF(Integer(5))(Integer(1))
>>> GF(Integer(5))(a) == b
True
>>> GF(Integer(2))(b) == a
True
```

Cependant, une conversion *implicite* (c'est-à-dire automatique) n'est possible que si elle peut se faire *systématiquement* et de manière *cohérente*. Il faut ici absolument faire preuve de rigueur.

Une telle conversion implicite s'appelle une **coercition**. Si une coercition est définie entre deux parents, elle doit coïncider avec la conversion. De plus, les coercitions doivent obéir aux deux conditions suivantes :

1. Une coercition de `P1` dans `P2` doit être un morphisme (par exemple un morphisme d'anneaux). Elle doit être définie pour *tous* les éléments de `P1`, et préserver la structure algébrique de celui-ci.
2. Le choix des applications de coercition doit être fait de manière cohérente. Si `P3` est un troisième parent, la composée de la coercition choisie de `P1` dans `P2` et de celle de `P2` dans `P3` doit être la coercition de `P1` dans `P3`. En particulier, s'il existe des coercitions de `P1` dans `P2` et de `P2` dans `P1`, leur composée doit être l'identité sur `P1`.

Ainsi, bien qu'il soit possible de convertir tout élément de `GF(2)` en un élément de `GF(5)`, la conversion ne peut être une coercition, puisque il n'existe pas de morphisme d'anneaux de `GF(2)` dans `GF(5)`.

Le second point — la cohérence des choix — est un peu plus compliqué à expliquer. Illustrons-le sur l'exemple des anneaux de polynômes multivariés. Dans les applications, il s'avère utile que les coercitions respectent les noms des variables. Nous avons donc :

```
sage: R1.<x,y> = ZZ[]
sage: R2 = ZZ['y','x']
sage: R2.has_coerce_map_from(R1)
True
sage: R2(x)
x
sage: R2(y)
y
sage: R2.coerce(y)
y
```

```
>>> from sage.all import *
>>> R1 = ZZ['x, y']; (x, y,) = R1._first_ngens(2)
>>> R2 = ZZ['y','x']
>>> R2.has_coerce_map_from(R1)
True
>>> R2(x)
x
>>> R2(y)
y
>>> R2.coerce(y)
y
```

En l'absence d'un morphisme d'anneau qui préserve les noms de variable, la coercition entre anneaux de polynômes multivariés n'est pas définie. Il peut tout de même exister une conversion qui envoie les variables d'un anneau sur celle de l'autre en fonction de leur position dans la liste des générateurs :

```
sage: R3 = ZZ['z','x']
sage: R3.has_coerce_map_from(R1)
False
sage: R3(x)
z
sage: R3(y)
x
sage: R3.coerce(y)
Traceback (most recent call last):
...
TypeError: no canonical coercion
from Multivariate Polynomial Ring in x, y over Integer Ring
to Multivariate Polynomial Ring in z, x over Integer Ring
```

```
>>> from sage.all import *
>>> R3 = ZZ['z','x']
>>> R3.has_coerce_map_from(R1)
False
>>> R3(x)
z
>>> R3(y)
x
>>> R3.coerce(y)
Traceback (most recent call last):
...
TypeError: no canonical coercion
from Multivariate Polynomial Ring in x, y over Integer Ring
to Multivariate Polynomial Ring in z, x over Integer Ring
```

Mais une telle conversion ne répond pas aux critères pour être une coercition : en effet, en composant l'application de `ZZ['x','y']` dans `ZZ['y','x']` avec celle qui préserve les positions de `ZZ['y','x']` dans `ZZ['a','b']`, nous obtiendrions une application qui ne préserve ni les noms ni les positions, ce qui viole la règle de cohérence.

Lorsqu'une coercition est définie, elle est souvent utilisée pour comparer des éléments d'anneaux différents ou pour effectuer des opérations arithmétiques. Cela est commode, mais il faut être prudent en étendant la relation d'égalité == au-delà des frontières d'un parent donné. Par exemple, si == est bien censé être une relation d'équivalence entre éléments d'"*un* anneau, il n'en va pas forcément de même quand on compare des éléments d'anneaux différents. Ainsi, les éléments `1` de `ZZ` et d'un corps fini sont considérés comme égaux, puisqu'il existe une coercition canonique des entiers dans tout corps fini. En revanche, il n'y a en général pas de coercition entre deux corps finis quelconques. On a donc

```
sage: GF(5)(1) == 1
True
sage: 1 == GF(2)(1)
True
sage: GF(5)(1) == GF(2)(1)
False
sage: GF(5)(1) != GF(2)(1)
True
```

```
>>> from sage.all import *
>>> GF(Integer(5))(Integer(1)) == Integer(1)
True
>>> Integer(1) == GF(Integer(2))(Integer(1))
True
>>> GF(Integer(5))(Integer(1)) == GF(Integer(2))(Integer(1))
False
>>> GF(Integer(5))(Integer(1)) != GF(Integer(2))(Integer(1))
True
```

De même, on a

```
sage: R3(R1.1) == R3.1
True
sage: R1.1 == R3.1
False
sage: R1.1 != R3.1
True
```

```
>>> from sage.all import *
>>> R3(R1.gen(1)) == R3.gen(1)
True
>>> R1.gen(1) == R3.gen(1)
False
>>> R1.gen(1) != R3.gen(1)
True
```

Une autre conséquence de la condition de cohérence est que les coercitions ne sont possibles que des anneaux exacts (comme les rationnels `QQ`) vers les anneaux inexacts (comme les réels à précision donnée `RR`), jamais l'inverse. En effet, pour qu'une conversion de `RR` dans `QQ` puisse être une coercition, il faudrait que la composée de la coercition de `QQ` dans `RR` et de cette conversion soit l'identité sur `QQ`, ce qui n'est pas possible puisque des rationnels distincts peuvent très bien être envoyés sur le même élément de `RR` :

```
sage: RR(1/10^200+1/10^100) == RR(1/10^100)
True
sage: 1/10^200+1/10^100 == 1/10^100
False
```

```
>>> from sage.all import *
>>> RR(Integer(1)/Integer(10)**Integer(200)+Integer(1)/Integer(10)**Integer(100)) ==␣
↪RR(Integer(1)/Integer(10)**Integer(100))
True
>>> Integer(1)/Integer(10)**Integer(200)+Integer(1)/Integer(10)**Integer(100) ==␣
↪Integer(1)/Integer(10)**Integer(100)
False
```

Lorsque l'on compare des éléments de deux parents `P1` et `P2`, il peut arriver qu'il n'existe pas de coercition entre `P1` et `P2`, mais qu'il y ait un choix canonique de parent `P3` tel que `P1` et `P2` admettent tous deux des coercitions dans `P3`. Dans ce cas aussi, la coercition a lieu. Un exemple typique de ce mécanisme est l'addition d'un rationnel et d'un polynôme à coefficients entiers, qui produit un polynôme à coefficients rationnels :

```
sage: P1.<x> = ZZ[]
sage: p = 2*x+3
sage: q = 1/2
sage: parent(p)
Univariate Polynomial Ring in x over Integer Ring
sage: parent(p+q)
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> P1 = ZZ['x']; (x,) = P1._first_ngens(1)
>>> p = Integer(2)*x+Integer(3)
>>> q = Integer(1)/Integer(2)
>>> parent(p)
Univariate Polynomial Ring in x over Integer Ring
>>> parent(p+q)
Univariate Polynomial Ring in x over Rational Field
```

Notons qu'en principe, on aurait très bien pu choisir pour `P3` le corps des fractions de `ZZ['x']`. Cependant, Sage tente de choisir un parent commun *canonique* aussi naturel que possible (ici `QQ['x']`). Afin que cela fonctionne de façon fiable, Sage ne se contente *pas* de prendre n'importe lequel lorsque plusieurs candidats semblent aussi naturels les uns que les autres. La manière dont le choix est fait est décrite dans le tutoriel spécifique déjà mentionné.

Dans l'exemple suivant, il n'y a pas de coercition vers un parent commun :

```
sage: R.<x> = QQ[]
sage: S.<y> = QQ[]
sage: x+y
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +: 'Univariate Polynomial Ring in x over␣
↪Rational Field' and 'Univariate Polynomial Ring in y over Rational Field'
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> S = QQ['y']; (y,) = S._first_ngens(1)
>>> x+y
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +: 'Univariate Polynomial Ring in x over␣
↪Rational Field' and 'Univariate Polynomial Ring in y over Rational Field'
```

En effet, Sage refuse de choisir entre les candidats `QQ['x']['y']`, `QQ['y']['x']`, `QQ['x','y']` et `QQ['y','x']`, car ces quatre structures deux à deux distinctes semblent toutes des parents communs naturels, et aucun choix canonique ne s'impose.

2.10 Algèbre linéaire

Sage fournit les constructions standards d'algèbre linéaire, par exemple le polynôme caractéristique, la forme échelonnée, la trace, diverses décompositions, etc. d'une matrice.

La création de matrices et la multiplication matricielle sont très faciles et naturelles :

```
sage: A = Matrix([[1,2,3],[3,2,1],[1,1,1]])
sage: w = vector([1,1,-4])
sage: w*A
(0, 0, 0)
sage: A*w
(-9, 1, -2)
sage: kernel(A)
Free module of degree 3 and rank 1 over Integer Ring
Echelon basis matrix:
[ 1  1 -4]
```

```
>>> from sage.all import *
>>> A = Matrix([[Integer(1),Integer(2),Integer(3)],[Integer(3),Integer(2),Integer(1)],
↪[Integer(1),Integer(1),Integer(1)]])
>>> w = vector([Integer(1),Integer(1),-Integer(4)])
>>> w*A
(0, 0, 0)
>>> A*w
(-9, 1, -2)
>>> kernel(A)
Free module of degree 3 and rank 1 over Integer Ring
Echelon basis matrix:
[ 1  1 -4]
```

Notez bien qu'avec Sage, le noyau d'une matrice A est le « noyau à gauche », c'est-à-dire l'espace des vecteurs w tels que $wA = 0$.

La résolution d'équations matricielles est facile et se fait avec la méthode `solve_right`. L'évaluation de `A.solve_right(Y)` renvoie une matrice (ou un vecteur) X tel que $AX = Y$:

```
sage: Y = vector([0, -4, -1])
sage: X = A.solve_right(Y)
sage: X
(-2, 1, 0)
sage: A * X   # vérifions la réponse...
(0, -4, -1)
```

```
>>> from sage.all import *
>>> Y = vector([Integer(0), -Integer(4), -Integer(1)])
>>> X = A.solve_right(Y)
>>> X
(-2, 1, 0)
>>> A * X   # vérifions la réponse...
(0, -4, -1)
```

S'il n'y a aucune solution, Sage renvoie une erreur :

```
sage: A.solve_right(w)
Traceback (most recent call last):
...
ValueError: matrix equation has no solutions
```

```
>>> from sage.all import *
>>> A.solve_right(w)
Traceback (most recent call last):
...
ValueError: matrix equation has no solutions
```

De même, il faut utiliser `A.solve_left(Y)` pour résoudre en X l'équation $XA = Y$.

Sage sait aussi calculer les valeurs propres et vecteurs propres :

```
sage: A = matrix([[0, 4], [-1, 0]])
sage: A.eigenvalues ()
[-2*I, 2*I]
sage: B = matrix([[1, 3], [3, 1]])
sage: B.eigenvectors_left()
[(4, [(1, 1)], 1), (-2, [(1, -1)], 1)]
```

```
>>> from sage.all import *
>>> A = matrix([[Integer(0), Integer(4)], [-Integer(1), Integer(0)]])
>>> A.eigenvalues ()
[-2*I, 2*I]
>>> B = matrix([[Integer(1), Integer(3)], [Integer(3), Integer(1)]])
>>> B.eigenvectors_left()
[(4, [(1, 1)], 1), (-2, [(1, -1)], 1)]
```

(La sortie de `eigenvectors_left` est une liste de triplets (valeur propre, vecteur propre, multiplicité).) Sur `QQ` et `RR`, on peut aussi utiliser Maxima (voir la section *Maxima* ci-dessous).

Comme signalé en *Anneaux de base*, l'anneau sur lequel une matrice est définie a une influence sur les propriétés de la matrice. Dans l'exemple suivant, le premier argument de la commande `matrix` indique à Sage s'il faut traiter la matrice comme une matrice d'entier (`ZZ`), de rationnels (`QQ`) ou de réels (`RR`) :

```
sage: AZ = matrix(ZZ, [[2,0], [0,1]])
sage: AQ = matrix(QQ, [[2,0], [0,1]])
sage: AR = matrix(RR, [[2,0], [0,1]])
sage: AZ.echelon_form()
[2 0]
[0 1]
sage: AQ.echelon_form()
[1 0]
[0 1]
sage: AR.echelon_form()
[ 1.00000000000000 0.000000000000000]
[0.000000000000000  1.00000000000000]
```

```
>>> from sage.all import *
>>> AZ = matrix(ZZ, [[Integer(2),Integer(0)], [Integer(0),Integer(1)]])
>>> AQ = matrix(QQ, [[Integer(2),Integer(0)], [Integer(0),Integer(1)]])
>>> AR = matrix(RR, [[Integer(2),Integer(0)], [Integer(0),Integer(1)]])
>>> AZ.echelon_form()
[2 0]
[0 1]
>>> AQ.echelon_form()
[1 0]
[0 1]
>>> AR.echelon_form()
[ 1.00000000000000 0.000000000000000]
[0.000000000000000  1.00000000000000]
```

Pour le calcul de valeurs propres et vecteurs propres sur les nombres à virgule flottante réels ou complexes, la matrice doit être respectivement à coefficients dans `RDF` (*Real Double Field*, nombres réels à précision machine) ou `CDF` (*Complex Double Field*). Lorsque l'on définit une matrice avec des coefficients flottants sans spécifier explicitement l'anneau de base, ce ne sont pas `RDF` ou `CDF` qui sont utilisés par défaut, mais `RR` et `CC`, sur lesquels ces calculs ne sont pas implémentés dans tous les cas :

```
sage: ARDF = matrix(RDF, [[1.2, 2], [2, 3]])
sage: ARDF.eigenvalues() # rel tol 8e-16
[-0.09317121994613098, 4.293171219946131]
sage: ACDF = matrix(CDF, [[1.2, I], [2, 3]])
sage: ACDF.eigenvectors_right()  # rel tol 3e-15
[(0.8818456983293743 - 0.8209140653434135*I, [(0.7505608183809549, -0.616145932704589␣
↪+ 0.2387941530333261*I)], 1),
(3.3181543016706256 + 0.8209140653434133*I, [(0.14559469829270957 + 0.
↪3756690858502104*I, 0.9152458258662108)], 1)]
```

```
>>> from sage.all import *
>>> ARDF = matrix(RDF, [[RealNumber('1.2'), Integer(2)], [Integer(2), Integer(3)]])
>>> ARDF.eigenvalues() # rel tol 8e-16
[-0.09317121994613098, 4.293171219946131]
>>> ACDF = matrix(CDF, [[RealNumber('1.2'), I], [Integer(2), Integer(3)]])
>>> ACDF.eigenvectors_right()  # rel tol 3e-15
```

(suite sur la page suivante)

(suite de la page précédente)

```
[(0.8818456983293743 - 0.8209140653434135*I, [(0.7505608183809549, -0.616145932704589␣
↪+ 0.2387941530333261*I)], 1),
(3.3181543016706256 + 0.8209140653434133*I, [(0.14559469829270957 + 0.
↪3756690858502104*I, 0.9152458258662108)], 1)]
```

2.10.1 Espaces de matrices

Créons l'espace $\mathrm{Mat}_{3\times 3}(\mathbf{Q})$:

```
sage: M = MatrixSpace(QQ,3)
sage: M
Full MatrixSpace of 3 by 3 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> M = MatrixSpace(QQ,Integer(3))
>>> M
Full MatrixSpace of 3 by 3 dense matrices over Rational Field
```

(Pour indiquer l'espace des matrices 3 par 4, il faudrait utiliser `MatrixSpace(QQ,3,4)`. Si le nombre de colonnes est omis, il est égal par défaut au nombre de lignes. Ainsi `MatrixSpace(QQ,3)` est un synonyme de `MatrixSpace(QQ, 3,3)`). L'espace des matrices est muni de sa base canonique :

```
sage: B = M.basis()
sage: len(B)
9
sage: B[0,1]
[0 1 0]
[0 0 0]
[0 0 0]
```

```
>>> from sage.all import *
>>> B = M.basis()
>>> len(B)
9
>>> B[Integer(0),Integer(1)]
[0 1 0]
[0 0 0]
[0 0 0]
```

Nous créons une matrice comme un élément de `M`.

```
sage: A = M(range(9)); A
[0 1 2]
[3 4 5]
[6 7 8]
```

```
>>> from sage.all import *
>>> A = M(range(Integer(9))); A
[0 1 2]
[3 4 5]
[6 7 8]
```

Puis, nous calculons sa forme échelonnée en ligne et son noyau.

```
sage: A.echelon_form()
[ 1  0 -1]
[ 0  1  2]
[ 0  0  0]
sage: A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

```
>>> from sage.all import *
>>> A.echelon_form()
[ 1  0 -1]
[ 0  1  2]
[ 0  0  0]
>>> A.kernel()
Vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[ 1 -2  1]
```

Puis nous illustrons les possibilités de calcul de matrices définies sur des corps finis :

```
sage: M = MatrixSpace(GF(2),4,8)
sage: A = M([1,1,0,0, 1,1,1,1, 0,1,0,0, 1,0,1,1,
....:        0,0,1,0, 1,1,0,1, 0,0,1,1, 1,1,1,0])
sage: A
[1 1 0 0 1 1 1 1]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 1 1 1 1 1 0]
sage: rows = A.rows()
sage: A.columns()
[(1, 0, 0, 0), (1, 1, 0, 0), (0, 0, 1, 1), (0, 0, 0, 1),
 (1, 1, 1, 1), (1, 0, 1, 1), (1, 1, 0, 1), (1, 1, 1, 0)]
sage: rows
[(1, 1, 0, 0, 1, 1, 1, 1), (0, 1, 0, 0, 1, 0, 1, 1),
 (0, 0, 1, 0, 1, 1, 0, 1), (0, 0, 1, 1, 1, 1, 1, 0)]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(GF(Integer(2)),Integer(4),Integer(8))
>>> A = M([Integer(1),Integer(1),Integer(0),Integer(0), Integer(1),Integer(1),
↪Integer(1),Integer(1), Integer(0),Integer(1),Integer(0),Integer(0), Integer(1),
↪Integer(0),Integer(1),Integer(1),
...        Integer(0),Integer(0),Integer(1),Integer(0), Integer(1),Integer(1),
↪Integer(0),Integer(1), Integer(0),Integer(0),Integer(1),Integer(1), Integer(1),
↪Integer(1),Integer(1),Integer(0)])
>>> A
[1 1 0 0 1 1 1 1]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 1 1 1 1 1 0]
>>> rows = A.rows()
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> A.columns()
[(1, 0, 0, 0), (1, 1, 0, 0), (0, 0, 1, 1), (0, 0, 0, 1),
 (1, 1, 1, 1), (1, 0, 1, 1), (1, 1, 0, 1), (1, 1, 1, 0)]
>>> rows
[(1, 1, 0, 0, 1, 1, 1, 1), (0, 1, 0, 0, 1, 0, 1, 1),
 (0, 0, 1, 0, 1, 1, 0, 1), (0, 0, 1, 1, 1, 1, 1, 0)]
```

Nous créons le sous-espace engendré sur $\mathbf{F}_2$ par les vecteurs lignes ci-dessus.

```
sage: V = VectorSpace(GF(2),8)
sage: S = V.subspace(rows)
sage: S
Vector space of degree 8 and dimension 4 over Finite Field of size 2
Basis matrix:
[1 0 0 0 0 1 0 0]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 0 1 0 0 1 1]
sage: A.echelon_form()
[1 0 0 0 0 1 0 0]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 0 1 0 0 1 1]
```

```
>>> from sage.all import *
>>> V = VectorSpace(GF(Integer(2)),Integer(8))
>>> S = V.subspace(rows)
>>> S
Vector space of degree 8 and dimension 4 over Finite Field of size 2
Basis matrix:
[1 0 0 0 0 1 0 0]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 0 1 0 0 1 1]
>>> A.echelon_form()
[1 0 0 0 0 1 0 0]
[0 1 0 0 1 0 1 1]
[0 0 1 0 1 1 0 1]
[0 0 0 1 0 0 1 1]
```

La base de S utilisée par Sage est obtenue à partir des lignes non nulles de la matrice des générateurs de S réduite sous forme échelonnée en lignes.

2.10.2 Algèbre linéaire creuse

Sage permet de travailler avec des matrices creuses sur des anneaux principaux.

```
sage: M = MatrixSpace(QQ, 100, sparse=True)
sage: A = M.random_element(density = 0.05)
sage: E = A.echelon_form()
```

```
>>> from sage.all import *
>>> M = MatrixSpace(QQ, Integer(100), sparse=True)
>>> A = M.random_element(density = RealNumber('0.05'))
>>> E = A.echelon_form()
```

L'algorithme multi-modulaire présent dans Sage fonctionne bien pour les matrices carrées (mais moins pour les autres) :

```
sage: M = MatrixSpace(QQ, 50, 100, sparse=True)
sage: A = M.random_element(density = 0.05)
sage: E = A.echelon_form()
sage: M = MatrixSpace(GF(2), 20, 40, sparse=True)
sage: A = M.random_element()
sage: E = A.echelon_form()
```

```
>>> from sage.all import *
>>> M = MatrixSpace(QQ, Integer(50), Integer(100), sparse=True)
>>> A = M.random_element(density = RealNumber('0.05'))
>>> E = A.echelon_form()
>>> M = MatrixSpace(GF(Integer(2)), Integer(20), Integer(40), sparse=True)
>>> A = M.random_element()
>>> E = A.echelon_form()
```

Notez que Python distingue les majuscules des minuscules :

```
sage: M = MatrixSpace(QQ, 10,10, Sparse=True)
Traceback (most recent call last):
...
TypeError: ...__init__() got an unexpected keyword argument 'Sparse'...
```

```
>>> from sage.all import *
>>> M = MatrixSpace(QQ, Integer(10),Integer(10), Sparse=True)
Traceback (most recent call last):
...
TypeError: ...__init__() got an unexpected keyword argument 'Sparse'...
```

2.11 Groupes finis, groupes abéliens

Sage permet de faire des calculs avec des groupes de permutation, des groupes classiques finis (tels que $SU(n, q)$), des groupes finis de matrices (avec vos propres générateurs) et des groupes abéliens (même infinis). La plupart de ces fonctionnalités est implémentée par une interface vers GAP.

Par exemple, pour créer un groupe de permutation, il suffit de donner une liste de générateurs, comme dans l'exemple suivant.

```
sage: G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
sage: G
Permutation Group with generators [(3,4), (1,2,3)(4,5)]
sage: G.order()
120
sage: G.is_abelian()
False
sage: G.derived_series()           # sortie plus ou moins aléatoire (random)
```

(suite sur la page suivante)

(suite de la page précédente)

```
[Permutation Group with generators [(1,2,3)(4,5), (3,4)],
 Permutation Group with generators [(1,5)(3,4), (1,5)(2,4), (1,3,5)]]
sage: G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
sage: G.random_element()            # sortie aléatoire (random)
(1,5,3)(2,4)
sage: print(latex(G))
\langle (3,4), (1,2,3)(4,5) \rangle
```

```
>>> from sage.all import *
>>> G = PermutationGroup(['(1,2,3)(4,5)', '(3,4)'])
>>> G
Permutation Group with generators [(3,4), (1,2,3)(4,5)]
>>> G.order()
120
>>> G.is_abelian()
False
>>> G.derived_series()            # sortie plus ou moins aléatoire (random)
[Permutation Group with generators [(1,2,3)(4,5), (3,4)],
 Permutation Group with generators [(1,5)(3,4), (1,5)(2,4), (1,3,5)]]
>>> G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
>>> G.random_element()            # sortie aléatoire (random)
(1,5,3)(2,4)
>>> print(latex(G))
\langle (3,4), (1,2,3)(4,5) \rangle
```

On peut obtenir la table des caractères (au format LaTeX) à partir de Sage :

```
sage: G = PermutationGroup([[(1,2),(3,4)], [(1,2,3)]])
sage: latex(G.character_table()) # random
\left(\begin{array}{rrrr}
1 & 1 & 1 & 1 \\
1 & -\zeta_{3} - 1 & \zeta_{3} & 1 \\
1 & \zeta_{3} & -\zeta_{3} - 1 & 1 \\
3 & 0 & 0 & -1
\end{array}\right)
```

```
>>> from sage.all import *
>>> G = PermutationGroup([[(Integer(1),Integer(2)),(Integer(3),Integer(4))],␣
↪[(Integer(1),Integer(2),Integer(3))]])
>>> latex(G.character_table()) # random
\left(\begin{array}{rrrr}
1 & 1 & 1 & 1 \\
1 & -\zeta_{3} - 1 & \zeta_{3} & 1 \\
1 & \zeta_{3} & -\zeta_{3} - 1 & 1 \\
3 & 0 & 0 & -1
\end{array}\right)
```

Sage inclut aussi les groupes classiques ou matriciels définis sur des corps finis :

```
sage: MS = MatrixSpace(GF(7), 2)
sage: gens = [MS([[1,0],[-1,1]]),MS([[1,1],[0,1]])]
sage: G = MatrixGroup(gens)
sage: G.conjugacy_classes_representatives()
(
[1 0]  [0 6]  [0 4]  [6 0]  [0 6]  [0 4]  [0 6]  [0 6]  [0 6]  [4 0]
[0 1], [1 5], [5 5], [0 6], [1 2], [5 2], [1 0], [1 4], [1 3], [0 2],

[5 0]
[0 3]
)
sage: G = Sp(4,GF(7))
sage: G
Symplectic Group of degree 4 over Finite Field of size 7
sage: G.random_element()  # élément du groupe tiré au hasard (random)
[5 5 5 1]
[0 2 6 3]
[5 0 1 0]
[4 6 3 4]
sage: G.order()
276595200
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(GF(Integer(7)), Integer(2))
>>> gens = [MS([[Integer(1),Integer(0)],[-Integer(1),Integer(1)]]),MS([[Integer(1),
↪Integer(1)],[Integer(0),Integer(1)]])]
>>> G = MatrixGroup(gens)
>>> G.conjugacy_classes_representatives()
(
[1 0]  [0 6]  [0 4]  [6 0]  [0 6]  [0 4]  [0 6]  [0 6]  [0 6]  [4 0]
[0 1], [1 5], [5 5], [0 6], [1 2], [5 2], [1 0], [1 4], [1 3], [0 2],
<BLANKLINE>
[5 0]
[0 3]
)
>>> G = Sp(Integer(4),GF(Integer(7)))
>>> G
Symplectic Group of degree 4 over Finite Field of size 7
>>> G.random_element()  # élément du groupe tiré au hasard (random)
[5 5 5 1]
[0 2 6 3]
[5 0 1 0]
[4 6 3 4]
>>> G.order()
276595200
```

On peut aussi effectuer des calculs dans des groupes abéliens (infinis ou finis) :

```
sage: F = AbelianGroup(5, [5,5,7,8,9], names='abcde')
sage: (a, b, c, d, e) = F.gens()
sage: d * b**2 * c**3
b^2*c^3*d
sage: F = AbelianGroup(3,[2]*3); F
```

(suite sur la page suivante)

(suite de la page précédente)

```
Multiplicative Abelian group isomorphic to C2 x C2 x C2
sage: H = AbelianGroup([2,3], names="xy"); H
Multiplicative Abelian group isomorphic to C2 x C3
sage: AbelianGroup(5)
Multiplicative Abelian group isomorphic to Z x Z x Z x Z x Z
sage: AbelianGroup(5).order()
+Infinity
```

```
>>> from sage.all import *
>>> F = AbelianGroup(Integer(5), [Integer(5),Integer(5),Integer(7),Integer(8),
↪Integer(9)], names='abcde')
>>> (a, b, c, d, e) = F.gens()
>>> d * b**Integer(2) * c**Integer(3)
b^2*c^3*d
>>> F = AbelianGroup(Integer(3),[Integer(2)]*Integer(3)); F
Multiplicative Abelian group isomorphic to C2 x C2 x C2
>>> H = AbelianGroup([Integer(2),Integer(3)], names="xy"); H
Multiplicative Abelian group isomorphic to C2 x C3
>>> AbelianGroup(Integer(5))
Multiplicative Abelian group isomorphic to Z x Z x Z x Z x Z
>>> AbelianGroup(Integer(5)).order()
+Infinity
```

2.12 Théorie des nombres

Sage possède des fonctionnalités étendues de théorie des nombres. Par exemple, on peut faire de l'arithmétique dans $\mathbf{Z}/N\mathbf{Z}$ comme suit :

```
sage: R = IntegerModRing(97)
sage: a = R(2) / R(3)
sage: a
33
sage: a.rational_reconstruction()
2/3
sage: b = R(47)
sage: b^20052005
50
sage: b.modulus()
97
sage: b.is_square()
True
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(97))
>>> a = R(Integer(2)) / R(Integer(3))
>>> a
33
>>> a.rational_reconstruction()
2/3
>>> b = R(Integer(47))
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> b**Integer(20052005)
50
>>> b.modulus()
97
>>> b.is_square()
True
```

Sage contient les fonctions standards de théorie des nombres. Par exemple,

```
sage: gcd(515,2005)
5
sage: factor(2005)
5 * 401
sage: c = factorial(25); c
15511210043330985984000000
sage: [valuation(c,p) for p in prime_range(2,23)]
[22, 10, 6, 3, 2, 1, 1, 1]
sage: next_prime(2005)
2011
sage: previous_prime(2005)
2003
sage: divisors(28); sum(divisors(28)); 2*28
[1, 2, 4, 7, 14, 28]
56
56
```

```
>>> from sage.all import *
>>> gcd(Integer(515),Integer(2005))
5
>>> factor(Integer(2005))
5 * 401
>>> c = factorial(Integer(25)); c
15511210043330985984000000
>>> [valuation(c,p) for p in prime_range(Integer(2),Integer(23))]
[22, 10, 6, 3, 2, 1, 1, 1]
>>> next_prime(Integer(2005))
2011
>>> previous_prime(Integer(2005))
2003
>>> divisors(Integer(28)); sum(divisors(Integer(28))); Integer(2)*Integer(28)
[1, 2, 4, 7, 14, 28]
56
56
```

Voilà qui est parfait !

La fonction `sigma(n,k)` de Sage additionne les k-ièmes puissances des diviseurs de `n` :

```
sage: sigma(28,0); sigma(28,1); sigma(28,2)
6
56
1050
```

```
>>> from sage.all import *
>>> sigma(Integer(28),Integer(0)); sigma(Integer(28),Integer(1)); sigma(Integer(28),
↪Integer(2))
6
56
1050
```

Nous illustrons à présent l'algorithme d'Euclide de recherche d'une relation de Bézout, l'indicatrice d'Euler ϕ et le théorème des restes chinois :

```
sage: d,u,v = xgcd(12,15)
sage: d == u*12 + v*15
True
sage: n = 2005
sage: inverse_mod(3,n)
1337
sage: 3 * 1337
4011
sage: prime_divisors(n)
[5, 401]
sage: phi = n*prod([1 - 1/p for p in prime_divisors(n)]); phi
1600
sage: euler_phi(n)
1600
sage: prime_to_m_part(n, 5)
401
```

```
>>> from sage.all import *
>>> d,u,v = xgcd(Integer(12),Integer(15))
>>> d == u*Integer(12) + v*Integer(15)
True
>>> n = Integer(2005)
>>> inverse_mod(Integer(3),n)
1337
>>> Integer(3) * Integer(1337)
4011
>>> prime_divisors(n)
[5, 401]
>>> phi = n*prod([Integer(1) - Integer(1)/p for p in prime_divisors(n)]); phi
1600
>>> euler_phi(n)
1600
>>> prime_to_m_part(n, Integer(5))
401
```

Voici une petite expérience concernant la conjecture de Syracuse :

```
sage: n = 2005
sage: for i in range(1000):
....:     n = 3*odd_part(n) + 1
....:     if odd_part(n)==1:
....:         print(i)
....:         break
```

(suite sur la page suivante)

(suite de la page précédente)

```
38
```

```
>>> from sage.all import *
>>> n = Integer(2005)
>>> for i in range(Integer(1000)):
...     n = Integer(3)*odd_part(n) + Integer(1)
...     if odd_part(n)==Integer(1):
...         print(i)
...         break
38
```

Et finalement un exemple d'utilisation du théorème chinois :

```
sage: x = crt(2, 1, 3, 5); x
11
sage: x % 3  # x mod 3 = 2
2
sage: x % 5  # x mod 5 = 1
1
sage: [binomial(13,m) for m in range(14)]
[1, 13, 78, 286, 715, 1287, 1716, 1716, 1287, 715, 286, 78, 13, 1]
sage: [binomial(13,m)%2 for m in range(14)]
[1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
sage: [kronecker(m,13) for m in range(1,13)]
[1, -1, 1, 1, -1, -1, -1, -1, 1, 1, -1, 1]
sage: n = 10000; sum([moebius(m) for m in range(1,n)])
-23
sage: Partitions(4).list()
[[4], [3, 1], [2, 2], [2, 1, 1], [1, 1, 1, 1]]
```

```
>>> from sage.all import *
>>> x = crt(Integer(2), Integer(1), Integer(3), Integer(5)); x
11
>>> x % Integer(3)  # x mod 3 = 2
2
>>> x % Integer(5)  # x mod 5 = 1
1
>>> [binomial(Integer(13),m) for m in range(Integer(14))]
[1, 13, 78, 286, 715, 1287, 1716, 1716, 1287, 715, 286, 78, 13, 1]
>>> [binomial(Integer(13),m)%Integer(2) for m in range(Integer(14))]
[1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1]
>>> [kronecker(m,Integer(13)) for m in range(Integer(1),Integer(13))]
[1, -1, 1, 1, -1, -1, -1, -1, 1, 1, -1, 1]
>>> n = Integer(10000); sum([moebius(m) for m in range(Integer(1),n)])
-23
>>> Partitions(Integer(4)).list()
[[4], [3, 1], [2, 2], [2, 1, 1], [1, 1, 1, 1]]
```

2.12.1 Nombres p-adiques

Le corps des nombres p-adiques est implémenté en Sage. Notez qu'une fois qu'un corps p-adique est créé, il n'est plus possible d'en changer la précision.

```
sage: K = Qp(11); K
11-adic Field with capped relative precision 20
sage: a = K(211/17); a
4 + 4*11 + 11^2 + 7*11^3 + 9*11^5 + 5*11^6 + 4*11^7 + 8*11^8 + 7*11^9
  + 9*11^10 + 3*11^11 + 10*11^12 + 11^13 + 5*11^14 + 6*11^15 + 2*11^16
  + 3*11^17 + 11^18 + 7*11^19 + O(11^20)
sage: b = K(3211/11^2); b
10*11^-2 + 5*11^-1 + 4 + 2*11 + O(11^18)
```

```
>>> from sage.all import *
>>> K = Qp(Integer(11)); K
11-adic Field with capped relative precision 20
>>> a = K(Integer(211)/Integer(17)); a
4 + 4*11 + 11^2 + 7*11^3 + 9*11^5 + 5*11^6 + 4*11^7 + 8*11^8 + 7*11^9
  + 9*11^10 + 3*11^11 + 10*11^12 + 11^13 + 5*11^14 + 6*11^15 + 2*11^16
  + 3*11^17 + 11^18 + 7*11^19 + O(11^20)
>>> b = K(Integer(3211)/Integer(11)**Integer(2)); b
10*11^-2 + 5*11^-1 + 4 + 2*11 + O(11^18)
```

Beaucoup de travail a été accompli afin d'implémenter l'anneau des entiers dans des corps p-adiques ou des corps de nombres distincts de $\mathbf{Q}$. Le lecteur intéressé est invité à poser ses questions aux experts sur le groupe Google `sage-support` pour plus de détails.

Un certain nombre de méthodes associées sont d'ores et déjà implémentées dans la classe NumberField.

```
sage: R.<x> = PolynomialRing(QQ)
sage: K = NumberField(x^3 + x^2 - 2*x + 8, 'a')
sage: K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> K = NumberField(x**Integer(3) + x**Integer(2) - Integer(2)*x + Integer(8), 'a')
>>> K.integral_basis()
[1, 1/2*a^2 + 1/2*a, a^2]
```

```
sage: K.galois_group()
Galois group 3T2 (S3) with order 6 of x^3 + x^2 - 2*x + 8
```

```
>>> from sage.all import *
>>> K.galois_group()
Galois group 3T2 (S3) with order 6 of x^3 + x^2 - 2*x + 8
```

```
sage: K.polynomial_quotient_ring()
Univariate Quotient Polynomial Ring in a over Rational Field with modulus
x^3 + x^2 - 2*x + 8
sage: K.units()
(-3*a^2 - 13*a - 13,)
```

(suite sur la page suivante)

(suite de la page précédente)

```
sage: K.discriminant()
-503
sage: K.class_group()
Class group of order 1 of Number Field in a with
defining polynomial x^3 + x^2 - 2*x + 8
sage: K.class_number()
1
```

```
>>> from sage.all import *
>>> K.polynomial_quotient_ring()
Univariate Quotient Polynomial Ring in a over Rational Field with modulus
x^3 + x^2 - 2*x + 8
>>> K.units()
(-3*a^2 - 13*a - 13,)
>>> K.discriminant()
-503
>>> K.class_group()
Class group of order 1 of Number Field in a with
defining polynomial x^3 + x^2 - 2*x + 8
>>> K.class_number()
1
```

2.13 Quelques mathématiques plus avancées

2.13.1 Géométrie algébrique

Il est possible de définir des variétés algébriques arbitraires avec Sage, mais les fonctionnalités non triviales sont parfois limitées aux anneaux sur **Q** ou sur les corps finis. Calculons par exemple la réunion de deux courbes planes affines, puis récupérons chaque courbe en tant que composante irréductible de la réunion.

```
sage: x, y = AffineSpace(2, QQ, 'xy').gens()
sage: C2 = Curve(x^2 + y^2 - 1)
sage: C3 = Curve(x^3 + y^3 - 1)
sage: D = C2 + C3
sage: D
Affine Plane Curve over Rational Field defined by
   x^5 + x^3*y^2 + x^2*y^3 + y^5 - x^3 - y^3 - x^2 - y^2 + 1
sage: D.irreducible_components()
[Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x^2 + y^2 - 1,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x^3 + y^3 - 1]
```

```
>>> from sage.all import *
>>> x, y = AffineSpace(Integer(2), QQ, 'xy').gens()
>>> C2 = Curve(x**Integer(2) + y**Integer(2) - Integer(1))
>>> C3 = Curve(x**Integer(3) + y**Integer(3) - Integer(1))
>>> D = C2 + C3
>>> D
Affine Plane Curve over Rational Field defined by
   x^5 + x^3*y^2 + x^2*y^3 + y^5 - x^3 - y^3 - x^2 - y^2 + 1
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> D.irreducible_components()
[Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x^2 + y^2 - 1,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x^3 + y^3 - 1]
```

Nous pouvons également trouver tous les points d'intersection des deux courbes en les intersectant et en calculant les composantes irréductibles.

```
sage: V = C2.intersection(C3)
sage: V.irreducible_components()
[Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   y - 1,
   x,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   y,
   x - 1,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x + y + 2,
   2*y^2 + 4*y + 3]
```

```
>>> from sage.all import *
>>> V = C2.intersection(C3)
>>> V.irreducible_components()
[Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   y - 1,
   x,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   y,
   x - 1,
 Closed subscheme of Affine Space of dimension 2 over Rational Field defined by:
   x + y + 2,
   2*y^2 + 4*y + 3]
```

Ainsi, par exemple, $(1, 0)$ et $(0, 1)$ appartiennent aux deux courbes (ce dont on pouvait directement s'apercevoir) ; il en va de même des points (quadratiques), dont la coordonnée en y satisfait à l'équation $2y^2 + 4y + 3 = 0$.

Sage peut calculer l'idéal torique de la cubique gauche dans l'espace projectif de dimension 3.

```
sage: R.<a,b,c,d> = PolynomialRing(QQ, 4)
sage: I = ideal(b^2-a*c, c^2-b*d, a*d-b*c)
sage: F = I.groebner_fan(); F
Groebner fan of the ideal:
Ideal (b^2 - a*c, c^2 - b*d, -b*c + a*d) of Multivariate Polynomial Ring
in a, b, c, d over Rational Field
sage: list(map(sorted, F.reduced_groebner_bases()))
[[-b^2 + a*c, -b*c + a*d, -c^2 + b*d],
 [-b*c + a*d, -c^2 + b*d, b^2 - a*c],
 [-c^3 + a*d^2, -c^2 + b*d, b*c - a*d, b^2 - a*c],
 [-c^2 + b*d, b*c - a*d, b^2 - a*c, c^3 - a*d^2],
 [-b^2 + a*c, -b*c + a*d, c^2 - b*d],
 [-b^3 + a^2*d, -b^2 + a*c, c^2 - b*d, b*c - a*d],
 [-b^2 + a*c, c^2 - b*d, b*c - a*d, b^3 - a^2*d],
```

(suite sur la page suivante)

(suite de la page précédente)

```
 [c^2 - b*d, b*c - a*d, b^2 - a*c]]
sage: F.polyhedralfan()
Polyhedral fan in 4 dimensions of dimension 4
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, Integer(4), names=('a', 'b', 'c', 'd',)); (a, b, c, d,) =␣
↪R._first_ngens(4)
>>> I = ideal(b**Integer(2)-a*c, c**Integer(2)-b*d, a*d-b*c)
>>> F = I.groebner_fan(); F
Groebner fan of the ideal:
Ideal (b^2 - a*c, c^2 - b*d, -b*c + a*d) of Multivariate Polynomial Ring
in a, b, c, d over Rational Field
>>> list(map(sorted, F.reduced_groebner_bases()))
[[-b^2 + a*c, -b*c + a*d, -c^2 + b*d],
 [-b*c + a*d, -c^2 + b*d, b^2 - a*c],
 [-c^3 + a*d^2, -c^2 + b*d, b*c - a*d, b^2 - a*c],
 [-c^2 + b*d, b*c - a*d, b^2 - a*c, c^3 - a*d^2],
 [-b^2 + a*c, -b*c + a*d, c^2 - b*d],
 [-b^3 + a^2*d, -b^2 + a*c, c^2 - b*d, b*c - a*d],
 [-b^2 + a*c, c^2 - b*d, b*c - a*d, b^3 - a^2*d],
 [c^2 - b*d, b*c - a*d, b^2 - a*c]]
>>> F.polyhedralfan()
Polyhedral fan in 4 dimensions of dimension 4
```

2.13.2 Courbes elliptiques

Les fonctionnalités relatives aux courbes elliptiques comprennent la plupart des fonctionnalités de PARI, l'accès aux données des tables en ligne de Cremona (ceci requiert le chargement d'une base de donnée optionnelle), les fonctionnalités de mwrank, c'est-à-dire la 2-descente avec calcul du groupe de Mordell-Weil complet, l'algorithme SEA, le calcul de toutes les isogénies, beaucoup de nouveau code pour les courbes sur $\mathbf{Q}$ et une partie du code de descente algébrique de Denis Simon.

La commande `EllipticCurve` permet de créer une courbe elliptique avec beaucoup de souplesse :

— EllipticCurve([a_1, a_2, a_3, a_4, a_6]) : renvoie la courbe elliptique

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6,$$

où les a_i's sont convertis par coercition dans le parent de a_1. Si tous les a_i ont pour parent $\mathbf{Z}$, ils sont convertis par coercition dans $\mathbf{Q}$.

— EllipticCurve([a_4, a_6]) : idem avec $a_1 = a_2 = a_3 = 0$.

— EllipticCurve(label) : Renvoie la courbe elliptique sur $\mathbf{Q}$ de la base de données de Cremona selon son nom dans la (nouvelle !) nomenclature de Cremona. Les courbes sont étiquetées par une chaîne de caractère telle que `"11a"` ou `"37b2"`. La lettre doit être en minuscule (pour faire la différence avec l'ancienne nomenclature).

— EllipticCurve(j) : renvoie une courbe elliptique de j-invariant j.

— EllipticCurve(R, [a_1, a_2, a_3, a_4, a_6]) : Crée la courbe elliptique sur l'anneau R donnée par les coefficients a_i comme ci-dessus.

Illustrons chacune de ces constructions :

```
sage: EllipticCurve([0,0,1,-1,0])
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field

sage: EllipticCurve([GF(5)(0),0,1,-1,0])
```

(suite sur la page suivante)

(suite de la page précédente)

```
Elliptic Curve defined by y^2 + y = x^3 + 4*x over Finite Field of size 5

sage: EllipticCurve([1,2])
Elliptic Curve defined by y^2  = x^3 + x + 2 over Rational Field

sage: EllipticCurve('37a')
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field

sage: EllipticCurve_from_j(1)
Elliptic Curve defined by y^2 + x*y = x^3 + 36*x + 3455 over Rational Field

sage: EllipticCurve(GF(5), [0,0,1,-1,0])
Elliptic Curve defined by y^2 + y = x^3 + 4*x over Finite Field of size 5
```

```
>>> from sage.all import *
>>> EllipticCurve([Integer(0),Integer(0),Integer(1),-Integer(1),Integer(0)])
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field

>>> EllipticCurve([GF(Integer(5))(Integer(0)),Integer(0),Integer(1),-Integer(1),
↪Integer(0)])
Elliptic Curve defined by y^2 + y = x^3 + 4*x over Finite Field of size 5

>>> EllipticCurve([Integer(1),Integer(2)])
Elliptic Curve defined by y^2  = x^3 + x + 2 over Rational Field

>>> EllipticCurve('37a')
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field

>>> EllipticCurve_from_j(Integer(1))
Elliptic Curve defined by y^2 + x*y = x^3 + 36*x + 3455 over Rational Field

>>> EllipticCurve(GF(Integer(5)), [Integer(0),Integer(0),Integer(1),-Integer(1),
↪Integer(0)])
Elliptic Curve defined by y^2 + y = x^3 + 4*x over Finite Field of size 5
```

Le couple $(0, 0)$ est un point de la courbe elliptique E définie par $y^2 + y = x^3 - x$. Pour créer ce point avec Sage, il convient de taper `E([0,0])`. Sage peut additionner des points sur une telle courbe elliptique (rappelons qu'une courbe elliptique possède une structure de groupe additif où le point à l'infini représente l'élément neutre et où trois points alignés de la courbe sont de somme nulle) :

```
sage: E = EllipticCurve([0,0,1,-1,0])
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field
sage: P = E([0,0])
sage: P + P
(1 : 0 : 1)
sage: 10*P
(161/16 : -2065/64 : 1)
sage: 20*P
(683916417/264517696 : -18784454671297/4302115807744 : 1)
sage: E.conductor()
37
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(0),Integer(0),Integer(1),-Integer(1),Integer(0)])
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x over Rational Field
>>> P = E([Integer(0),Integer(0)])
>>> P + P
(1 : 0 : 1)
>>> Integer(10)*P
(161/16 : -2065/64 : 1)
>>> Integer(20)*P
(683916417/264517696 : -18784454671297/4302115807744 : 1)
>>> E.conductor()
37
```

Les courbes elliptiques sur les nombres complexes sont paramétrées par leur j-invariant. Sage calcule le j-invariant comme suit :

```
sage: E = EllipticCurve([0,0,0,-4,2]); E
Elliptic Curve defined by y^2 = x^3 - 4*x + 2 over Rational Field
sage: E.conductor()
2368
sage: E.j_invariant()
110592/37
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(0),Integer(0),Integer(0),-Integer(4),Integer(2)]); E
Elliptic Curve defined by y^2 = x^3 - 4*x + 2 over Rational Field
>>> E.conductor()
2368
>>> E.j_invariant()
110592/37
```

Si l'on fabrique une courbe avec le même j-invariant que celui de E, elle n'est pas nécessairement isomorphe à E. Dans l'exemple suivant, les courbes ne sont pas isomorphes parce que leur conducteur est différent.

```
sage: F = EllipticCurve_from_j(110592/37)
sage: F.conductor()
37
```

```
>>> from sage.all import *
>>> F = EllipticCurve_from_j(Integer(110592)/Integer(37))
>>> F.conductor()
37
```

Toutefois, le twist de F par 2 donne une courbe isomorphe.

```
sage: G = F.quadratic_twist(2); G
Elliptic Curve defined by y^2 = x^3 - 4*x + 2 over Rational Field
sage: G.conductor()
2368
sage: G.j_invariant()
110592/37
```

```
>>> from sage.all import *
>>> G = F.quadratic_twist(Integer(2)); G
Elliptic Curve defined by y^2 = x^3 - 4*x + 2 over Rational Field
>>> G.conductor()
2368
>>> G.j_invariant()
110592/37
```

On peut calculer les coefficients a_n de la série-L ou forme modulaire $\sum_{n=0}^{\infty} a_n q^n$ attachée à une courbe elliptique. Le calcul s'effectue en utilisant la bibliothèque PARI écrite en C :

```
sage: E = EllipticCurve([0,0,1,-1,0])
sage: E.anlist(30)
[0, 1, -2, -3, 2, -2, 6, -1, 0, 6, 4, -5, -6, -2, 2, 6, -4, 0, -12, 0, -4,
 3, 10, 2, 0, -1, 4, -9, -2, 6, -12]
sage: v = E.anlist(10000)
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(0),Integer(0),Integer(1),-Integer(1),Integer(0)])
>>> E.anlist(Integer(30))
[0, 1, -2, -3, 2, -2, 6, -1, 0, 6, 4, -5, -6, -2, 2, 6, -4, 0, -12, 0, -4,
 3, 10, 2, 0, -1, 4, -9, -2, 6, -12]
>>> v = E.anlist(Integer(10000))
```

Il faut à peine quelques secondes pour calculer tous les coefficients a_n pour $n \leq 10^5$:

```
sage: %time v = E.anlist(100000)
CPU times: user 0.98 s, sys: 0.06 s, total: 1.04 s
Wall time: 1.06
```

```
>>> from sage.all import *
>>> %time v = E.anlist(Integer(100000))
CPU times: user 0.98 s, sys: 0.06 s, total: 1.04 s
Wall time: 1.06
```

Les courbes elliptiques peuvent être construites en utilisant leur nom dans la nomenclature de Cremona. Ceci charge par avance la courbe elliptique avec les informations la concernant, telles que son rang, son nombre de Tamagawa, son régulateur, etc.

```
sage: E = EllipticCurve("37b2")
sage: E
Elliptic Curve defined by y^2 + y = x^3 + x^2 - 1873*x - 31833 over Rational
Field
sage: E = EllipticCurve("389a")
sage: E
Elliptic Curve defined by y^2 + y = x^3 + x^2 - 2*x  over Rational Field
sage: E.rank()
2
sage: E = EllipticCurve("5077a")
sage: E.rank()
3
```

```
>>> from sage.all import *
>>> E = EllipticCurve("37b2")
>>> E
Elliptic Curve defined by y^2 + y = x^3 + x^2 - 1873*x - 31833 over Rational
Field
>>> E = EllipticCurve("389a")
>>> E
Elliptic Curve defined by y^2 + y = x^3 + x^2 - 2*x  over Rational Field
>>> E.rank()
2
>>> E = EllipticCurve("5077a")
>>> E.rank()
3
```

On peut aussi accéder à la base de données de Cremona directement.

```
sage: db = sage.databases.cremona.CremonaDatabase()
sage: db.curves(37)
{'a1': [[0, 0, 1, -1, 0], 1, 1], 'b1': [[0, 1, 1, -23, -50], 0, 3]}
sage: db.allcurves(37)
{'a1': [[0, 0, 1, -1, 0], 1, 1],
 'b1': [[0, 1, 1, -23, -50], 0, 3],
 'b2': [[0, 1, 1, -1873, -31833], 0, 1],
 'b3': [[0, 1, 1, -3, 1], 0, 3]}
```

```
>>> from sage.all import *
>>> db = sage.databases.cremona.CremonaDatabase()
>>> db.curves(Integer(37))
{'a1': [[0, 0, 1, -1, 0], 1, 1], 'b1': [[0, 1, 1, -23, -50], 0, 3]}
>>> db.allcurves(Integer(37))
{'a1': [[0, 0, 1, -1, 0], 1, 1],
 'b1': [[0, 1, 1, -23, -50], 0, 3],
 'b2': [[0, 1, 1, -1873, -31833], 0, 1],
 'b3': [[0, 1, 1, -3, 1], 0, 3]}
```

Les objets extraits de la base de données ne sont pas de type `EllipticCurve`, mais de simples entrées de base de données formées de quelques champs. Par défaut, Sage est distribué avec une version réduite de la base de données de Cremona qui ne contient que des informations limitées sur les courbes elliptiques de conducteur ≤ 10000. Il existe également en option une version plus complète qui contient des données étendues portant sur toute les courbes de conducteur jusqu'à 120000 (à la date d'octobre 2005). Une autre - énorme (2GB) - base de données optionnelle, fournie dans un package séparé, contient des centaines de millions de courbes elliptiques de la bases de donnée de Stein-Watkins.

2.13.3 Caractères de Dirichlet

Un *caractère de Dirichlet* est une extension d'un homomorphisme $(\mathbf{Z}/N\mathbf{Z})^* \to R^*$, pour un certain anneau R, à l'application $\mathbf{Z} \to R$ obtenue en envoyant les entiers x tels que $\gcd(N, x) > 1$ vers 0.

```
sage: G = DirichletGroup(12)
sage: G.list()
[Dirichlet character modulo 12 of conductor 1 mapping 7 |--> 1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 4 mapping 7 |--> -1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 3 mapping 7 |--> 1, 5 |--> -1,
Dirichlet character modulo 12 of conductor 12 mapping 7 |--> -1, 5 |--> -1]
```

(suite sur la page suivante)

(suite de la page précédente)

```
sage: G.gens()
(Dirichlet character modulo 12 of conductor 4 mapping 7 |--> -1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 3 mapping 7 |--> 1, 5 |--> -1)
sage: len(G)
4
```

```
>>> from sage.all import *
>>> G = DirichletGroup(Integer(12))
>>> G.list()
[Dirichlet character modulo 12 of conductor 1 mapping 7 |--> 1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 4 mapping 7 |--> -1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 3 mapping 7 |--> 1, 5 |--> -1,
Dirichlet character modulo 12 of conductor 12 mapping 7 |--> -1, 5 |--> -1]
>>> G.gens()
(Dirichlet character modulo 12 of conductor 4 mapping 7 |--> -1, 5 |--> 1,
Dirichlet character modulo 12 of conductor 3 mapping 7 |--> 1, 5 |--> -1)
>>> len(G)
4
```

Une fois le groupe créé, on crée aussitôt un élément et on calcule avec lui.

```
sage: G = DirichletGroup(21)
sage: chi = G.1; chi
Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> zeta6
sage: chi.values()
[0, 1, zeta6 - 1, 0, -zeta6, -zeta6 + 1, 0, 0, 1, 0, zeta6, -zeta6, 0, -1,
 0, 0, zeta6 - 1, zeta6, 0, -zeta6 + 1, -1]
sage: chi.conductor()
7
sage: chi.modulus()
21
sage: chi.order()
6
sage: chi(19)
-zeta6 + 1
sage: chi(40)
-zeta6 + 1
```

```
>>> from sage.all import *
>>> G = DirichletGroup(Integer(21))
>>> chi = G.gen(1); chi
Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> zeta6
>>> chi.values()
[0, 1, zeta6 - 1, 0, -zeta6, -zeta6 + 1, 0, 0, 1, 0, zeta6, -zeta6, 0, -1,
 0, 0, zeta6 - 1, zeta6, 0, -zeta6 + 1, -1]
>>> chi.conductor()
7
>>> chi.modulus()
21
>>> chi.order()
6
>>> chi(Integer(19))
```

(suite sur la page suivante)

(suite de la page précédente)

```
-zeta6 + 1
>>> chi(Integer(40))
-zeta6 + 1
```

Il est possible aussi de calculer l'action d'un groupe de Galois $\mathrm{Gal}(\mathbf{Q}(\zeta_N)/\mathbf{Q})$ sur l'un de ces caractères, de même qu'une décomposition en produit direct correspondant à la factorisation du module.

```
sage: chi.galois_orbit()
[Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> -zeta6 + 1,
 Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> zeta6]

sage: go = G.galois_orbits()
sage: [len(orbit) for orbit in go]
[1, 2, 2, 1, 1, 2, 2, 1]

sage: G.decomposition()
[Group of Dirichlet characters modulo 3 with values in Cyclotomic Field of order 6␣
↪and degree 2,
 Group of Dirichlet characters modulo 7 with values in Cyclotomic Field of order 6␣
↪and degree 2]
```

```
>>> from sage.all import *
>>> chi.galois_orbit()
[Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> -zeta6 + 1,
 Dirichlet character modulo 21 of conductor 7 mapping 8 |--> 1, 10 |--> zeta6]

>>> go = G.galois_orbits()
>>> [len(orbit) for orbit in go]
[1, 2, 2, 1, 1, 2, 2, 1]

>>> G.decomposition()
[Group of Dirichlet characters modulo 3 with values in Cyclotomic Field of order 6␣
↪and degree 2,
 Group of Dirichlet characters modulo 7 with values in Cyclotomic Field of order 6␣
↪and degree 2]
```

Construisons à present le groupe de caractères de Dirichlet modulo 20, mais à valeur dans $\mathbf{Q}(i)$:

```
sage: K.<i> = NumberField(x^2+1)
sage: G = DirichletGroup(20,K)
sage: G
Group of Dirichlet characters modulo 20 with values in Number Field in i with␣
↪defining polynomial x^2 + 1
```

```
>>> from sage.all import *
>>> K = NumberField(x**Integer(2)+Integer(1), names=('i',)); (i,) = K._first_ngens(1)
>>> G = DirichletGroup(Integer(20),K)
>>> G
Group of Dirichlet characters modulo 20 with values in Number Field in i with␣
↪defining polynomial x^2 + 1
```

Nous calculons ensuite différents invariants de `G` :

```
sage: G.gens()
(Dirichlet character modulo 20 of conductor 4 mapping 11 |--> -1, 17 |--> 1,
Dirichlet character modulo 20 of conductor 5 mapping 11 |--> 1, 17 |--> i)

sage: G.unit_gens()
(11, 17)
sage: G.zeta()
i
sage: G.zeta_order()
4
```

```
>>> from sage.all import *
>>> G.gens()
(Dirichlet character modulo 20 of conductor 4 mapping 11 |--> -1, 17 |--> 1,
Dirichlet character modulo 20 of conductor 5 mapping 11 |--> 1, 17 |--> i)

>>> G.unit_gens()
(11, 17)
>>> G.zeta()
i
>>> G.zeta_order()
4
```

Dans cet exemple, nous créons un caractère de Dirichlet à valeurs dans un corps de nombres. Nous spécifions ci-dessous explicitement le choix de la racine de l'unité par le troisième argument de la fonction `DirichletGroup`.

```
sage: x = polygen(QQ, 'x')
sage: K = NumberField(x^4 + 1, 'a'); a = K.0
sage: b = K.gen(); a == b
True
sage: K
Number Field in a with defining polynomial x^4 + 1
sage: G = DirichletGroup(5, K, a); G
Group of Dirichlet characters modulo 5 with values in the group of order 8 generated␣
↪by a in Number Field in a with defining polynomial x^4 + 1
sage: chi = G.0; chi
Dirichlet character modulo 5 of conductor 5 mapping 2 |--> a^2
sage: [(chi^i)(2) for i in range(4)]
[1, a^2, -1, -a^2]
```

```
>>> from sage.all import *
>>> x = polygen(QQ, 'x')
>>> K = NumberField(x**Integer(4) + Integer(1), 'a'); a = K.gen(0)
>>> b = K.gen(); a == b
True
>>> K
Number Field in a with defining polynomial x^4 + 1
>>> G = DirichletGroup(Integer(5), K, a); G
Group of Dirichlet characters modulo 5 with values in the group of order 8 generated␣
↪by a in Number Field in a with defining polynomial x^4 + 1
>>> chi = G.gen(0); chi
Dirichlet character modulo 5 of conductor 5 mapping 2 |--> a^2
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> [(chi**i)(Integer(2)) for i in range(Integer(4))]
[1, a^2, -1, -a^2]
```

Ici, `NumberField(x^4 + 1, 'a')` indique à Sage d'utiliser le symbole « a » dans l'affichage de ce qu'est `K` (un corps de nombre en « a » défini par le polynôme x^4+1). Le nom « a » n'est pas déclaré à ce point. Une fois que `a = K.0` (ou de manière équivalente `a = K.gen()`) est évalué, le symbole « a » représente une racine du polynôme générateur x^4+1.

2.13.4 Formes modulaires

Sage peut accomplir des calculs relatifs aux formes modulaires, notamment des calculs de dimension, d'espace de symboles modulaires, d'opérateurs de Hecke et de décomposition.

Il y a plusieurs fonctions disponibles pour calculer la dimension d'espaces de formes modulaires. Par exemple,

```
sage: from sage.modular.dims import dimension_cusp_forms
sage: dimension_cusp_forms(Gamma0(11),2)
1
sage: dimension_cusp_forms(Gamma0(1),12)
1
sage: dimension_cusp_forms(Gamma1(389),2)
6112
```

```
>>> from sage.all import *
>>> from sage.modular.dims import dimension_cusp_forms
>>> dimension_cusp_forms(Gamma0(Integer(11)),Integer(2))
1
>>> dimension_cusp_forms(Gamma0(Integer(1)),Integer(12))
1
>>> dimension_cusp_forms(Gamma1(Integer(389)),Integer(2))
6112
```

Nous illustrons ci-dessous le calcul des opérateurs de Hecke sur un espace de symboles modulaires de niveau 1 et de poids 12.

```
sage: M = ModularSymbols(1,12)
sage: M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
sage: t2 = M.T(2)
sage: t2
Hecke operator T_2 on Modular Symbols space of dimension 3 for Gamma_0(1)
of weight 12 with sign 0 over Rational Field
sage: t2.matrix()
[ -24    0    0]
[   0  -24    0]
[4860    0 2049]
sage: f = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
sage: factor(f)
(x - 2049) * (x + 24)^2
sage: M.T(11).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

```
>>> from sage.all import *
>>> M = ModularSymbols(Integer(1),Integer(12))
>>> M.basis()
([X^8*Y^2,(0,0)], [X^9*Y,(0,0)], [X^10,(0,0)])
>>> t2 = M.T(Integer(2))
>>> t2
Hecke operator T_2 on Modular Symbols space of dimension 3 for Gamma_0(1)
of weight 12 with sign 0 over Rational Field
>>> t2.matrix()
[ -24    0    0]
[   0  -24    0]
[4860    0 2049]
>>> f = t2.charpoly('x'); f
x^3 - 2001*x^2 - 97776*x - 1180224
>>> factor(f)
(x - 2049) * (x + 24)^2
>>> M.T(Integer(11)).charpoly('x').factor()
(x - 285311670612) * (x - 534612)^2
```

Nous pouvons aussi créer des espaces pour $\Gamma_0(N)$ et $\Gamma_1(N)$.

```
sage: ModularSymbols(11,2)
Modular Symbols space of dimension 3 for Gamma_0(11) of weight 2 with sign
 0 over Rational Field
sage: ModularSymbols(Gamma1(11),2)
Modular Symbols space of dimension 11 for Gamma_1(11) of weight 2 with
sign 0 over Rational Field
```

```
>>> from sage.all import *
>>> ModularSymbols(Integer(11),Integer(2))
Modular Symbols space of dimension 3 for Gamma_0(11) of weight 2 with sign
 0 over Rational Field
>>> ModularSymbols(Gamma1(Integer(11)),Integer(2))
Modular Symbols space of dimension 11 for Gamma_1(11) of weight 2 with
sign 0 over Rational Field
```

Calculons quelques polynômes caractéristiques et développements en série de Fourier.

```
sage: M = ModularSymbols(Gamma1(11),2)
sage: M.T(2).charpoly('x')
x^11 - 8*x^10 + 20*x^9 + 10*x^8 - 145*x^7 + 229*x^6 + 58*x^5 - 360*x^4
     + 70*x^3 - 515*x^2 + 1804*x - 1452
sage: M.T(2).charpoly('x').factor()
(x - 3) * (x + 2)^2 * (x^4 - 7*x^3 + 19*x^2 - 23*x + 11)
        * (x^4 - 2*x^3 + 4*x^2 + 2*x + 11)
sage: S = M.cuspidal_submodule()
sage: S.T(2).matrix()
[-2  0]
[ 0 -2]
sage: S.q_expansion_basis(10)
[q - 2*q^2 - q^3 + 2*q^4 + q^5 + 2*q^6 - 2*q^7 - 2*q^9 + O(q^10)]
```

```
>>> from sage.all import *
>>> M = ModularSymbols(Gamma1(Integer(11)),Integer(2))
>>> M.T(Integer(2)).charpoly('x')
x^11 - 8*x^10 + 20*x^9 + 10*x^8 - 145*x^7 + 229*x^6 + 58*x^5 - 360*x^4
     + 70*x^3 - 515*x^2 + 1804*x - 1452
>>> M.T(Integer(2)).charpoly('x').factor()
(x - 3) * (x + 2)^2 * (x^4 - 7*x^3 + 19*x^2 - 23*x + 11)
        * (x^4 - 2*x^3 + 4*x^2 + 2*x + 11)
>>> S = M.cuspidal_submodule()
>>> S.T(Integer(2)).matrix()
[-2  0]
[ 0 -2]
>>> S.q_expansion_basis(Integer(10))
[q - 2*q^2 - q^3 + 2*q^4 + q^5 + 2*q^6 - 2*q^7 - 2*q^9 + O(q^10)]
```

On peut même calculer des espaces de formes modulaires avec caractères.

```
sage: G = DirichletGroup(13)
sage: e = G.0^2
sage: M = ModularSymbols(e,2); M
Modular Symbols space of dimension 4 and level 13, weight 2, character
[zeta6], sign 0, over Cyclotomic Field of order 6 and degree 2
sage: M.T(2).charpoly('x').factor()
(x - zeta6 - 2) * (x - 2*zeta6 - 1) * (x + zeta6 + 1)^2
sage: S = M.cuspidal_submodule(); S
Modular Symbols subspace of dimension 2 of Modular Symbols space of
dimension 4 and level 13, weight 2, character [zeta6], sign 0, over
Cyclotomic Field of order 6 and degree 2
sage: S.T(2).charpoly('x').factor()
(x + zeta6 + 1)^2
sage: S.q_expansion_basis(10)
[q + (-zeta6 - 1)*q^2 + (2*zeta6 - 2)*q^3 + zeta6*q^4 + (-2*zeta6 + 1)*q^5 + (-
↪2*zeta6 + 4)*q^6 + (2*zeta6 - 1)*q^8 - zeta6*q^9 + O(q^10)]
```

```
>>> from sage.all import *
>>> G = DirichletGroup(Integer(13))
>>> e = G.gen(0)**Integer(2)
>>> M = ModularSymbols(e,Integer(2)); M
Modular Symbols space of dimension 4 and level 13, weight 2, character
[zeta6], sign 0, over Cyclotomic Field of order 6 and degree 2
>>> M.T(Integer(2)).charpoly('x').factor()
(x - zeta6 - 2) * (x - 2*zeta6 - 1) * (x + zeta6 + 1)^2
>>> S = M.cuspidal_submodule(); S
Modular Symbols subspace of dimension 2 of Modular Symbols space of
dimension 4 and level 13, weight 2, character [zeta6], sign 0, over
Cyclotomic Field of order 6 and degree 2
>>> S.T(Integer(2)).charpoly('x').factor()
(x + zeta6 + 1)^2
>>> S.q_expansion_basis(Integer(10))
[q + (-zeta6 - 1)*q^2 + (2*zeta6 - 2)*q^3 + zeta6*q^4 + (-2*zeta6 + 1)*q^5 + (-
↪2*zeta6 + 4)*q^6 + (2*zeta6 - 1)*q^8 - zeta6*q^9 + O(q^10)]
```

Voici un autre exemple montrant comment Sage peut calculer l'action d'un opérateur de Hecke sur un espace de formes

modulaires.

```
sage: T = ModularForms(Gamma0(11),2)
sage: T
Modular Forms space of dimension 2 for Congruence Subgroup Gamma0(11) of
weight 2 over Rational Field
sage: T.degree()
2
sage: T.level()
11
sage: T.group()
Congruence Subgroup Gamma0(11)
sage: T.dimension()
2
sage: T.cuspidal_subspace()
Cuspidal subspace of dimension 1 of Modular Forms space of dimension 2 for
Congruence Subgroup Gamma0(11) of weight 2 over Rational Field
sage: T.eisenstein_subspace()
Eisenstein subspace of dimension 1 of Modular Forms space of dimension 2
for Congruence Subgroup Gamma0(11) of weight 2 over Rational Field
sage: M = ModularSymbols(11); M
Modular Symbols space of dimension 3 for Gamma_0(11) of weight 2 with sign
0 over Rational Field
sage: M.weight()
2
sage: M.basis()
((1,0), (1,8), (1,9))
sage: M.sign()
0
```

```
>>> from sage.all import *
>>> T = ModularForms(Gamma0(Integer(11)),Integer(2))
>>> T
Modular Forms space of dimension 2 for Congruence Subgroup Gamma0(11) of
weight 2 over Rational Field
>>> T.degree()
2
>>> T.level()
11
>>> T.group()
Congruence Subgroup Gamma0(11)
>>> T.dimension()
2
>>> T.cuspidal_subspace()
Cuspidal subspace of dimension 1 of Modular Forms space of dimension 2 for
Congruence Subgroup Gamma0(11) of weight 2 over Rational Field
>>> T.eisenstein_subspace()
Eisenstein subspace of dimension 1 of Modular Forms space of dimension 2
for Congruence Subgroup Gamma0(11) of weight 2 over Rational Field
>>> M = ModularSymbols(Integer(11)); M
Modular Symbols space of dimension 3 for Gamma_0(11) of weight 2 with sign
0 over Rational Field
>>> M.weight()
```

(suite sur la page suivante)

(suite de la page précédente)

```
2
>>> M.basis()
((1,0), (1,8), (1,9))
>>> M.sign()
0
```

Notons T_p les opérateurs de Hecke usuels (p premier). Comment agissent les opérateurs de Hecke T_2, T_3, T_5 sur l'espace des symboles modulaires ?

```
sage: M.T(2).matrix()
[ 3  0 -1]
[ 0 -2  0]
[ 0  0 -2]
sage: M.T(3).matrix()
[ 4  0 -1]
[ 0 -1  0]
[ 0  0 -1]
sage: M.T(5).matrix()
[ 6  0 -1]
[ 0  1  0]
[ 0  0  1]
```

```
>>> from sage.all import *
>>> M.T(Integer(2)).matrix()
[ 3  0 -1]
[ 0 -2  0]
[ 0  0 -2]
>>> M.T(Integer(3)).matrix()
[ 4  0 -1]
[ 0 -1  0]
[ 0  0 -1]
>>> M.T(Integer(5)).matrix()
[ 6  0 -1]
[ 0  1  0]
[ 0  0  1]
```

CHAPITRE 3

La ligne de commande interactive

Dans la plus grande partie de ce tutoriel, nous supposons que vous avez lancé l'interpréteur Sage avec la commande `sage`. Cela démarre une version adaptée du shell (interpréteur de commandes) IPython et importe un grand nombre de fonctions et de classes qui sont ainsi prêtes à l'emploi depuis l'invite de commande. D'autres personnalisations sont possibles en éditant le fichier `$SAGE_ROOT/ipythonrc`. Au démarrage, le shell Sage affiche un message de ce genre :

```
┌────────────────────────────────────────────────────────────────────┐
│ SageMath version 9.7, Release Date: 2022-01-10                     │
│ Using Python 3.10.4. Type "help()" for help.                       │
└────────────────────────────────────────────────────────────────────┘


sage:
```

Pour quitter Sage, tapez Ctrl-D ou tapez `quit` ou `exit`.

```
sage: quit
Exiting Sage (CPU time 0m0.00s, Wall time 0m0.89s)
```

```
>>> from sage.all import *
>>> quit
Exiting Sage (CPU time 0m0.00s, Wall time 0m0.89s)
```

L'indication *wall time* donne le temps écoulé à votre montre (ou l'horloge suspendue au mur) pendant l'exécution. C'est une donnée pertinente car le temps processeur (*CPU time*) ne tient pas compte du temps utilisé par les sous-processus comme GAP et Singular.

(Il vaut mieux éviter de tuer un processus Sage depuis un terminal avec `kill -9`, car il est possible que Sage ne tue pas ses processus enfants, par exemple des processus Maple qu'il aurait lancés, ou encore qu'il ne nettoie pas les fichiers temporaires de `$HOME/.sage/tmp`.)

3.1 Votre session Sage

Une session est la suite des entrées et sorties qui interviennent entre le moment où vous démarrez Sage et celui où vous le quittez. Sage enregistre un journal de toutes les entrées via IPython. Si vous utilisez le shell interactif (par opposition à l'interface *jupyter*), vous pouvez taper `%history` (ou `%hist`) à n'importe quel moment pour obtenir la liste de toutes les lignes de commandes entrées depuis le début de la session. Tapez `?` à l'invite de commande Sage pour plus d'informations sur IPython. Par exemple : « IPython fournit des invites de commande numérotées […] avec un cache des entrées-sorties. Toutes les entrées sont sauvegardées et peuvent être rappelées comme variables (en plus de la navigation habituelle dans l'historique avec les flèches du clavier). Les variables GLOBALES suivantes existent toujours (ne les écrasez pas !) » :

```
_ : dernière entrée
__ : avant-dernière entrée
_oh : liste de toutes les entrées précédentes
```

Voici un exemple :

```
sage: factor(100)
 _1 = 2^2 * 5^2
sage: kronecker_symbol(3,5)
 _2 = -1
sage: %hist   #Fonctionne depuis le shell mais pas depuis le bloc-note.
1: factor(100)
2: kronecker_symbol(3,5)
3: %hist
sage: _oh
 _4 = {1: 2^2 * 5^2, 2: -1}
sage: _i1
 _5 = 'factor(ZZ(100))\n'
sage: eval(_i1)
 _6 = 2^2 * 5^2
sage: %hist
1: factor(100)
2: kronecker_symbol(3,5)
3: %hist
4: _oh
5: _i1
6: eval(_i1)
7: %hist
```

```
>>> from sage.all import *
>>> factor(Integer(100))
 _1 = 2^2 * 5^2
>>> kronecker_symbol(Integer(3),Integer(5))
 _2 = -1
>>> %hist   #Fonctionne depuis le shell mais pas depuis le bloc-note.
1: factor(100)
2: kronecker_symbol(3,5)
3: %hist
>>> _oh
 _4 = {1: 2^2 * 5^2, 2: -1}
>>> _i1
 _5 = 'factor(ZZ(100))\n'
>>> eval(_i1)
```

(suite sur la page suivante)

(suite de la page précédente)

```
 _6 = 2^2 * 5^2
>>> %hist
1: factor(100)
2: kronecker_symbol(3,5)
3: %hist
4: _oh
5: _i1
6: eval(_i1)
7: %hist
```

Dans la suite de ce tutoriel et le reste de la documentation de Sage, nous omettrons la numérotation des sorties.

Il est possible de créer (pour la durée d'une session) une macro qui rappelle une liste de plusieurs lignes d'entrée.

```
sage: E = EllipticCurve([1,2,3,4,5])
sage: M = ModularSymbols(37)
sage: %hist
1: E = EllipticCurve([1,2,3,4,5])
2: M = ModularSymbols(37)
3: %hist
sage: %macro em 1-2
Macro `em` created. To execute, type its name (without quotes).
```

```
>>> from sage.all import *
>>> E = EllipticCurve([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
>>> M = ModularSymbols(Integer(37))
>>> %hist
1: E = EllipticCurve([1,2,3,4,5])
2: M = ModularSymbols(37)
3: %hist
>>> %macro em Integer(1)-Integer(2)
Macro `em` created. To execute, type its name (without quotes).
```

```
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
sage: E = 5
sage: M = None
sage: em
Executing Macro...
sage: E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
```

```
>>> from sage.all import *
>>> E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
>>> E = Integer(5)
>>> M = None
>>> em
Executing Macro...
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> E
Elliptic Curve defined by y^2 + x*y + 3*y = x^3 + 2*x^2 + 4*x + 5 over
Rational Field
```

Depuis le shell interactif Sage, il est possible d'exécuter une commande Unix en la faisant précéder d'un point d'exclamation `!`. Par exemple,

```
sage: !ls
auto  example.sage glossary.tex  t  tmp  tut.log  tut.tex
```

```
>>> from sage.all import *
>>> !ls
auto  example.sage glossary.tex  t  tmp  tut.log  tut.tex
```

renvoie la liste des fichiers du répertoire courant.

Dans ce contexte, le `PATH` commence par le répertoire des binaires de Sage, de sorte que les commandes `gp`, `gap`, `singular`, `maxima`, etc. appellent les versions inclues dans Sage.

```
sage: !gp
Reading GPRC: /etc/gprc ...Done.

                         GP/PARI CALCULATOR Version 2.2.11 (alpha)
                 i686 running linux (ix86/GMP-4.1.4 kernel) 32-bit version
...
sage: !singular
                     SINGULAR                              /  Development
 A Computer Algebra System for Polynomial Computations    /   version 3-0-1
                                                        0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann         \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern     \
```

```
>>> from sage.all import *
>>> !gp
Reading GPRC: /etc/gprc ...Done.

                         GP/PARI CALCULATOR Version 2.2.11 (alpha)
                 i686 running linux (ix86/GMP-4.1.4 kernel) 32-bit version
...
>>> !singular
                     SINGULAR                              /  Development
 A Computer Algebra System for Polynomial Computations    /   version 3-0-1
                                                        0<
     by: G.-M. Greuel, G. Pfister, H. Schoenemann         \   October 2005
FB Mathematik der Universitaet, D-67653 Kaiserslautern     \
```

3.2 Journal des entrées-sorties

Enregistrer le journal d'une session Sage n'est pas la même chose que sauvegarder la session (voir *Enregister et recharger des sessions entières* pour cette possibilité). Pour tenir un journal des entrées (et optionnellement des sorties) de Sage, utilisez la commande `logstart`. Tapez `logsatart?` pour plus d'informations. Cette commande permet d'enregistrer

toutes les entrées que vous tapez, toutes les sorties, et de rejouer ces entrées dans une session future (en rechargeant le fichier journal).

```
was@form:~$ sage
┌────────────────────────────────────────────────────────────────────┐
│ SageMath version 9.7, Release Date: 2022-01-10                     │
│ Using Python 3.10.4. Type "help()" for help.                       │
└────────────────────────────────────────────────────────────────────┘

sage: logstart setup
Activating auto-logging. Current session state plus future input saved.
Filename       : setup
Mode           : backup
Output logging : False
Timestamping   : False
State          : active
sage: E = EllipticCurve([1,2,3,4,5]).minimal_model()
sage: F = QQ^3
sage: x,y = QQ['x,y'].gens()
sage: G = E.gens()
sage:
Exiting Sage (CPU time 0m0.61s, Wall time 0m50.39s).
was@form:~$ sage
┌────────────────────────────────────────────────────────────────────┐
│ SageMath version 9.7, Release Date: 2022-01-10                     │
│ Using Python 3.10.4. Type "help()" for help.                       │
└────────────────────────────────────────────────────────────────────┘

sage: load("setup")
Loading log file <setup> one line at a time...
Finished replaying log file <setup>
sage: E
Elliptic Curve defined by y^2 + x*y  = x^3 - x^2 + 4*x + 3 over Rational
Field
sage: x*y
x*y
sage: G
[(2 : 3 : 1)]
```

Si vous utilisez le terminal Konsole de KDE, vous pouvez aussi sauver votre session comme suit : après avoir lancé Sage dans la `konsole`, ouvrez le menu « Configuration » et choisissez « Historique… » puis comme nombre de lignes « Illimité ». Ensuite, lorsque vous souhaitez enregistrer l'état de votre session, sélectionnez « Enregistrer l'historique sous… » dans le menu « Édition » et entrez le nom d'un fichier où enregistrer le texte de votre session. Une fois le fichier sauvegardé, vous pouvez par exemple l'ouvrir dans un éditeur comme xemacs et l'imprimer.

3.3 Coller du texte ignore les invites

Imaginons que vous lisiez une session Sage ou Python et que vous vouliez copier-coller les calculs dans Sage. Le problème est qu'il y a des invites `>>>` ou `sage:` en plus des entrées. En fait, vous pouvez tout à fait copier un exemple complet, invites comprises : par défaut, l'analyseur syntaxique de Sage supprime les `>>>` et `sage:` en début de ligne avant de passer la ligne à Python. Par exemple, les lignes suivantes sont interprétées correctement :

```
sage: 2^10
1024
sage: sage: sage: 2^10
1024
sage: >>> 2^10
1024
```

```
>>> from sage.all import *
>>> Integer(2)**Integer(10)
1024
>>> sage: sage: Integer(2)**Integer(10)
1024
>>> >>> Integer(2)**Integer(10)
1024
```

3.4 Mesure du temps d'exécution d'une commande

Si une ligne d'entrée commence par `%time`, le temps d'exécution de la commande correspondante est affiché après la sortie. Nous pouvons par exemple comparer le temps que prend le calcul d'une certaine puissance entière par diverses méthodes. Les temps de calcul ci-dessous seront sans doute très différents suivant l'ordinateur, voire la version de Sage utilisés. Premièrement, en pur Python :

```
sage: %time a = int(1938)^int(99484)
CPU times: user 0.66 s, sys: 0.00 s, total: 0.66 s
Wall time: 0.66
```

```
>>> from sage.all import *
>>> %time a = int(Integer(1938))**int(Integer(99484))
CPU times: user 0.66 s, sys: 0.00 s, total: 0.66 s
Wall time: 0.66
```

Le calcul a pris 0.66 seconde, pendant un intervalle de *wall time* (le temps de votre montre) lui aussi de 0.66 seconde. Si d'autres programmes qui s'exécutent en même temps que Sage chargent l'ordinateur avec de gros calculs, le *wall time* peut être nettement plus important que le temps processeur.

Chronométrons maintenant le calcul de la même puissance avec le type `Integer` de Sage, qui est implémenté (en Cython) en utilisant la bibliothèque GMP :

```
sage: %time a = 1938^99484
CPU times: user 0.04 s, sys: 0.00 s, total: 0.04 s
Wall time: 0.04
```

```
>>> from sage.all import *
>>> %time a = Integer(1938)**Integer(99484)
CPU times: user 0.04 s, sys: 0.00 s, total: 0.04 s
Wall time: 0.04
```

Avec l'interface à la bibliothèque C PARI :

```
sage: %time a = pari(1938)^pari(99484)
CPU times: user 0.05 s, sys: 0.00 s, total: 0.05 s
Wall time: 0.05
```

```
>>> from sage.all import *
>>> %time a = pari(Integer(1938))**pari(Integer(99484))
CPU times: user 0.05 s, sys: 0.00 s, total: 0.05 s
Wall time: 0.05
```

GMP est plus rapide, mais de peu (ce n'est pas une surprise, car la version de PARI incluse dans Sage utilise GMP pour l'arithmétique entière).

Il est aussi possible de chronométrer tout un bloc de commandes avec la commande `cputime`, comme dans l'exemple suivant :

```
sage: t = cputime()
sage: a = int(1938)^int(99484)
sage: b = 1938^99484
sage: c = pari(1938)^pari(99484)
sage: cputime(t)                          #random
0.64
```

```
>>> from sage.all import *
>>> t = cputime()
>>> a = int(Integer(1938))**int(Integer(99484))
>>> b = Integer(1938)**Integer(99484)
>>> c = pari(Integer(1938))**pari(Integer(99484))
>>> cputime(t)                          #random
0.64
```

```
sage: cputime?
...
    Return the time in CPU second since Sage started, or with optional
    argument t, return the time since time t.
    INPUT:
        t -- (optional) float, time in CPU seconds
    OUTPUT:
        float -- time in CPU seconds
```

```
>>> from sage.all import *
>>> cputime?
...
    Return the time in CPU second since Sage started, or with optional
    argument t, return the time since time t.
    INPUT:
        t -- (optional) float, time in CPU seconds
    OUTPUT:
        float -- time in CPU seconds
```

La commande `walltime` fonctionne comme `cputime`, à ceci près qu'elle mesure le temps total écoulé « à la montre ».

Nous pouvons aussi faire faire le calcul de puissance ci-dessus à chacun des systèmes de calcul formel inclus dans Sage. Dans chaque cas, nous commençons par lancer une commande triviale dans le système en question, de façon à démarrer son serveur. La mesure la plus pertinente est le *wall time*. Cependant, si la différence entre celui-ci et le temps processeur est importante, cela peut indiquer un problème de performance qui mérite d'être examiné.

```
sage: time 1938^99484;
CPU times: user 0.01 s, sys: 0.00 s, total: 0.01 s
Wall time: 0.01
sage: gp(0)
0
sage: time g = gp('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.04
sage: maxima(0)
0
sage: time g = maxima('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.30
sage: kash(0)
0
sage: time g = kash('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.04
sage: mathematica(0)
        0
sage: time g = mathematica('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.03
sage: maple(0)
0
sage: time g = maple('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.11
sage: libgap(0)
0
sage: time g = libgap.eval('1938^99484;')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 1.02
```

```
>>> from sage.all import *
>>> time Integer(1938)**Integer(99484);
CPU times: user 0.01 s, sys: 0.00 s, total: 0.01 s
Wall time: 0.01
>>> gp(Integer(0))
0
>>> time g = gp('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.04
>>> maxima(Integer(0))
0
>>> time g = maxima('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.30
>>> kash(Integer(0))
0
>>> time g = kash('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
```

(suite sur la page suivante)

(suite de la page précédente)

```
Wall time: 0.04
>>> mathematica(Integer(0))
        0
>>> time g = mathematica('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.03
>>> maple(Integer(0))
0
>>> time g = maple('1938^99484')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 0.11
>>> libgap(Integer(0))
0
>>> time g = libgap.eval('1938^99484;')
CPU times: user 0.00 s, sys: 0.00 s, total: 0.00 s
Wall time: 1.02
```

Nous voyons que GAP et Maxima sont les plus lents sur ce test (lancé sur la machine `sage.math.washington.edu`). Mais en raison du surcoût de l'interface pexpect, la comparaison avec Sage, qui est le plus rapide, n'est pas vraiment équitable.

3.5 Trucs et astuces IPython

Comme signalé plus haut, Sage utilise l'interpréteur de commandes IPython, et met donc à votre disposition toutes les commandes et fonctionnalités de celui-ci. Vous voudrez peut-être consulter la documentation complète de IPython. Voici en attendant quelques astuces utiles – qui reposent sur ce que IPython appelle des « commandes magiques » :

— Lorsque l'on souhaite saisir un morceau de code complexe, on peut utiliser `%edit` (ou `%ed`, ou `ed`) pour ouvrir un éditeur de texte. Assurez-vous que la variable d'environnement `EDITOR` est réglée à votre éditeur favori au démarrage de Sage (en plaçant si nécessaire quelque chose du genre `export EDITOR=/usr/bin/emacs` ou encore `export EDITOR=/usr/bin/vim` dans un fichier de configuration convenable, par exemple `.profile`). La commande `%edit` à l'invite de Sage ouvrira l'éditeur sélectionné. Vous pouvez alors par exemple saisir une définition de fonction :

```
def some_function(n):
    return n**2 + 3*n + 2
```

puis enregistrer le fichier et quitter l'éditeur. La fonction `some_function` est désormais disponible dans votre session Sage, et vous pouvez la modifier en saisissant `edit some_function` à l'invite de commande.

— Si vous souhaitez reprendre une version modifiée du résultat d'un calcul dans une nouvelle commande, tapez `%rep` après avoir fait le calcul. Cela récupère le texte du résultat et le place sur la ligne de commande, prêt à être modifié.

```
sage: f(x) = cos(x)
sage: f(x).derivative(x)
-sin(x)
```

```
>>> from sage.all import *
>>> __tmp__=var("x"); f = symbolic_expression(cos(x)).function(x)
>>> f(x).derivative(x)
-sin(x)
```

Ainsi, après les commandes ci-dessus, la commande `%rep` fournit un nouvel invite de commande pré-rempli avec le texte `-sin(x)` et le curseur en fin de ligne.

Pour plus d'information, entrez la commande `%quickref` pour un résumé des possibilités de IPython. Au moment où cette documentation est écrite (avril 2011), Sage emploie IPython 0.9.1. La documentation des commandes magiques est disponible en ligne, et divers aspects un peu plus avancés de leur fonctionnement sont décrits ici.

3.6 Erreurs et exceptions

Quand quelque chose ne marche pas, cela se manifeste habituellement par une « exception » Python. Python essaie de plus de donner une idée de ce qui a pu déclencher l'exception. Bien souvent, il affiche le nom de l'exception (par exemple `NameError` ou `ValueError`, voir le manuel de référence de la bibliothèque de Python [PyLR] pour une liste complète). Par exemple :

```
sage: 3_2
------------------------------------------------------------
   File "<console>", line 1
     ZZ(3)_2
           ^
SyntaxError: invalid ...

sage: EllipticCurve([0,infinity])
------------------------------------------------------------
Traceback (most recent call last):
...
TypeError: Unable to coerce Infinity (<class 'sage...Infinity'>) to Rational
```

```
>>> from sage.all import *
>>> Integer(3_2)
------------------------------------------------------------
   File "<console>", line 1
     ZZ(3)_2
           ^
SyntaxError: invalid ...

>>> EllipticCurve([Integer(0),infinity])
------------------------------------------------------------
Traceback (most recent call last):
...
TypeError: Unable to coerce Infinity (<class 'sage...Infinity'>) to Rational
```

Le débogueur interactif est parfois utile pour comprendre ce qu'il s'est passé. Il s'active ou se désactive avec `%pdb` (et est désactivé par défaut). L'invite `ipdb>>` du débogueur apparaît si une exception a lieu alors que celui-ci est actif. Le débogueur permet d'afficher l'état de n'importe quelle variable locale et de monter ou descendre dans la pile d'exécution. Par exemple :

```
sage: %pdb
Automatic pdb calling has been turned ON
sage: EllipticCurve([1,infinity])
---------------------------------------------------------------------------
<class 'exceptions.TypeError'>             Traceback (most recent call last)
...

ipdb>
```

```
>>> from sage.all import *
>>> %pdb
Automatic pdb calling has been turned ON
>>> EllipticCurve([Integer(1),infinity])
---------------------------------------------------------------------------
<class 'exceptions.TypeError'>              Traceback (most recent call last)
...

ipdb>
```

Pour obtenir une liste des commandes disponibles dans le débogueur, tapez `?` à l'invite `ipdb>` :

```
ipdb> ?

Documented commands (type help <topic>):
========================================
EOF    break  commands  debug    h       l     pdef   quit    tbreak
a      bt     condition  disable  help    list  pdoc   r       u
alias  c      cont       down     ignore  n     pinfo  return  unalias
args   cl     continue   enable   j       next  pp     s       up
b      clear  d          exit     jump    p     q      step    w
whatis where

Miscellaneous help topics:
==========================
exec  pdb

Undocumented commands:
======================
retval  rv
```

Tapez Ctrl-D ou `quit` pour revenir à Sage.

3.7 Recherche en arrière et complétion de ligne de commande

Commençons par créer l'espace vectoriel de dimension trois $V = \mathbf{Q}^3$ comme suit :

```
sage: V = VectorSpace(QQ,3)
sage: V
Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> V = VectorSpace(QQ,Integer(3))
>>> V
Vector space of dimension 3 over Rational Field
```

Nous pouvons aussi utiliser la variante plus concise :

```
sage: V = QQ^3
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3)
```

Tapez ensuite le début d'une commande, puis `Ctrl-p` (ou flèche vers le haut) pour passer en revue les lignes qui commencent par les mêmes lettres parmi celles que vous avez entrées jusque-là. Cela fonctionne même si vous avez quitté et relancé Sage entre-temps. Vous pouvez aussi rechercher une portion de commande en remontant dans l'historique avec `Ctrl-r`. Toutes ces fonctionnalités reposent sur la bibliothèque `readline`, qui existe pour la plupart des variantes de Linux.

La complétion de ligne de commande permet d'obtenir facilement la liste des fonctions membres de V : tapez simplement `V.` puis appuyez sur la touche tabulation.

```
sage: V.[tab key]
V._VectorSpace_generic__base_field
...
V.ambient_space
V.base_field
V.base_ring
V.basis
V.coordinates
...
V.zero_vector
```

```
>>> from sage.all import *
>>> V.[tab key]
V._VectorSpace_generic__base_field
...
V.ambient_space
V.base_field
V.base_ring
V.basis
V.coordinates
...
V.zero_vector
```

Si vous tapez les quelques premières lettres d'un nom de fonction avant d'appuyer sur `tab`, vous n'obtiendrez que les fonctions qui commencent par ces quelques lettres :

```
sage: V.i[tab key]
V.is_ambient  V.is_dense    V.is_full     V.is_sparse
```

```
>>> from sage.all import *
>>> V.i[tab key]
V.is_ambient  V.is_dense    V.is_full     V.is_sparse
```

Si vous cherchez à savoir ce que fait une fonction, par exemple la fonction coordinates, `V.coordinates?` affiche un message d'aide et `V.coordinates??` le code source de la fonction, comme expliqué dans la section suivante.

3.8 Aide en ligne

Sage dispose d'un système d'aide intégré. Pour obtenir la documentation d'une fonction, tapez son nom suivi d'un point d'interrogation.

```
sage: V = QQ^3
sage: V.coordinates?
Type:           instancemethod
```

(suite sur la page suivante)

(suite de la page précédente)

```
Base Class:     <class 'instancemethod'>
String Form:    <bound method FreeModule_ambient_field.coordinates of Vector
space of dimension 3 over Rational Field>
Namespace:      Interactive
File:           /home/was/s/local/lib/python2.4/site-packages/sage/modules/f
ree_module.py
Definition:     V.coordinates(self, v)
Docstring:
    Write v in terms of the basis for self.

    Returns a list c such that if B is the basis for self, then

            sum c_i B_i = v.

    If v is not in self, raises an ArithmeticError exception.

    EXAMPLES:
        sage: M = FreeModule(IntegerRing(), 2); M0,M1=M.gens()
        sage: W = M.submodule([M0 + M1, M0 - 2*M1])
        sage: W.coordinates(2*M0-M1)
        [2, -1]
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3)
>>> V.coordinates?
Type:           instancemethod
Base Class:     <class 'instancemethod'>
String Form:    <bound method FreeModule_ambient_field.coordinates of Vector
space of dimension 3 over Rational Field>
Namespace:      Interactive
File:           /home/was/s/local/lib/python2.4/site-packages/sage/modules/f
ree_module.py
Definition:     V.coordinates(self, v)
Docstring:
    Write v in terms of the basis for self.

    Returns a list c such that if B is the basis for self, then

            sum c_i B_i = v.

    If v is not in self, raises an ArithmeticError exception.

    EXAMPLES:
>>> M = FreeModule(IntegerRing(), Integer(2)); M0,M1=M.gens()
>>> W = M.submodule([M0 + M1, M0 - Integer(2)*M1])
>>> W.coordinates(Integer(2)*M0-M1)
        [2, -1]
```

Comme nous pouvons le voir ci-dessus, la sortie indique le type de l'objet, le nom du fichier où il est défini, et donne une description de l'effet de la fonction, avec des exemples que vous pouvez copier dans votre session Sage. Pratiquement tous ces exemples sont automatiquement testés régulièrement pour s'assurer qu'ils se comportent exactement comme indiqué.

Une autre fonctionnalité, nettement dans l'esprit du caractère ouvert de Sage, est que lorsque `f` est une fonction Python,

taper `f??` affiche son code source. Par exemple,

```
sage: V = QQ^3
sage: V.coordinates??
Type:           instancemethod
...
Source:
def coordinates(self, v):
        """
        Write $v$ in terms of the basis for self.
        ...
        """
        return self.coordinate_vector(v).list()
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3)
>>> V.coordinates??
Type:           instancemethod
...
Source:
def coordinates(self, v):
        """
        Write $v$ in terms of the basis for self.
        ...
        """
        return self.coordinate_vector(v).list()
```

Nous voyons que la fonction `coordinates` ne fait qu'appeler `coordinate_vector` et transformer le résultat en une liste. Mais alors, que fait la fonction `coordinate_vector` ?

```
sage: V = QQ^3
sage: V.coordinate_vector??
...
def coordinate_vector(self, v):
        ...
        return self.ambient_vector_space()(v)
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3)
>>> V.coordinate_vector??
...
def coordinate_vector(self, v):
        ...
        return self.ambient_vector_space()(v)
```

La fonction `coordinate_vector` convertit son entrée en un élément de l'espace ambiant, ce qui a pour effet de calculer le vecteur des coefficients de v dans V. L'espace V est déjà « l'espace ambiant » puisque c'est simplement $\mathbf{Q}^3$. Il y a aussi une fonction `coordinate_vector` différente pour les sous-espaces. Créons un sous-espace et examinons-là :

```
sage: V = QQ^3; W = V.span_of_basis([V.0, V.1])
sage: W.coordinate_vector??
...
def coordinate_vector(self, v):
```

(suite sur la page suivante)

(suite de la page précédente)

```
        """
         ...
        """
        # First find the coordinates of v wrt echelon basis.
        w = self.echelon_coordinate_vector(v)
        # Next use transformation matrix from echelon basis to
        # user basis.
        T = self.echelon_to_user_matrix()
        return T.linear_combination_of_rows(w)
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3); W = V.span_of_basis([V.gen(0), V.gen(1)])
>>> W.coordinate_vector??
...
def coordinate_vector(self, v):
        """
         ...
        """
        # First find the coordinates of v wrt echelon basis.
        w = self.echelon_coordinate_vector(v)
        # Next use transformation matrix from echelon basis to
        # user basis.
        T = self.echelon_to_user_matrix()
        return T.linear_combination_of_rows(w)
```

(Si vous pensez que cette implémentation est inefficace, venez nous aider à optimiser l'algèbre linéaire !)

Vous pouvez aussi taper `help(commande)` ou `help(classe)` pour appeler une sorte de page de manuel relative à une commande ou une classe.

```
sage: help(VectorSpace)
Help on function VectorSpace in module sage.modules.free_module:

VectorSpace(K, dimension_or_basis_keys=None, sparse=False, inner_product_matrix=None,␣
↪*,
            with_basis='standard', dimension=None, basis_keys=None, **args)
EXAMPLES:

The base can be complicated, as long as it is a field.

::

    sage: V = VectorSpace(FractionField(PolynomialRing(ZZ,'x')),3)
    sage: V
    Vector space of dimension 3 over Fraction Field of Univariate Polynomial Ring in x
     over Integer Ring
    sage: V.basis()
    [
    (1, 0, 0),
    (0, 1, 0),
--More--
```

```
>>> from sage.all import *
>>> help(VectorSpace)
Help on function VectorSpace in module sage.modules.free_module:

VectorSpace(K, dimension_or_basis_keys=None, sparse=False, inner_product_matrix=None,␣
↪*,
            with_basis='standard', dimension=None, basis_keys=None, **args)
EXAMPLES:

The base can be complicated, as long as it is a field.

::

>>> V = VectorSpace(FractionField(PolynomialRing(ZZ,'x')),Integer(3))
>>> V
    Vector space of dimension 3 over Fraction Field of Univariate Polynomial Ring in x
     over Integer Ring
>>> V.basis()
    [
    (1, 0, 0),
    (0, 1, 0),
--More--
```

Pour quitter la page d'aide, appuyez sur `q`. Votre session revient à l'écran comme elle était : contrairement à la sortie de `fonction?`, celle de `help` n'encombre pas votre session. Une possibilité particulièrement utile est de consulter l'aide d'un module entier avec `help(nom_du_module`. Par exemple, les espaces vectoriels sont définis dans `sage.modules.free_module`, et on accède à la documentation de ce module en tapant `help(sage.modules.free_module)`. Lorsque vous lisez une page de documentation avec la commande `help`, vous pouvez faire des recherches en avant en tapant `/` et en arrière en tapant `?`.

3.9 Enregistrer et charger des objets individuellement

Imaginons que nous calculions une matrice, ou pire, un espace compliqué de symboles modulaires, et que nous souhaitions les sauvegarder pour un usage futur. Les systèmes de calcul formel ont différentes approches pour permettre cela.

1. **Sauver la partie :** il n'est possible de sauver que la session entière (p.ex. GAP, Magma).
2. **Format d'entrée/sortie unifié :** chaque objet est affiché sous une forme qui peut être relue (GP/PARI).
3. **Eval :** permettre d'évaluer facilement du code arbitraire dans l'interpréteur (p.ex. Singular, PARI).

Utilisant Python, Sage adopte une approche différente, à savoir que tous les objets peuvent être sérialisés, i.e. transformés en chaînes de caractères à partir desquelles ils peuvent être reconstruits. C'est une méthode semblable dans l'esprit à l'unification des entrées et sorties de PARI, avec l'avantage que l'affichage normal des objets n'a pas besoin d'être trop compliqué. En outre, cette fonction de sauvegarde et de relecture des objets ne nécessite (dans la plupart des cas) aucune programmation supplémentaire : il s'agit simplement une fonctionnalité de Python fournie par le langage depuis la base.

Quasiment n'importe quel objet Sage `x` peut être enregistré sur le disque, dans un format compressé, avec `save(x, nom_de_fichier)` (ou dans bien des cas `x.save(nom_de_fichier)`). Pour recharger les objets, on utilise `load(nom_de_fichier)`.

```
sage: A = MatrixSpace(QQ,3)(range(9))^2
sage: A
[ 15  18  21]
[ 42  54  66]
```

(suite sur la page suivante)

(suite de la page précédente)

```
[ 69  90 111]
sage: save(A, 'A')
```

```
>>> from sage.all import *
>>> A = MatrixSpace(QQ,Integer(3))(range(Integer(9)))**Integer(2)
>>> A
[ 15  18  21]
[ 42  54  66]
[ 69  90 111]
>>> save(A, 'A')
```

Quittez puis redémarrez maintenant Sage. Vous pouvez récupérer `A` :

```
sage: A = load('A')
sage: A
[ 15  18  21]
[ 42  54  66]
[ 69  90 111]
```

```
>>> from sage.all import *
>>> A = load('A')
>>> A
[ 15  18  21]
[ 42  54  66]
[ 69  90 111]
```

Vous pouvez faire de même avec des objets plus compliqués, par exemple des courbes elliptiques. Toute l'information en cache sur l'objet est stockée avec celui-ci :

```
sage: E = EllipticCurve('11a')
sage: v = E.anlist(100000)               # prend un moment
sage: save(E, 'E')
sage: quit
```

```
>>> from sage.all import *
>>> E = EllipticCurve('11a')
>>> v = E.anlist(Integer(100000))               # prend un moment
>>> save(E, 'E')
>>> quit
```

Ainsi, la version sauvegardée de `E` prend 153 kilo-octets car elle contient les 100000 premiers a_n.

```
~/tmp$ ls -l E.sobj
-rw-r--r--  1 was was 153500 2006-01-28 19:23 E.sobj
~/tmp$ sage [...]
sage: E = load('E')
sage: v = E.anlist(100000)               # instantané !
```

(En Python, les sauvegardes et rechargements s'effectuent à l'aide du module `pickle`. En particulier, on peut sauver un objet Sage `x` par la commande `pickle.dumps(x, 2)`. Attention au `2` !)

Sage n'est pas capable de sauvegarder les objets créés dans d'autres systèmes de calcul formel comme GAP, Singular, Maxima etc. : au rechargement, ils sont dans un état marqué « invalide ». Concernant GAP, un certain nombre d'objets

sont affichés sous une forme qui permet de les reconstruire, mais d'autres non, aussi la reconstruction d'objets GAP à partir de leur affichage est intentionnellement interdite.

```
sage: a = libgap(2)
sage: a.save('a')
sage: load('a')
Traceback (most recent call last):
...
ValueError: The session in which this object was defined is no longer
running.
```

```
>>> from sage.all import *
>>> a = libgap(Integer(2))
>>> a.save('a')
>>> load('a')
Traceback (most recent call last):
...
ValueError: The session in which this object was defined is no longer
running.
```

Les objets GP/PARI, en revanche, peuvent être sauvegardés et rechargés, puisque la forme imprimée d'un objet suffit à reconstruire celui-ci.

```
sage: a = gp(2)
sage: a.save('a')
sage: load('a')
2
```

```
>>> from sage.all import *
>>> a = gp(Integer(2))
>>> a.save('a')
>>> load('a')
2
```

Un objet sauvegardé peut être rechargé y compris sur un ordinateur doté d'une architecture ou d'un système d'exploitation différent. Ainsi, il est possible de sauvegarder une immense matrice sur un OS-X 32 bits, la recharger sur un Linux 64 bits, l'y mettre en forme échelon et rapatrier le résultat. Bien souvent, un objet peut même être rechargé avec une version de Sage différente de celle utilisée pour le sauver, pourvu que le code qui gère cet objet n'ait pas trop changé d'une version sur l'autre. Sauver un objet enregistre tous ses attributs ainsi que la classe à laquelle il appartient (mais pas son code source). Si cette classe n'existe plus dans une version ultérieure de Sage, l'objet ne peut pas y être rechargé. Mais il demeure possible de le charger dans l'ancienne version pour récupérer son dictionnaire (avec `x.__dict__`), sauver celui-ci, et le recharger dans la nouvelle version.

3.9.1 Enregistrer un objet comme texte

Une autre possibilité consiste à sauvegarder la représentation texte ASCII dans un fichier texte brut, ce qui se fait simplement en ouvrant le fichier en écriture et en y écrivant la représentation de l'objet (il est tout à fait possible d'écrire plusieurs objets). Une fois l'écriture terminée, nous refermons le fichier.

```
sage: R.<x,y> = PolynomialRing(QQ,2)
sage: f = (x+y)^7
sage: o = open('file.txt','w')
sage: o.write(str(f))
sage: o.close()
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = R._first_ngens(2)
>>> f = (x+y)**Integer(7)
>>> o = open('file.txt','w')
>>> o.write(str(f))
>>> o.close()
```

3.10 Enregister et recharger des sessions entières

Sage dispose de fonctions très souples de sauvegarde et relecture de sessions entières.

La commande `save_session(nom_de_session)` enregistre toutes les variables définies dans la session courante sous forme de dictionnaire dans le fichier `nom_de_session.sobj`. (Les éventuelles variables qui ne supportent pas la sauvegarde sont ignorées.) Le fichier `.sobj` obtenu peut être rechargé comme n'importe quel objet sauvegardé ; on obtient en le rechargeant un dictionnaire dont les clés sont les noms de variables et les valeurs les objets correspondants.

La commande `reload_session(nom_de_session)` charge toutes les variables sauvées dans `nom_de_session`. Cela n'efface pas les variables déjà définies dans la session courante : les deux sessions sont fusionnées.

Commençons par démarrer Sage et par définir quelques variables.

```
sage: E = EllipticCurve('11a')
sage: M = ModularSymbols(37)
sage: a = 389
sage: t = M.T(2003).matrix(); t.charpoly().factor()
 _4 = (x - 2004) * (x - 12)^2 * (x + 54)^2
```

```
>>> from sage.all import *
>>> E = EllipticCurve('11a')
>>> M = ModularSymbols(Integer(37))
>>> a = Integer(389)
>>> t = M.T(Integer(2003)).matrix(); t.charpoly().factor()
 _4 = (x - 2004) * (x - 12)^2 * (x + 54)^2
```

Nous sauvons maintenant notre session, ce qui a pour effet d'enregistrer dans un même fichier toutes les variables ci-dessus. Nous pouvons constater que le fichier fait environ 3 ko.

```
sage: save_session('misc')
Saving a
Saving M
Saving t
Saving E
sage: quit
was@form:~/tmp$ ls -l misc.sobj
-rw-r--r--  1 was was 2979 2006-01-28 19:47 misc.sobj
```

```
>>> from sage.all import *
>>> save_session('misc')
Saving a
Saving M
Saving t
Saving E
>>> quit
```

(suite sur la page suivante)

(suite de la page précédente)

```
was@form:~/tmp$ ls -l misc.sobj
-rw-r--r--  1 was was 2979 2006-01-28 19:47 misc.sobj
```

Enfin, nous redémarrons Sage, nous définissons une nouvelle variable, et nous rechargeons la session précédente.

```
sage: b = 19
sage: load_session('misc')
Loading a
Loading M
Loading E
Loading t
```

```
>>> from sage.all import *
>>> b = Integer(19)
>>> load_session('misc')
Loading a
Loading M
Loading E
Loading t
```

Toutes les variables sauvegardées sont à nouveau disponibles. En outre, la variable `b` n'a pas été écrasée.

```
sage: M
Full Modular Symbols space for Gamma_0(37) of weight 2 with sign 0
and dimension 5 over Rational Field
sage: E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational
Field
sage: b
19
sage: a
389
```

```
>>> from sage.all import *
>>> M
Full Modular Symbols space for Gamma_0(37) of weight 2 with sign 0
and dimension 5 over Rational Field
>>> E
Elliptic Curve defined by y^2 + y = x^3 - x^2 - 10*x - 20 over Rational
Field
>>> b
19
>>> a
389
```

CHAPITRE 4

Interfaces

L'un des aspects essentiels de Sage est qu'il permet d'effectuer des calculs utilisant des objets issus de nombreux systèmes de calcul formel de façon unifiée, avec une interface commune et un langage de programmation sain.

Les méthodes `console` et `interact` d'une interface avec un programme externe font des choses tout-à-fait différentes. Prenons l'exemple de GAP :

1. `gap.console()` : Cette commande ouvre la console GAP. Cela transfère le contrôle à GAP ; Sage n'est dans ce cas qu'un moyen commode de lancer des programmes, un peu comme le shell sous Unix.
2. `gap.interact()` : Cette commande permet d'interagir avec une instance de GAP en cours d'exécution, et éventuellement « remplie d'objets Sage ». Il est possible d'importer des objets Sage dans la session GAP (y compris depuis l'interface interactive), etc.

4.1 GP/PARI

PARI est un programme C compact, mature, fortement optimisé et spécialisé en théorie des nombres. Il possède deux interfaces très différentes utilisables depuis Sage :

— `gp` – l'interpréteur **PARI**, et
— `pari` – la bibliothèque C **PARI**.

Ainsi, les deux commandes suivantes font le même calcul de deux façons différentes. Les deux sorties ont l'air identiques, mais elles ne le sont pas en réalité, et ce qui ce passe en coulisses est radicalement différent.

```
sage: gp('znprimroot(10007)')
Mod(5, 10007)
sage: pari('znprimroot(10007)')
Mod(5, 10007)
```

```
>>> from sage.all import *
>>> gp('znprimroot(10007)')
Mod(5, 10007)
>>> pari('znprimroot(10007)')
Mod(5, 10007)
```

Dans le premier exemple, on démarre une instance de l'interpréteur GP et on lui envoie la chaîne `'znprimroot(10007)'`. Il l'évalue, et affecte le résultat à une variable GP (ce qui occupe un espace qui ne sera pas libéré dans la mémoire du processus fils GP). La valeur de la variable est ensuite affichée. Dans le second cas, nul programme séparé n'est démarré, la chaîne `'znprimroot(10007)'` est évaluée par une certaine fonction de la bibliothèque C PARI. Le résultat est stocké sur le tas de l'interpréteur Python, et la zone de mémoire utilisée est libérée lorsque son contenu n'est plus utilisé. Les objets renvoyés par ces deux commandes sont de types différents :

```
sage: type(gp('znprimroot(10007)'))
<class 'sage.interfaces.gp.GpElement'>
sage: type(pari('znprimroot(10007)'))
<class 'cypari2.gen.Gen'>
```

```
>>> from sage.all import *
>>> type(gp('znprimroot(10007)'))
<class 'sage.interfaces.gp.GpElement'>
>>> type(pari('znprimroot(10007)'))
<class 'cypari2.gen.Gen'>
```

Alors, laquelle des interfaces utiliser ? Tout dépend de ce que vous cherchez à faire. L'interface GP permet de faire absolument tout ce que vous pourriez faire avec la ligne de commande GP/PARI habituelle, puisqu'elle fait appel à celle-ci. En particulier, vous pouvez l'utiliser pour charger et exécuter des scripts PARI compliqués. L'interface PARI (via la bibliothèque C) est nettement plus restrictive. Tout d'abord, toutes les méthodes ne sont pas implémentées. Deuxièmement, beaucoup de code utilisant par exemple l'intégration numérique ne fonctionne pas via l'interface PARI. Ceci dit, l'interface PARI est souvent considérablement plus rapide et robuste que l'interface GP.

(Si l'interface GP manque de mémoire pour évaluer une ligne d'entrée donnée, elle double silencieusement la taille de la pile et réessaie d'évaluer la ligne. Ainsi votre calcul ne plantera pas même si vous n'avez pas évalué convenablement l'espace qu'il nécessite. C'est une caractéristique commode que l'interpréteur GP habituel ne semble pas fournir. L'interface PARI, quant à elle, déplace immédiatement les objets créés hors de la pile de PARI, de sorte que celle-ci ne grossit pas. Cependant, la taille de chaque objet est limitée à 100 Mo, sous peine que la pile ne déborde à la création de l'objet. Par ailleurs, cette copie supplémentaire a un léger impact sur les performances.)

En résumé, Sage fait appel à la bibliothèque C PARI pour fournir des fonctionnalités similaires à celle de l'interpréteur PARI/GP, mais depuis le langage Python, et avec un gestionnaire de mémoire plus perfectionné.

Commençons par créer une liste PARI à partir d'une liste Python.

```
sage: v = pari([1,2,3,4,5])
sage: v
[1, 2, 3, 4, 5]
sage: type(v)
<class 'cypari2.gen.Gen'>
```

```
>>> from sage.all import *
>>> v = pari([Integer(1),Integer(2),Integer(3),Integer(4),Integer(5)])
>>> v
[1, 2, 3, 4, 5]
>>> type(v)
<class 'cypari2.gen.Gen'>
```

En Sage, les objets PARI sont de type `Gen`. Le type PARI de l'objet sous-jacent est donné par la méthode `type`.

```
sage: v.type()
't_VEC'
```

```
>>> from sage.all import *
>>> v.type()
't_VEC'
```

Pour créer une courbe elliptique en PARI, on utiliserait `ellinit([1,2,3,4,5])`. La syntaxe Sage est semblable, à ceci près que `ellinit` devient une méthode qui peut être appelée sur n'importe quel objet PARI, par exemle notre `t_VEC` v.

```
sage: e = v.ellinit()
sage: e.type()
't_VEC'
sage: pari(e)[:13]
[1, 2, 3, 4, 5, 9, 11, 29, 35, -183, -3429, -10351, 6128487/10351]
```

```
>>> from sage.all import *
>>> e = v.ellinit()
>>> e.type()
't_VEC'
>>> pari(e)[:Integer(13)]
[1, 2, 3, 4, 5, 9, 11, 29, 35, -183, -3429, -10351, 6128487/10351]
```

À présent que nous disposons d'une courbe elliptique, faisons quelques calculs avec.

```
sage: e.elltors()
[1, [], []]
sage: e.ellglobalred()
[10351, [1, -1, 0, -1], 1, [11, 1; 941, 1], [[1, 5, 0, 1], [1, 5, 0, 1]]]
sage: f = e.ellchangecurve([1,-1,0,-1])
sage: f[:5]
[1, -1, 0, 4, 3]
```

```
>>> from sage.all import *
>>> e.elltors()
[1, [], []]
>>> e.ellglobalred()
[10351, [1, -1, 0, -1], 1, [11, 1; 941, 1], [[1, 5, 0, 1], [1, 5, 0, 1]]]
>>> f = e.ellchangecurve([Integer(1),-Integer(1),Integer(0),-Integer(1)])
>>> f[:Integer(5)]
[1, -1, 0, 4, 3]
```

4.2 GAP

Pour les mathématiques discrètes effectives et principalement la théorie des groupes, Sage utilise GAP.

Voici un exemple d'utilisation de la fonction GAP `IdGroup`, qui nécessite une base de données optionnelle de groupes de petit ordre, à installer séparément comme décrit plus bas.

```
sage: G = gap('Group((1,2,3)(4,5), (3,4))')
sage: G
Group( [ (1,2,3)(4,5), (3,4) ] )
sage: G.Center()
Group( () )
sage: G.IdGroup()
```

(suite sur la page suivante)

(suite de la page précédente)

```
[ 120, 34 ]
sage: G.Order()
120
```

```
>>> from sage.all import *
>>> G = gap('Group((1,2,3)(4,5), (3,4))')
>>> G
Group( [ (1,2,3)(4,5), (3,4) ] )
>>> G.Center()
Group( () )
>>> G.IdGroup()
[ 120, 34 ]
>>> G.Order()
120
```

On peut faire le même calcul en SAGE sans invoquer explicitement l'interface GAP comme suit :

```
sage: G = PermutationGroup([[(1,2,3),(4,5)],[(3,4)]])
sage: G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
sage: G.group_id()
[120, 34]
sage: n = G.order(); n
120
```

```
>>> from sage.all import *
>>> G = PermutationGroup([[(Integer(1),Integer(2),Integer(3)),(Integer(4),
↪Integer(5))],[(Integer(3),Integer(4))]])
>>> G.center()
Subgroup generated by [()] of (Permutation Group with generators [(3,4), (1,2,3)(4,
↪5)])
>>> G.group_id()
[120, 34]
>>> n = G.order(); n
120
```

Pour utiliser certaines fonctionnalités de GAP, vous devez installer un paquet Sage optionnel. Cela peut être fait avec la commande :

```
sage -i gap_packages
```

4.3 Singular

Singular fournit une bibliothèque consistante et mature qui permet, entre autres, de calculer des pgcd de polynômes de plusieurs variables, des factorisations, des bases de Gröbner ou encore des bases d'espaces de Riemann-Roch de courbes planes. Considérons la factorisation de polynômes de plusieurs variables à l'aide de l'interface à Singular fournie par Sage (n'entrez pas les `....:`) :

```
sage: R1 = singular.ring(0, '(x,y)', 'dp')
sage: R1
```

(suite sur la page suivante)

(suite de la page précédente)

```
polynomial ring, over a field, global ordering
// coefficients: QQ...
// number of vars : 2
//        block   1 : ordering dp
//                  : names    x y
//        block   2 : ordering C
sage: f = singular('9*y^8 - 9*x^2*y^7 - 18*x^3*y^6 - 18*x^5*y^6 +'
....:      '9*x^6*y^4 + 18*x^7*y^5 + 36*x^8*y^4 + 9*x^10*y^4 - 18*x^11*y^2 -'
....:      '9*x^12*y^3 - 18*x^13*y^2 + 9*x^16')
```

```
>>> from sage.all import *
>>> R1 = singular.ring(Integer(0), '(x,y)', 'dp')
>>> R1
polynomial ring, over a field, global ordering
// coefficients: QQ...
// number of vars : 2
//        block   1 : ordering dp
//                  : names    x y
//        block   2 : ordering C
>>> f = singular('9*y^8 - 9*x^2*y^7 - 18*x^3*y^6 - 18*x^5*y^6 +'
...      '9*x^6*y^4 + 18*x^7*y^5 + 36*x^8*y^4 + 9*x^10*y^4 - 18*x^11*y^2 -'
...      '9*x^12*y^3 - 18*x^13*y^2 + 9*x^16')
```

Maintenant que nous avons défini f, affichons-le puis factorisons-le.

```
sage: f
9*x^16-18*x^13*y^2-9*x^12*y^3+9*x^10*y^4-18*x^11*y^2+36*x^8*y^4+18*x^7*y^5-18*x^5*y^
↪6+9*x^6*y^4-18*x^3*y^6-9*x^2*y^7+9*y^8
sage: f.parent()
Singular
sage: F = f.factorize(); F
[1]:
   _[1]=9
   _[2]=x^6-2*x^3*y^2-x^2*y^3+y^4
   _[3]=-x^5+y^2
[2]:
   1,1,2
sage: F[1][2]
x^6-2*x^3*y^2-x^2*y^3+y^4
```

```
>>> from sage.all import *
>>> f
9*x^16-18*x^13*y^2-9*x^12*y^3+9*x^10*y^4-18*x^11*y^2+36*x^8*y^4+18*x^7*y^5-18*x^5*y^
↪6+9*x^6*y^4-18*x^3*y^6-9*x^2*y^7+9*y^8
>>> f.parent()
Singular
>>> F = f.factorize(); F
[1]:
   _[1]=9
   _[2]=x^6-2*x^3*y^2-x^2*y^3+y^4
   _[3]=-x^5+y^2
[2]:
```

(suite sur la page suivante)

(suite de la page précédente)

```
   1,1,2
>>> F[Integer(1)][Integer(2)]
x^6-2*x^3*y^2-x^2*y^3+y^4
```

Comme avec GAP dans la section *GAP*, nous pouvons aussi calculer la factorisation sans utiliser explicitement l'interface Singular (Sage y fera tout de même appel en coulisses pour le calcul).

```
sage: x, y = QQ['x, y'].gens()
sage: f = (9*y^8 - 9*x^2*y^7 - 18*x^3*y^6 - 18*x^5*y^6 + 9*x^6*y^4
....:      + 18*x^7*y^5 + 36*x^8*y^4 + 9*x^10*y^4 - 18*x^11*y^2 - 9*x^12*y^3
....:      - 18*x^13*y^2 + 9*x^16)
sage: factor(f)
(9) * (-x^5 + y^2)^2 * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

```
>>> from sage.all import *
>>> x, y = QQ['x, y'].gens()
>>> f = (Integer(9)*y**Integer(8) - Integer(9)*x**Integer(2)*y**Integer(7) -␣
↪Integer(18)*x**Integer(3)*y**Integer(6) - Integer(18)*x**Integer(5)*y**Integer(6) +␣
↪Integer(9)*x**Integer(6)*y**Integer(4)
...      + Integer(18)*x**Integer(7)*y**Integer(5) +␣
↪Integer(36)*x**Integer(8)*y**Integer(4) + Integer(9)*x**Integer(10)*y**Integer(4) -␣
↪Integer(18)*x**Integer(11)*y**Integer(2) - Integer(9)*x**Integer(12)*y**Integer(3)
...      - Integer(18)*x**Integer(13)*y**Integer(2) + Integer(9)*x**Integer(16))
>>> factor(f)
(9) * (-x^5 + y^2)^2 * (x^6 - 2*x^3*y^2 - x^2*y^3 + y^4)
```

4.4 Maxima

Le système de calcul formel Maxima est fourni avec Sage accompagné d'une implémentation du langage Lisp. Le logiciel gnuplot (que Maxima utilise par défaut pour tracer des graphiques) est disponible comme paquet optionnel. Maxima fournit notamment des routines de calcul sur des expressions formelles. Il permet de calculer des dérivées, primitives et intégrales, de résoudre des équations différentielles d'ordre 1 et souvent d'ordre 2, et de résoudre par transformée de Laplace les équations différentielles linéaires d'ordre quelconque. Maxima dispose aussi d'un grand nombre de fonctions spéciales, permet de tracer des graphes de fonctions via gnuplot, et de manipuler des matrices (réduction en lignes, valeurs propres, vecteurs propres…) ou encore des équations polynomiales.

Utilisons par exemple l'interface Sage/Maxima pour construire la matrice dont le coefficient d'indice i, j vaut i/j, pour $i, j = 1, \ldots, 4$.

```
sage: f = maxima.eval('ij_entry[i,j] := i/j')
sage: A = maxima('genmatrix(ij_entry,4,4)'); A
matrix([1,1/2,1/3,1/4],[2,1,2/3,1/2],[3,3/2,1,3/4],[4,2,4/3,1])
sage: A.determinant()
0
sage: A.echelon()
matrix([1,1/2,1/3,1/4],[0,0,0,0],[0,0,0,0],[0,0,0,0])
sage: A.eigenvalues()
[[0,4],[3,1]]
sage: A.eigenvectors().sage()
[[[0, 4], [3, 1]], [[1, 0, 0, -4], [0, 1, 0, -2], [0, 0, 1, -4/3]], [[1, 2, 3, 4]]]
```

```
>>> from sage.all import *
>>> f = maxima.eval('ij_entry[i,j] := i/j')
>>> A = maxima('genmatrix(ij_entry,4,4)'); A
matrix([1,1/2,1/3,1/4],[2,1,2/3,1/2],[3,3/2,1,3/4],[4,2,4/3,1])
>>> A.determinant()
0
>>> A.echelon()
matrix([1,1/2,1/3,1/4],[0,0,0,0],[0,0,0,0],[0,0,0,0])
>>> A.eigenvalues()
[[0,4],[3,1]]
>>> A.eigenvectors().sage()
[[[0, 4], [3, 1]], [[1, 0, 0, -4], [0, 1, 0, -2], [0, 0, 1, -4/3]], [[1, 2, 3, 4]]]
```

Un deuxième exemple :

```
sage: A = maxima("matrix ([1, 0, 0], [1, -1, 0], [1, 3, -2])")
sage: eigA = A.eigenvectors()
sage: V = VectorSpace(QQ,3)
sage: eigA
[[[-2,-1,1],[1,1,1]],[[[0,0,1]],[[0,1,3]],[[1,1/2,5/6]]]]
sage: v1 = V(sage_eval(repr(eigA[1][0][0]))); lambda1 = eigA[0][0][0]
sage: v2 = V(sage_eval(repr(eigA[1][1][0]))); lambda2 = eigA[0][0][1]
sage: v3 = V(sage_eval(repr(eigA[1][2][0]))); lambda3 = eigA[0][0][2]

sage: M = MatrixSpace(QQ,3,3)
sage: AA = M([[1,0,0],[1, - 1,0],[1,3, - 2]])
sage: b1 = v1.base_ring()
sage: AA*v1 == b1(lambda1)*v1
True
sage: b2 = v2.base_ring()
sage: AA*v2 == b2(lambda2)*v2
True
sage: b3 = v3.base_ring()
sage: AA*v3 == b3(lambda3)*v3
True
```

```
>>> from sage.all import *
>>> A = maxima("matrix ([1, 0, 0], [1, -1, 0], [1, 3, -2])")
>>> eigA = A.eigenvectors()
>>> V = VectorSpace(QQ,Integer(3))
>>> eigA
[[[-2,-1,1],[1,1,1]],[[[0,0,1]],[[0,1,3]],[[1,1/2,5/6]]]]
>>> v1 = V(sage_eval(repr(eigA[Integer(1)][Integer(0)][Integer(0)]))); lambda1 =␣
↪eigA[Integer(0)][Integer(0)][Integer(0)]
>>> v2 = V(sage_eval(repr(eigA[Integer(1)][Integer(1)][Integer(0)]))); lambda2 =␣
↪eigA[Integer(0)][Integer(0)][Integer(1)]
>>> v3 = V(sage_eval(repr(eigA[Integer(1)][Integer(2)][Integer(0)]))); lambda3 =␣
↪eigA[Integer(0)][Integer(0)][Integer(2)]

>>> M = MatrixSpace(QQ,Integer(3),Integer(3))
>>> AA = M([[Integer(1),Integer(0),Integer(0)],[Integer(1), - Integer(1),Integer(0)],
↪[Integer(1),Integer(3), - Integer(2)]])
>>> b1 = v1.base_ring()
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> AA*v1 == b1(lambda1)*v1
True
>>> b2 = v2.base_ring()
>>> AA*v2 == b2(lambda2)*v2
True
>>> b3 = v3.base_ring()
>>> AA*v3 == b3(lambda3)*v3
True
```

Voici enfin quelques exemples de tracés de graphiques avec `openmath` depuis Sage. Un grand nombre de ces exemples sont des adaptations de ceux du manuel de référence de Maxima.

Tracé en 2D de plusieurs fonctions (n'entrez pas les `....:`) :

```
sage: maxima.plot2d('[cos(7*x),cos(23*x)^4,sin(13*x)^3]','[x,0,1]', # not tested
....:     '[plot_format,openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot2d('[cos(7*x),cos(23*x)^4,sin(13*x)^3]','[x,0,1]', # not tested
...     '[plot_format,openmath]')
```

Un graphique 3D interactif, que vous pouvez déplacer à la souris (n'entrez pas les `....:`) :

```
sage: maxima.plot3d ("2^(-u^2 + v^2)", "[u, -3, 3]", "[v, -2, 2]", # not tested
....:     '[plot_format, openmath]')
sage: maxima.plot3d("atan(-x^2 + y^3/4)", "[x, -4, 4]", "[y, -4, 4]", # not tested
....:     "[grid, 50, 50]",'[plot_format, openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot3d ("2^(-u^2 + v^2)", "[u, -3, 3]", "[v, -2, 2]", # not tested
...     '[plot_format, openmath]')
>>> maxima.plot3d("atan(-x^2 + y^3/4)", "[x, -4, 4]", "[y, -4, 4]", # not tested
...     "[grid, 50, 50]",'[plot_format, openmath]')
```

Le célèbre ruban de Möbius (n'entrez pas les `....:`) :

```
sage: maxima.plot3d("[cos(x)*(3 + y*cos(x/2)), sin(x)*(3 + y*cos(x/2)), y*sin(x/2)]",
↪# not tested
....:     "[x, -4, 4]", "[y, -4, 4]",
....:     '[plot_format, openmath]')
```

```
>>> from sage.all import *
>>> maxima.plot3d("[cos(x)*(3 + y*cos(x/2)), sin(x)*(3 + y*cos(x/2)), y*sin(x/2)]", #␣
↪not tested
...     "[x, -4, 4]", "[y, -4, 4]",
...     '[plot_format, openmath]')
```

Et la fameuse bouteille de Klein (n'entrez pas les `....:`) :

```
sage: _ = maxima("expr_1: 5*cos(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y)+ 3.0) - 10.0")
sage: _ = maxima("expr_2: -5*sin(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y)+ 3.0)")
sage: _ = maxima("expr_3: 5*(-sin(x/2)*cos(y) + cos(x/2)*sin(2*y))")
```

(suite sur la page suivante)

(suite de la page précédente)

```
sage: maxima.plot3d ("[expr_1, expr_2, expr_3]", "[x, -%pi, %pi]", # not tested
....:     "[y, -%pi, %pi]", "['grid, 40, 40]",
....:     '[plot_format, openmath]')
```

```
>>> from sage.all import *
>>> _ = maxima("expr_1: 5*cos(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y)+ 3.0) - 10.0")
>>> _ = maxima("expr_2: -5*sin(x)*(cos(x/2)*cos(y) + sin(x/2)*sin(2*y)+ 3.0)")
>>> _ = maxima("expr_3: 5*(-sin(x/2)*cos(y) + cos(x/2)*sin(2*y))")
>>> maxima.plot3d ("[expr_1, expr_2, expr_3]", "[x, -%pi, %pi]", # not tested
...     "[y, -%pi, %pi]", "['grid, 40, 40]",
...     '[plot_format, openmath]')
```

CHAPITRE 5

Sage, LaTeX et compagnie

AUTEUR : Rob Beezer (2010-05-23)

Sage et le dialecte LaTeX de TeX entretiennent une forte synergie. L'objet de cette section est de présenter leurs différentes interactions. Nous commençons par les plus basiques, avant de passer à des fonctionnalités plus obscures. (Une partie de cette section peut donc être sautée en première lecture.)

5.1 Vue d'ensemble

Le plus simple pour comprendre comment Sage peut faire appel à LaTeX est peut-être de passer en revue les trois principales techniques.

1. Dans Sage, chaque « objet » doit disposer d'une représentation LaTeX. La représentation d'un objet `foo` est accessible par la commande `latex(foo)`, utilisable dans le bloc-notes ou en ligne de commande. Celle-ci renvoie une chaîne de caractères qui, interprétée par TeX en mode mathématique (entre signes dollar simples par exemple) devrait produire une représentation raisonnable de l'objet `foo`. Nous verrons quelques exemples plus bas.

   On peut ainsi utiliser Sage pour préparer des fragments de document LaTeX : il suffit de définir ou obtenir par un calcul un objet Sage et de copier-coller le résultat de `latex()` appliqué à l'objet dans le document en question.
2. Le bloc-notes de Sage fait par défaut appel à MathJax pour afficher proprement les formules mathématiques dans le navigateur web. MathJax est un moteur de rendu mathématique open source écrit en JavaScript compatible avec tous les navigateurs récents. MathJax interprète un sous-ensemble conséquent de TeX, mais pas la totalité. Orienté avant tout vers le rendu correct de petits fragments de TeX, il ne gère par exemple ni les tableaux compliqués ni le découpage en sections du document. Le rendu automatique des formules mathématiques dans le bloc-notes fonctionne en convertissant la sortie de `latex()` mentionnée ci-dessus en une portion de document HTML que MathJax sait interpréter.

   MathJax utilise ses propres polices de caractères vectorielles, et fournit ainsi un rendu de meilleure qualité que les méthodes d'affichage d'équations ou d'autres fragments de TeX qui passent par des images bitmap statiques.
3. Il est possible de faire appel à une installation extérieure de LaTeX depuis la ligne de commande de Sage, ou depuis le bloc-notes pour interpréter du code plus compliqué que ce que MathJax sait traiter. La distribution Sage inclut pratiquement tout le nécessaire pour compiler et utiliser le logiciel Sage, à la notable exception de TeX. Il faut donc installer par ailleurs TeX et quelques utilitaires de conversion associés pour l'utiliser depuis Sage de cette manière.

Voici quelques exemples d'utilisation élémentaire de la fonction `latex()`.

```
sage: var('z')
z
sage: latex(z^12)
z^{12}
sage: latex(integrate(z^4, z))
\frac{1}{5} \, z^{5}
sage: latex('a string')
\text{\texttt{a{ }string}}
sage: latex(QQ)
\Bold{Q}
sage: latex(matrix(QQ, 2, 3, [[2,4,6],[-1,-1,-1]]))
\left(\begin{array}{rrr}
2 & 4 & 6 \\
-1 & -1 & -1
\end{array}\right)
```

```
>>> from sage.all import *
>>> var('z')
z
>>> latex(z**Integer(12))
z^{12}
>>> latex(integrate(z**Integer(4), z))
\frac{1}{5} \, z^{5}
>>> latex('a string')
\text{\texttt{a{ }string}}
>>> latex(QQ)
\Bold{Q}
>>> latex(matrix(QQ, Integer(2), Integer(3), [[Integer(2),Integer(4),Integer(6)],[-
↪Integer(1),-Integer(1),-Integer(1)]]))
\left(\begin{array}{rrr}
2 & 4 & 6 \\
-1 & -1 & -1
\end{array}\right)
```

L'utilisation de base de MathJax dans le bloc-notes est largement automatique. Nous pouvons tout de même en voir quelques exemples en employant la classe `MathJax`. La méthode `eval` de celle-ci convertit un objet Sage en sa représentation LaTeX, puis emballe le résultat dans du code HTML qui fait possède la classe CSS « math », laquelle indique de faire appel à MathJax.

```
sage: from sage.misc.html import MathJax
sage: mj = MathJax()
sage: var('z')
z
sage: mj(z^12)
<html>\[z^{12}\]</html>
sage: mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Q}\]</html>
sage: mj(ZZ['x'])
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Z}[x]\]</html>
sage: mj(integrate(z^4, z))
<html>\[\frac{1}{5} \, z^{5}\]</html>
```

```
>>> from sage.all import *
>>> from sage.misc.html import MathJax
>>> mj = MathJax()
>>> var('z')
z
>>> mj(z**Integer(12))
<html>\[z^{12}\]</html>
>>> mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Q}\]</html>
>>> mj(ZZ['x'])
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Z}[x]\]</html>
>>> mj(integrate(z**Integer(4), z))
<html>\[\frac{1}{5} \, z^{5}\]</html>
```

5.2 Utilisation de base

Comme indiqué dans la vue d'ensemble, la manière la plus simple d'exploiter le support LaTeX de Sage consiste à appeler la fonction `latex()` pour produire du code LaTeX représentant un objet mathématique. Les chaînes obtenues peuvent ensuite être incorporées dans des documents LaTeX indépendants. Cela fonctionne de la même manière dans le bloc-notes et en ligne de commande.

L'autre extrême est la commande `view()`, qui fait tout le nécessaire pour afficher le rendu correspondant au code LaTeX.

En ligne de commande, `view(foo)` produit la représentation LaTeX de `foo`, la place dans un document LaTeX simple, et compile ce document en utilisant l'installation de TeX du système. Elle appelle ensuite un autre programme externe pour afficher le résultat de la compilation. La version de TeX (et donc le format de sortie) ainsi que la visionneuse à utiliser sont configurables, voir *Personnaliser le traitement du code par LaTeX*.

Dans le bloc-notes, `view(foo)` produit une combinaison de HTML et CSS qui indique à MathJax de s'occuper du rendu de la représentation LaTeX. L'utilisateur voit une version joliment formatée de la sortie à la place de la sortie ASCII par défaut de Sage. Certains objets ont cependant des représentations LaTeX trop compliquées pour être affichés par MathJax. Lorsque c'est le cas, il est possible de contourner l'interprétation par MathJax, d'appeler l'installation LaTeX du système, et de convertir le document produit en image pour l'afficher dans le bloc-note. La section *Personnaliser le code LaTeX produit* ci-dessous explique comment configurer et contrôler ce processus.

La commande interne `pretty_print()` permet de convertir un objet Sage en code HTML utilisant MathJax. C'est le code qui sera ensuite utilisé dans le bloc-notes

```
sage: pretty_print(x^12)
x^12
sage: pretty_print(integrate(sin(x), x))
-cos(x)
```

```
>>> from sage.all import *
>>> pretty_print(x**Integer(12))
x^12
>>> pretty_print(integrate(sin(x), x))
-cos(x)
```

Le bloc-notes dispose de deux autres fonctionnalités pour appeler LaTeX. Premièrement, lorsque la case « Typeset » (juste au-dessus de la première cellule d'une feuille de travail, à droite des quatre listes déroulantes) est cochée, le résultat de l'évaluation d'une cellule est automatiquement interprété par MathJax et affiché sous forme de formule plutôt que de texte brut. Les sorties déjà affichées ne sont pas modifiées tant que l'on ne ré-évalue pas les cellules correspondantes. Cocher la case « Typeset » revient essentiellement à appeler `view()` sur le résultat de chaque cellule.

Deuxièmement, le bloc-notes permet d'annoter une feuille de travail en saisissant du TeX. Un clic en tenant la touche Maj enfoncée sur la barre bleue qui apparaît lorsque l'on place le curseur de la souris entre deux cellules ouvre un mini-traitement de texte appelé TinyMCE. Cela permet de saisir du texte pour commenter la feuille de travail, et de le mettre en forme avec un éditeur WYSIWIG de HTML et CSS. Mais le texte placé entre signes dollar simples ou doubles est interprété par MathJax, respectivement comme formule composée en ligne ou hors texte.

5.3 Personnaliser le code LaTeX produit

Les méthodes de l'objet prédéfini `latex` permettent de personnaliser le code LaTeX produit par la commande `latex()` de différentes manières. Cela s'applique dans le bloc-notes comme en ligne de commande. On obtient la liste des méthodes en saisissant `latex.` (noter la présence du point) puis en appuyant sur la touche tabulation.

Un bon exemple est la méthode `latex.matrix_delimiters`, qui sert à modifier les symboles entourant les matrices – parenthèses, crochets, accolades ou barres verticales par exemple. Les délimiteurs gauche et droit sont donnés par des chaînes LaTeX. Ils n'ont pas besoin de se correspondre. Notons comment les contre-obliques qui doivent être interprétées par TeX sont protégées par une seconde contre-oblique dans l'exemple ci-dessous.

```
sage: A = matrix(ZZ, 2, 2, range(4))
sage: latex(A)
\left(\begin{array}{rr}
0 & 1 \\
2 & 3
\end{array}\right)
sage: latex.matrix_delimiters(left='[', right=']')
sage: latex(A)
\left[\begin{array}{rr}
0 & 1 \\
2 & 3
\end{array}\right]
sage: latex.matrix_delimiters(left='\\{', right='\\}')
sage: latex(A)
\left\{\begin{array}{rr}
0 & 1 \\
2 & 3
\end{array}\right\}
```

```
>>> from sage.all import *
>>> A = matrix(ZZ, Integer(2), Integer(2), range(Integer(4)))
>>> latex(A)
\left(\begin{array}{rr}
0 & 1 \\
2 & 3
\end{array}\right)
>>> latex.matrix_delimiters(left='[', right=']')
>>> latex(A)
\left[\begin{array}{rr}
0 & 1 \\
2 & 3
\end{array}\right]
>>> latex.matrix_delimiters(left='\\{', right='\\}')
>>> latex(A)
\left\{\begin{array}{rr}
0 & 1 \\
```

(suite sur la page suivante)

(suite de la page précédente)

```
2 & 3
\end{array}\right\}
```

La méthode `latex.vector_delimiters` fonctionne de manière analogue.

Les anneaux et corps usuels (entiers, rationnels, réels, etc.) sont par défaut composés en gras. La méthode `latex.blackboard_bold` permet de changer pour des lettres ajourées. Elle ne change pas la sortie de la commande `latex()` mais la définition de la macro TeX `\Bold{}` fournie par Sage.

```
sage: latex(QQ)
\Bold{Q}
sage: from sage.misc.html import MathJax
sage: mj=MathJax()
sage: mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Q}\]</html>
sage: latex.blackboard_bold(True)
sage: mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbb{#1}}\Bold{Q}\]</html>
sage: latex.blackboard_bold(False)
```

```
>>> from sage.all import *
>>> latex(QQ)
\Bold{Q}
>>> from sage.misc.html import MathJax
>>> mj=MathJax()
>>> mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbf{#1}}\Bold{Q}\]</html>
>>> latex.blackboard_bold(True)
>>> mj(QQ)
<html>\[\newcommand{\Bold}[1]{\mathbb{#1}}\Bold{Q}\]</html>
>>> latex.blackboard_bold(False)
```

On peut aussi définir de nouvelles macros TeX ou charger des packages supplémentaires. L'exemple suivant montre comment ajouter des macros qui seront utilisées à chaque fois que MathJax interprète un fragment de TeX dans le bloc-notes.

```
sage: latex.extra_macros()
''
sage: latex.add_macro("\\newcommand{\\foo}{bar}")
sage: latex.extra_macros()
'\\newcommand{\\foo}{bar}'
sage: var('x y')
(x, y)
sage: latex(x+y)
x + y
sage: from sage.misc.html import MathJax
sage: mj=MathJax()
sage: mj(x+y)
<html>\[\newcommand{\foo}{bar}x + y\]</html>
```

```
>>> from sage.all import *
>>> latex.extra_macros()
```

(suite sur la page suivante)

(suite de la page précédente)

```
''
>>> latex.add_macro("\\newcommand{\\foo}{bar}")
>>> latex.extra_macros()
'\\newcommand{\\foo}{bar}'
>>> var('x y')
(x, y)
>>> latex(x+y)
x + y
>>> from sage.misc.html import MathJax
>>> mj=MathJax()
>>> mj(x+y)
<html>\[\newcommand{\foo}{bar}x + y\]</html>
```

Ces macros supplémentaires sont disponibles aussi quand Sage appelle TeX pour compiler un fragment de document trop gros pour MathJax. C'est la fonction `latex_extra_preamble`, appelée pour préparer le préambule du document LaTeX, qui les définit, comme l'illustre l'exemple suivant. Notons à nouveau le dédoublement des `\` dans les chaînes Python.

```
sage: latex.extra_macros('')
sage: latex.extra_preamble('')
sage: from sage.misc.latex import latex_extra_preamble
sage: print(latex_extra_preamble())
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
sage: latex.add_macro("\\newcommand{\\foo}{bar}")
sage: print(latex_extra_preamble())
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
\newcommand{\foo}{bar}
```

```
>>> from sage.all import *
>>> latex.extra_macros('')
>>> latex.extra_preamble('')
>>> from sage.misc.latex import latex_extra_preamble
>>> print(latex_extra_preamble())
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
>>> latex.add_macro("\\newcommand{\\foo}{bar}")
>>> print(latex_extra_preamble())
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
\newcommand{\foo}{bar}
```

On peut aussi charger des packages LaTeX, ou ajouter n'importe quelle autre commande au préambule, grâce à la méthode `latex.add_package_to_preamble`. Sa variante plus spécialisée `latex.add_package_to_preamble_if_available` vérifie qu'un package donné est disponible avant de l'ajouter au préambule si c'est bien le cas.

Dans l'exemple suivant, nous ajoutons au préambule la commande qui charge le package `geometry`, puis nous l'utilisons

pour régler la taille de la zone de texte (et donc indirectement les marges) du document TeX. Une fois encore, les contre-obliques sont dédoublées.

```
sage: from sage.misc.latex import latex_extra_preamble
sage: latex.extra_macros('')
sage: latex.extra_preamble('')
sage: latex.add_to_preamble('\\usepackage{geometry}')
sage: latex.add_to_preamble('\\geometry{letterpaper,total={8in,10in}}')
sage: latex.extra_preamble()
'\\usepackage{geometry}\\geometry{letterpaper,total={8in,10in}}'
sage: print(latex_extra_preamble())
\usepackage{geometry}\geometry{letterpaper,total={8in,10in}}
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
```

```
>>> from sage.all import *
>>> from sage.misc.latex import latex_extra_preamble
>>> latex.extra_macros('')
>>> latex.extra_preamble('')
>>> latex.add_to_preamble('\\usepackage{geometry}')
>>> latex.add_to_preamble('\\geometry{letterpaper,total={8in,10in}}')
>>> latex.extra_preamble()
'\\usepackage{geometry}\\geometry{letterpaper,total={8in,10in}}'
>>> print(latex_extra_preamble())
\usepackage{geometry}\geometry{letterpaper,total={8in,10in}}
\newcommand{\ZZ}{\Bold{Z}}
...
\newcommand{\Bold}[1]{\mathbf{#1}}
```

Voici enfin comment ajouter un package en vérifiant sa disponibilité, et ce qu'il se passe quand le package n'existe pas.

```
sage: latex.extra_preamble('')
sage: latex.extra_preamble()
''
sage: latex.add_to_preamble('\\usepackage{foo-bar-unchecked}')
sage: latex.extra_preamble()
'\\usepackage{foo-bar-unchecked}'
sage: latex.add_package_to_preamble_if_available('foo-bar-checked')
sage: latex.extra_preamble()
'\\usepackage{foo-bar-unchecked}'
```

```
>>> from sage.all import *
>>> latex.extra_preamble('')
>>> latex.extra_preamble()
''
>>> latex.add_to_preamble('\\usepackage{foo-bar-unchecked}')
>>> latex.extra_preamble()
'\\usepackage{foo-bar-unchecked}'
>>> latex.add_package_to_preamble_if_available('foo-bar-checked')
>>> latex.extra_preamble()
'\\usepackage{foo-bar-unchecked}'
```

5.4 Personnaliser le traitement du code par LaTeX

En plus de modifier LaTeX produit par Sage, on peut choisir la variante de TeX appelée pour le traiter, et donc la nature du document produit. De même, il est possible de contrôler dans quelles circonstances le bloc-notes utilisera MathJax (c'est-à-dire quels fragments de code TeX seront jugés suffisamment simples) et quand il choisira de se rabattre sur l'installation de TeX du système.

La méthode `latex.engine()` permet de choisir lequel des moteurs TeX `latex`, `pdflatex` et `xelatex` doit servir à compiler les expressions LaTeX complexes. Lorsque l'on appelle `view` en ligne de commande, si le moteur actif est `latex`, celui-ci produit un fichier dvi, puis Sage fait appel à une visionneuse dvi (par exemple xdvi) pour afficher le résultat. Si en revanche le moteur est `pdflatex`, il produit par défaut un fichier PDF, que Sage affiche grâce à la visionneuse PDF du système (Adobe Reader, Okular, evince…).

Dans le bloc-notes, la première étape est de décider s'il faut utiliser MathJax ou LaTeX pour interpréter un fragment de TeX donné. La décision se fonde sur une liste de chaînes « interdites » dont la présence dans le fragment indique d'appeler latex (ou plus généralement le moteur choisi via `latex.engine()`) au lieu MathJax. Les méthodes `latex.add_to_mathjax_avoid_list` et `latex.mathjax_avoid_list` permettent de gérer le contenu de cette liste.

```
sage: # not tested
sage: latex.mathjax_avoid_list([])
sage: latex.mathjax_avoid_list()
[]
sage: latex.mathjax_avoid_list(['foo', 'bar'])
sage: latex.mathjax_avoid_list()
['foo', 'bar']
sage: latex.add_to_mathjax_avoid_list('tikzpicture')
sage: latex.mathjax_avoid_list()
['foo', 'bar', 'tikzpicture']
sage: latex.mathjax_avoid_list([])
sage: latex.mathjax_avoid_list()
[]
```

```
>>> from sage.all import *
>>> # not tested
>>> latex.mathjax_avoid_list([])
>>> latex.mathjax_avoid_list()
[]
>>> latex.mathjax_avoid_list(['foo', 'bar'])
>>> latex.mathjax_avoid_list()
['foo', 'bar']
>>> latex.add_to_mathjax_avoid_list('tikzpicture')
>>> latex.mathjax_avoid_list()
['foo', 'bar', 'tikzpicture']
>>> latex.mathjax_avoid_list([])
>>> latex.mathjax_avoid_list()
[]
```

Supposons maintenant que, dans le bloc-notes, un appel à `view()` ou l'évaluation d'une cellule lorsque la case « Typeset » est cochée produise un résultat dont le mécanisme décrit ci-dessus détermine qu'il doit être passé au moteur LaTeX externe. Comme en ligne de commande, l'exécutable spécifié par `latex.engine()` traite alors le document. Cependant, au lieu d'appeler une visionneuse externe pour afficher le document produit, Sage tente de recadrer le document en rognant les zones blanches, et de le convertir en une image qui est ensuite insérée dans le bloc-notes comme sortie associée à la cellule.

Plusieurs facteurs influencent la conversion, principalement le moteur TeX choisi et la palette d'utilitaires de conversion

disponibles sur le système. Les convertisseurs suivants couvrent à eux quatre toutes les situations : `dvips`, `ps2pdf`, `dvipng` et `convert` (de la collection ImageMagick). Dans tous les cas, il s'agit d'arriver à produire une image PNG à insérer dans la feuille de travail. Lorsque le moteur LaTeX produit un fichier dvi, le programme dvipng suffit en général à effectuer la conversion. Il peut cependant arriver que le fichier dvi contienne des instructions spécifiques à un pilote (commande TeX `\special`) que dvipng est incapable d'interpréter, auquel cas on utilise `dvips` pour créer un fichier PostScript. Celui-ci, de même que le fichier PDF produit par le moteur le cas échéant, est ensuite converti en image png avec `convert`. Les commandes `have_dvipng()` et `have_convert()` permettent de tester la présence sur le système des utilitaires en question.

Toutes ces conversions sont automatiques lorsque les outils nécessaires sont installés. Dans le cas contraire, un message d'erreur indique ce qu'il manque et où le télécharger.

La section suivante (*Example : rendu de graphes avec tkz-graph*) présente un exemple concret de traitement d'expressions LaTeX complexes, en l'occurrence pour obtenir un rendu de qualité de graphes grâce au package LaTeX `tkz-graph`. Les tests inclus dans Sage contiennent d'autres exemples. On y accède en important l'objet prédéfini `sage.misc.latex.latex_examples`, instance de la classe `sage.misc.latex.LatexExamples`, comme illustré ci-dessous. Les exemples fournis actuellement couvrent les types d'objets suivants : diagrammes commutatifs (utilisant le package xy), graphes combinatoires ($tkz - graph$), nœuds ($xypic$), schémas pstricks ($pstricks$). Pour obtenir la liste des exemples, utilisez la complétion de ligne de commande après avoir importé `latex_examples`. Chaque exemple affiche quand on l'appelle des instructions sur la configuration nécessaire pour qu'il fonctionne correctement. Une fois le préambule, le moteur LaTeX etc. configurés comme indiqué, il suffit d'appeler la commande `view()` pour visualiser l'exemple.

```
sage: from sage.misc.latex import latex_examples
sage: latex_examples.diagram()
LaTeX example for testing display of a commutative diagram produced
by xypic.

To use, try to view this object -- it will not work.  Now try
'latex.add_to_preamble("\\usepackage[matrix,arrow,curve,cmtip]{xy}")',
and try viewing again. You should get a picture (a part of the diagram arising
from a filtered chain complex).
```

```
>>> from sage.all import *
>>> from sage.misc.latex import latex_examples
>>> latex_examples.diagram()
LaTeX example for testing display of a commutative diagram produced
by xypic.
<BLANKLINE>
To use, try to view this object -- it will not work.  Now try
'latex.add_to_preamble("\\usepackage[matrix,arrow,curve,cmtip]{xy}")',
and try viewing again. You should get a picture (a part of the diagram arising
from a filtered chain complex).
```

5.5 Example : rendu de graphes avec tkz-graph

Le package `tkz-graph` permet de produire des dessins de graphes (combinatoires) de qualité. Il repose sur TikZ, lui-même une interface pour la bibliothèque TeX pgf : pgf, TikZ et tkz-graph doivent donc tous être présents dans l'installation TeX du système pour que cet exemple fonctionne. Les versions fournies par certaines distributions TeX sont parfois trop anciennes, et il peut donc être souhaitable de les installer manuellement dans son arbre texmf personnel. On consultera la documentation de la distribution TeX pour la procédure à suivre, qui dépasse le cadre de ce document. La section *Une installation TeX pleinement opérationnelle* donne la liste des fichiers nécessaires.

Il nous faut tout d'abord nous assurer que les packages requis sont inclus dans le document LaTeX, en les ajoutant au préambule. Le rendu des graphes n'est pas correct quand on passe par le format dvi, aussi il est préférable de sélectionner

`pdflatex` comme moteur TeX. Après ces réglages, une instruction du genre `view(graphs.CompleteGraph(4))` saisie dans l'interface en ligne de commande doit produire un fichier PDF contenant un dessin du graphe complet K_4.

Pour que la même chose fonctionne dans le bloc-notes, il faut de plus désactiver l'interprétation du code LaTeX produisant le graphe par MathJax, à l'aide de la liste de motifs exclus. Le nom de l'environnement `tikzpicture`, dans lequel sont placés les graphes, est un bon choix de chaîne à exclure. Une fois cela fait, la commande `view(graphs.CompleteGraph(4))` dans une feuille de travail du bloc-notes appelle pdflatex pour produire un fichier PDF, puis `convert` pour en extraire une image PNG à placer dans la zone de sortie de la feuille de travail. Les commandes suivantes reprennent l'ensemble des étapes de configuration.

```
sage: from sage.graphs.graph_latex import setup_latex_preamble
sage: setup_latex_preamble()
sage: latex.extra_preamble() # random - depends on system's TeX installation
'\\usepackage{tikz}\n\\usepackage{tkz-graph}\n\\usepackage{tkz-berge}\n'
sage: latex.engine('pdflatex')
sage: latex.add_to_mathjax_avoid_list('tikzpicture')  # not tested
sage: latex.mathjax_avoid_list()                      # not tested
['tikz', 'tikzpicture']
```

```
>>> from sage.all import *
>>> from sage.graphs.graph_latex import setup_latex_preamble
>>> setup_latex_preamble()
>>> latex.extra_preamble() # random - depends on system's TeX installation
'\\usepackage{tikz}\n\\usepackage{tkz-graph}\n\\usepackage{tkz-berge}\n'
>>> latex.engine('pdflatex')
>>> latex.add_to_mathjax_avoid_list('tikzpicture')  # not tested
>>> latex.mathjax_avoid_list()                      # not tested
['tikz', 'tikzpicture']
```

La mise en forme du graphe est faite en traitant des commandes `tkz-graph` qui le décrivent avec `pdflatex`. Diverses options pour influencer ce rendu, qui sortent du cadre de cette section, sont décrites dans la section intitulée « LaTeX Options for Graphs » du manuel de référence de Sage.

5.6 Une installation TeX pleinement opérationnelle

Beaucoup de fonctionnalités avancées de l'intégration Sage-TeX nécessitent qu'une installation extérieure de TeX soit disponible sur le système. Les distributions Linux en fournissent généralement, sous forme de paquets basés sur TeX Live ; sous OS X, on peut installer TeXshop ; et sous Windows, MikTeX. L'utilitaire `convert` fait partie de la boîte à outils ImageMagick (probablement disponible dans l'archive de paquets de votre système ou facile à télécharger et installer). Les programmes `dvipng`, `ps2pdf`, and `dvips` sont parfois inclus dans les installations de TeX, et les deux premiers sont par ailleurs disponibles respectivement à l'adresse http://sourceforge.net/projects/dvipng/ et dans Ghostscript.

Le rendu des graphes nécessite une version suffisamment récente de PGF, ainsi que les fichiers `tkz-graph.sty`, disponible sur le site web https://www.ctan.org/pkg/tkz-graph, `tkz-arith.sty` et suivant les cas `tkz-berge.sty`, disponibles sur le site web https://www.ctan.org/pkg/tkz-berge.

5.7 Programmes externes

Trois programmes séparés contribuent encore à l'intégration TeX-Sage.

Le premier, sagetex, est (pour simplifier) une collection de macros TeX qui permettent d'introduire dans un document LaTeX des instructions qui seront interprétées par Sage pour effectuer des calculs et/ou mettre en forme des objets mathématiques avec la commande `latex()` de Sage. Il est donc possible de faire faire des calculs à Sage et de produire les sorties LaTeX associées comme étape intermédiaire de la compilation d'un document LaTeX. Par exemple, on peut

imaginer de maintenir la correspondance entre questions et réponses dans un sujet d'examen en utilisant Sage pour calculer les unes à partir des autres. Sagetex est décrit plus en détail en section *Utiliser SageTeX* de ce document.

CHAPITRE 6

Programmation

6.1 Charger et attacher des fichiers Sage

Nous décrivons maintenant la manière de charger dans Sage des programmes écrits dans des fichiers séparés. Créons un fichier appelé `example.sage` avec le contenu suivant :

```
print("Hello World")
print(2^3)
```

Nous pouvons lire et exécuter le contenu du fichier `example.sage` en utilisant la commande `load`.

```
sage: load("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> load("example.sage")
Hello World
8
```

Nous pouvons aussi attacher un fichier Sage à la session en cours avec la commande `attach` :

```
sage: attach("example.sage")
Hello World
8
```

```
>>> from sage.all import *
>>> attach("example.sage")
Hello World
8
```

L'effet de cette commande est que si nous modifions maintenant `example.sage` et entrons une ligne vierge (i.e. appuyons sur `entrée`), le contenu de `example.sage` sera automatiquement rechargé dans Sage.

Avec `attach`, le fichier est rechargé automatiquement dans Sage à chaque modification, ce qui est pratique pour déboguer du code ; tandis qu'avec `load` il n'est chargé qu'une fois.

Lorsque Sage lit `example.sage`, il le convertit en un programme Python qui est ensuite exécuté par l'interpréteur Python. Cette conversion est minimale, elle se résume essentiellement à encapsuler les littéraux entiers dans des `Integer()` et les littéraux flottants dans des `RealNumber()`, à remplacer les `^` par des `**`, et à remplacer par exemple `R.2` par `R.gen(2)`. La version convertie en Python de `example.sage` est placée dans le même répertoire sous le nom `example.sage.py`. Elle contient le code suivant :

```
print("Hello World")
print(Integer(2)**Integer(3))
```

On voit que les littéraux entiers ont été encapsulés et que le `^` a été remplacé par `**` (en effet, en Python, `^` représente le ou exclusif et `**` l'exponentiation).

(Ce prétraitement est implémenté dans le fichier `sage/misc/interpreter.py`.)

On peut coller dans Sage des blocs de code de plusieurs lignes avec des indentations pourvu que les blocs soient délimités par des retours à la ligne (cela n'est pas nécessaire quand le code est dans un fichier). Cependant, la meilleure façon d'entrer ce genre de code dans Sage est de l'enregistrer dans un fichier et d'utiliser la commande `attach` comme décrit ci-dessus.

6.2 Écrire des programmes compilés

Dans les calculs mathématiques sur ordinateur, la vitesse a une importance cruciale. Or, si Python est un langage commode et de très haut niveau, certaines opérations peuvent être plus rapides de plusieurs ordres de grandeur si elles sont implémentées sur des types statiques dans un langage compilé. Certaines parties de Sage auraient été trop lentes si elles avaient été écrites entièrement en Python. Pour pallier ce problème, Sage supporte une sorte de « version compilée » de Python appelée Cython (voir [Cyt] et [Pyr]). Cython ressemble à la fois à Python et à C. La plupart des constructions Python, dont la définition de listes par compréhension, les expressions conditionnelles, les constructions comme `+=` sont autorisées en Cython. Vous pouvez aussi importer du code depuis d'autres modules Python. En plus de cela, vous pouvez déclarer des variables C arbitraires, et faire directement des appels arbitraires à des bibliothèques C. Le code Cython est converti en C et compilé avec le compilateur C.

Pour créer votre propre code Sage compilé, donnez à votre fichier source l'exension `.spyx` (à la place de `.sage`). Avec l'interface en ligne de commande, vous pouvez charger ou attacher des fichiers de code compilé exactement comme les fichiers interprétés. Pour l'instant, le *notebook* ne permet pas d'attacher des fichiers compilés. La compilation proprement dite a lieu « en coulisse », sans que vous ayez à la déclencher explicitement. La bibliothèque d'objets partagés compilés se trouve dans `$HOME/.sage/temp/hostname/pid/spyx`. Ces fichiers sont supprimés lorsque vous quittez Sage.

Attention, le prétraitement des fichiers Sage mentionné plus haut N'EST PAS appliqué aux fichiers spyx, ainsi, dans un fichier spyx, `1/3` vaut 0 et non le nombre rationnel $1/3$. Pour appeler une fonction `foo` de la bibliothèque Sage depuis un fichier spyx, importez `sage.all` et appelez `sage.all.foo`.

```
import sage.all
def foo(n):
    return sage.all.factorial(n)
```

6.2.1 Appeler des fonctions définies dans des fichiers C séparés

Il n'est pas difficile non plus d'accéder à des fonctions écrites en C, dans des fichiers *.c séparés. Créez dans un même répertoire deux fichiers `test.c` et `test.spyx` avec les contenus suivants :

Le code C pur : `test.c`

```
int add_one(int n) {
  return n + 1;
}
```

Le code Cython : `test.spyx` :

```
cdef extern from "test.c":
    int add_one(int n)

def test(n):
    return add_one(n)
```

Vous pouvez alors faire :

```
sage: attach("test.spyx")
Compiling (...)/test.spyx...
sage: test(10)
11
```

```
>>> from sage.all import *
>>> attach("test.spyx")
Compiling (...)/test.spyx...
>>> test(Integer(10))
11
```

Si la compilation du code C généré à partir d'un fichier Cython nécessite une bibliothèque supplémentaire `foo`, ajoutez au source Cython la ligne `clib foo`. De même, il est possible d'ajouter un fichier C supplémentaire `bar` aux fichiers à compiler avec la déclaration `cfile bar`.

6.3 Scripts Python/Sage autonomes

Le script autonome suivant, écrit en Sage, permet de factoriser des entiers, des polynômes, etc. :

```
#!/usr/bin/env sage

import sys

if len(sys.argv) != 2:
    print("Usage: %s <n>" % sys.argv[0])
    print("Outputs the prime factorization of n.")
    sys.exit(1)

print(factor(sage_eval(sys.argv[1])))
```

Pour utiliser ce script, votre répertoire `SAGE_ROOT` doit apparaître dans la variable d'environnement PATH. Supposons que le script ci-dessus soit appelé `factor`, il peut alors être utilisé comme dans l'exemple suivant :

```
bash $ ./factor 2006
2 * 17 * 59
```

6.4 Types de données

Chaque objet Sage a un type bien défini. Python dispose d'une vaste gamme de types intégrés et la bibliothèque Sage en fournit de nombreux autres. Parmi les types intégrés de Python, citons les chaînes, les listes, les n-uplets, les entiers et les flottants :

```
sage: s = "sage"; type(s)
<... 'str'>
sage: s = 'sage'; type(s)      # guillemets simples ou doubles
<... 'str'>
sage: s = [1,2,3,4]; type(s)
<... 'list'>
sage: s = (1,2,3,4); type(s)
<... 'tuple'>
sage: s = int(2006); type(s)
<... 'int'>
sage: s = float(2006); type(s)
<... 'float'>
```

```
>>> from sage.all import *
>>> s = "sage"; type(s)
<... 'str'>
>>> s = 'sage'; type(s)      # guillemets simples ou doubles
<... 'str'>
>>> s = [Integer(1),Integer(2),Integer(3),Integer(4)]; type(s)
<... 'list'>
>>> s = (Integer(1),Integer(2),Integer(3),Integer(4)); type(s)
<... 'tuple'>
>>> s = int(Integer(2006)); type(s)
<... 'int'>
>>> s = float(Integer(2006)); type(s)
<... 'float'>
```

Sage ajoute de nombreux autres types. Par exemple, les espaces vectoriels :

```
sage: V = VectorSpace(QQ, 1000000); V
Vector space of dimension 1000000 over Rational Field
sage: type(V)
<class 'sage.modules.free_module.FreeModule_ambient_field_with_category'>
```

```
>>> from sage.all import *
>>> V = VectorSpace(QQ, Integer(1000000)); V
Vector space of dimension 1000000 over Rational Field
>>> type(V)
<class 'sage.modules.free_module.FreeModule_ambient_field_with_category'>
```

Seules certaines fonctions peuvent être appelées sur `V`. Dans d'autres logiciels mathématiques, cela se fait en notation « fonctionnelle », en écrivant `foo(V,...)`. En Sage, certaines fonctions sont attachés au type (ou classe) de l'objet et appelées avec une syntaxe « orientée objet » comme en Java ou en C++, par exemple `V.foo(...)`. Cela évite de polluer l'espace de noms global avec des dizaines de milliers de fonctions, et cela permet d'avoir plusieurs fonctions appelées `foo`, avec des comportements différents, sans devoir se reposer sur le type des arguments (ni sur des instructions case) pour décider laquelle appeler. De plus, une fonction dont vous réutilisez le nom demeure disponible : par exemple, si vous appelez quelque chose `zeta` et si ensuite vous voulez calculer la valeur de la fonction zêta de Riemann au point 0.5, vous pouvez encore écrire `s=.5; s.zeta()`.

```
sage: zeta = -1
sage: s=.5; s.zeta()
-1.46035450880959
```

```
>>> from sage.all import *
>>> zeta = -Integer(1)
>>> s=RealNumber('.5'); s.zeta()
-1.46035450880959
```

La notation fonctionnelle usuelle est aussi acceptée dans certains cas courants, par commodité et parce que certaines expressions mathématiques ne sont pas claires en notation orientée objet. Voici quelques exemples.

```
sage: n = 2; n.sqrt()
sqrt(2)
sage: sqrt(2)
sqrt(2)
sage: V = VectorSpace(QQ,2)
sage: V.basis()
    [(1, 0), (0, 1)]
sage: basis(V)
    [(1, 0), (0, 1)]
sage: M = MatrixSpace(GF(7), 2); M
Full MatrixSpace of 2 by 2 dense matrices over Finite Field of size 7
sage: A = M([1,2,3,4]); A
[1 2]
[3 4]
sage: A.charpoly('x')
x^2 + 2*x + 5
sage: charpoly(A, 'x')
x^2 + 2*x + 5
```

```
>>> from sage.all import *
>>> n = Integer(2); n.sqrt()
sqrt(2)
>>> sqrt(Integer(2))
sqrt(2)
>>> V = VectorSpace(QQ,Integer(2))
>>> V.basis()
    [(1, 0), (0, 1)]
>>> basis(V)
    [(1, 0), (0, 1)]
>>> M = MatrixSpace(GF(Integer(7)), Integer(2)); M
Full MatrixSpace of 2 by 2 dense matrices over Finite Field of size 7
>>> A = M([Integer(1),Integer(2),Integer(3),Integer(4)]); A
[1 2]
[3 4]
>>> A.charpoly('x')
x^2 + 2*x + 5
>>> charpoly(A, 'x')
x^2 + 2*x + 5
```

Pour obtenir la liste de toutes les fonctions membres de A, utilisez la complétion de ligne de commande : tapez `A.`, puis appuyez sur la touche `[tab]` de votre clavier, comme expliqué dans la section *Recherche en arrière et complétion de ligne*

de commande.

6.5 Listes, n-uplets et séquences

Une liste stocke des éléments qui peuvent être de type arbitraire. Comme en C, en C++ etc. (mais au contraire de ce qu'il se passe dans la plupart des systèmes de calcul formel usuels) les éléments de la liste sont indexés à partir de 0 :

```
sage: v = [2, 3, 5, 'x', SymmetricGroup(3)]; v
[2, 3, 5, 'x', Symmetric group of order 3! as a permutation group]
sage: type(v)
<... 'list'>
sage: v[0]
2
sage: v[2]
5
```

```
>>> from sage.all import *
>>> v = [Integer(2), Integer(3), Integer(5), 'x', SymmetricGroup(Integer(3))]; v
[2, 3, 5, 'x', Symmetric group of order 3! as a permutation group]
>>> type(v)
<... 'list'>
>>> v[Integer(0)]
2
>>> v[Integer(2)]
5
```

Lors d'un accès à une liste, l'index n'a pas besoin d'être un entier Python. Un entier (Integer) Sage (ou un Rational, ou n'importe quoi d'autre qui a une méthode `__index__`) fait aussi l'affaire.

```
sage: v = [1,2,3]
sage: v[2]
3
sage: n = 2       # Integer (entier Sage)
sage: v[n]        # ça marche !
3
sage: v[int(n)]   # Ok aussi
3
```

```
>>> from sage.all import *
>>> v = [Integer(1),Integer(2),Integer(3)]
>>> v[Integer(2)]
3
>>> n = Integer(2)       # Integer (entier Sage)
>>> v[n]        # ça marche !
3
>>> v[int(n)]   # Ok aussi
3
```

La fonction `range` crée une liste d'entiers Python (et non d'entiers Sage) :

```
sage: list(range(1, 15))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
>>> from sage.all import *
>>> list(range(Integer(1), Integer(15)))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

Cela est utile pour construire des listes par compréhension :

```
sage: L = [factor(n) for n in range(1, 15)]
sage: L
[1, 2, 3, 2^2, 5, 2 * 3, 7, 2^3, 3^2, 2 * 5, 11, 2^2 * 3, 13, 2 * 7]
sage: L[12]
13
sage: type(L[12])
<class 'sage.structure.factorization_integer.IntegerFactorization'>
sage: [factor(n) for n in range(1, 15) if is_odd(n)]
[1, 3, 5, 7, 3^2, 11, 13]
```

```
>>> from sage.all import *
>>> L = [factor(n) for n in range(Integer(1), Integer(15))]
>>> L
[1, 2, 3, 2^2, 5, 2 * 3, 7, 2^3, 3^2, 2 * 5, 11, 2^2 * 3, 13, 2 * 7]
>>> L[Integer(12)]
13
>>> type(L[Integer(12)])
<class 'sage.structure.factorization_integer.IntegerFactorization'>
>>> [factor(n) for n in range(Integer(1), Integer(15)) if is_odd(n)]
[1, 3, 5, 7, 3^2, 11, 13]
```

Pour plus d'information sur les compréhensions, voir [PyT].

Une fonctionnalité merveilleuse est l'extraction de tranches d'une liste. Si `L` est une liste, `L[m:n]` renvoie la sous-liste de `L` formée des éléments d'indices m à $n-1$ inclus :

```
sage: L = [factor(n) for n in range(1, 20)]
sage: L[4:9]
[5, 2 * 3, 7, 2^3, 3^2]
sage: L[:4]
[1, 2, 3, 2^2]
sage: L[14:4]
[]
sage: L[14:]
[3 * 5, 2^4, 17, 2 * 3^2, 19]
```

```
>>> from sage.all import *
>>> L = [factor(n) for n in range(Integer(1), Integer(20))]
>>> L[Integer(4):Integer(9)]
[5, 2 * 3, 7, 2^3, 3^2]
>>> L[:Integer(4)]
[1, 2, 3, 2^2]
>>> L[Integer(14):Integer(4)]
[]
>>> L[Integer(14):]
[3 * 5, 2^4, 17, 2 * 3^2, 19]
```

Les n-uplets ressemblent aux listes, à ceci près qu'ils sont non mutables, ce qui signifie qu'ils ne peuvent plus être modifiés

une fois créés.

```
sage: v = (1,2,3,4); v
(1, 2, 3, 4)
sage: type(v)
<... 'tuple'>
sage: v[1] = 5
Traceback (most recent call last):
...
TypeError: 'tuple' object does not support item assignment
```

```
>>> from sage.all import *
>>> v = (Integer(1),Integer(2),Integer(3),Integer(4)); v
(1, 2, 3, 4)
>>> type(v)
<... 'tuple'>
>>> v[Integer(1)] = Integer(5)
Traceback (most recent call last):
...
TypeError: 'tuple' object does not support item assignment
```

Les séquences sont un troisième type Sage analogue aux listes. Contrairement aux listes et aux n-uplets, il ne s'agit pas d'un type interne de Python. Par défaut, les séquences sont mutables, mais on peut interdire leur modification en utilisant la méthode `set_immutable` de la classe `Sequence`, comme dans l'exemple suivant. Tous les éléments d'une séquence ont un parent commun, appelé l'univers de la séquence.

```
sage: v = Sequence([1,2,3,4/5])
sage: v
[1, 2, 3, 4/5]
sage: type(v)
<class 'sage.structure.sequence.Sequence_generic'>
sage: type(v[1])
<class 'sage.rings.rational.Rational'>
sage: v.universe()
Rational Field
sage: v.is_immutable()
False
sage: v.set_immutable()
sage: v[0] = 3
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
```

```
>>> from sage.all import *
>>> v = Sequence([Integer(1),Integer(2),Integer(3),Integer(4)/Integer(5)])
>>> v
[1, 2, 3, 4/5]
>>> type(v)
<class 'sage.structure.sequence.Sequence_generic'>
>>> type(v[Integer(1)])
<class 'sage.rings.rational.Rational'>
>>> v.universe()
Rational Field
```

(suite sur la page suivante)

(suite de la page précédente)

```
>>> v.is_immutable()
False
>>> v.set_immutable()
>>> v[Integer(0)] = Integer(3)
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
```

Les séquences sont des objets dérivés des listes, et peuvent être utilisées partout où les listes peuvent l'être :

```
sage: v = Sequence([1,2,3,4/5])
sage: isinstance(v, list)
True
sage: list(v)
[1, 2, 3, 4/5]
sage: type(list(v))
<... 'list'>
```

```
>>> from sage.all import *
>>> v = Sequence([Integer(1),Integer(2),Integer(3),Integer(4)/Integer(5)])
>>> isinstance(v, list)
True
>>> list(v)
[1, 2, 3, 4/5]
>>> type(list(v))
<... 'list'>
```

Autre exemple : les bases d'espaces vectoriels sont des séquences non mutables, car il ne faut pas les modifier.

```
sage: V = QQ^3; B = V.basis(); B
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
sage: type(B)
<class 'sage.structure.sequence.Sequence_generic'>
sage: B[0] = B[1]
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
sage: B.universe()
Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3); B = V.basis(); B
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]
>>> type(B)
<class 'sage.structure.sequence.Sequence_generic'>
>>> B[Integer(0)] = B[Integer(1)]
Traceback (most recent call last):
...
ValueError: object is immutable; please change a copy instead.
>>> B.universe()
Vector space of dimension 3 over Rational Field
```

6.6 Dictionnaires

Un dictionnaire (parfois appelé un tableau associatif) est une correspondance entre des objets « hachables » (par exemple des chaînes, des nombres, ou des n-uplets de tels objets, voir http://docs.python.org/3/tutorial/datastructures.html et https://docs.python.org/3/library/stdtypes.html#typesmapping dans la documentation de Python pour plus de détails) vers des objets arbitraires.

```
sage: d = {1:5, 'sage':17, ZZ:GF(7)}
sage: type(d)
<... 'dict'>
sage: list(d.keys())
[1, 'sage', Integer Ring]
sage: d['sage']
17
sage: d[ZZ]
Finite Field of size 7
sage: d[1]
5
```

```
>>> from sage.all import *
>>> d = {Integer(1):Integer(5), 'sage':Integer(17), ZZ:GF(Integer(7))}
>>> type(d)
<... 'dict'>
>>> list(d.keys())
[1, 'sage', Integer Ring]
>>> d['sage']
17
>>> d[ZZ]
Finite Field of size 7
>>> d[Integer(1)]
5
```

La troisième clé utilisée ci-dessus, l'anneau des entiers relatifs, montre que les indices d'un dictionnaire peuvent être des objets compliqués.

Un dictionnaire peut être transformé en une liste de couples clé-objet contenant les mêmes données :

```
sage: list(d.items())
[(1, 5), ('sage', 17), (Integer Ring, Finite Field of size 7)]
```

```
>>> from sage.all import *
>>> list(d.items())
[(1, 5), ('sage', 17), (Integer Ring, Finite Field of size 7)]
```

Le parcours itératifs des paires d'un dictionnaire est un idiome de programmation fréquent :

```
sage: d = {2:4, 3:9, 4:16}
sage: [a*b for a, b in d.items()]
[8, 27, 64]
```

```
>>> from sage.all import *
>>> d = {Integer(2):Integer(4), Integer(3):Integer(9), Integer(4):Integer(16)}
>>> [a*b for a, b in d.items()]
[8, 27, 64]
```

Comme le montre la dernière sortie ci-dessus, un dictionnaire stocke ses éléments sans ordre particulier.

6.7 Ensembles

Python dispose d'un type ensemble intégré. Sa principale caractéristique est qu'il est possible de tester très rapidement si un élément appartient ou non à un ensemble. Le type ensemble fournit les opérations ensemblistes usuelles.

```
sage: X = set([1,19,'a']);   Y = set([1,1,1, 2/3])
sage: X   # random sort order
{1, 19, 'a'}
sage: X == set(['a', 1, 1, 19])
True
sage: Y
{2/3, 1}
sage: 'a' in X
True
sage: 'a' in Y
False
sage: X.intersection(Y)
{1}
```

```
>>> from sage.all import *
>>> X = set([Integer(1),Integer(19),'a']);   Y = set([Integer(1),Integer(1),
↪Integer(1), Integer(2)/Integer(3)])
>>> X   # random sort order
{1, 19, 'a'}
>>> X == set(['a', Integer(1), Integer(1), Integer(19)])
True
>>> Y
{2/3, 1}
>>> 'a' in X
True
>>> 'a' in Y
False
>>> X.intersection(Y)
{1}
```

Sage a son propre type ensemble, qui est (dans certains cas) implémenté au-dessus du type Python, mais offre quelques fonctionnalités supplémentaires utiles à Sage. Pour créer un ensemble Sage, on utilise `Set(...)`. Par exemple,

```
sage: X = Set([1,19,'a']);   Y = Set([1,1,1, 2/3])
sage: X   # random sort order
{'a', 1, 19}
sage: X == Set(['a', 1, 1, 19])
True
sage: Y
{1, 2/3}
sage: X.intersection(Y)
{1}
sage: print(latex(Y))
\left\{1, \frac{2}{3}\right\}
sage: Set(ZZ)
Set of elements of Integer Ring
```

```
>>> from sage.all import *
>>> X = Set([Integer(1),Integer(19),'a']);   Y = Set([Integer(1),Integer(1),
↪Integer(1), Integer(2)/Integer(3)])
>>> X   # random sort order
{'a', 1, 19}
>>> X == Set(['a', Integer(1), Integer(1), Integer(19)])
True
>>> Y
{1, 2/3}
>>> X.intersection(Y)
{1}
>>> print(latex(Y))
\left\{1, \frac{2}{3}\right\}
>>> Set(ZZ)
Set of elements of Integer Ring
```

6.8 Itérateurs

Les itérateurs sont un ajout récent à Python, particulièrement utile dans les applications mathématiques. Voici quelques exemples, consultez [PyT] pour plus de détails. Fabriquons un itérateur sur les carrés d'entiers positifs jusqu'à 10000000.

```
sage: v = (n^2 for n in range(10000000))
sage: next(v)
0
sage: next(v)
1
sage: next(v)
4
```

```
>>> from sage.all import *
>>> v = (n**Integer(2) for n in range(Integer(10000000)))
>>> next(v)
0
>>> next(v)
1
>>> next(v)
4
```

Nous créons maintenant un itérateur sur les nombres premiers de la forme $4p + 1$ où p est lui aussi premier, et nous examinons les quelques premières valeurs qu'il prend.

```
sage: w = (4*p + 1 for p in Primes() if is_prime(4*p+1))
sage: w
<generator object <genexpr> at 0x...>
sage: next(w)
13
sage: next(w)
29
sage: next(w)
53
```

```
>>> from sage.all import *
>>> w = (Integer(4)*p + Integer(1) for p in Primes() if is_
↪prime(Integer(4)*p+Integer(1)))
>>> w
<generator object <genexpr> at 0x...>
>>> next(w)
13
>>> next(w)
29
>>> next(w)
53
```

Certains anneaux, par exemple les corps finis et les entiers, disposent d'itérateurs associés :

```
sage: [x for x in GF(7)]
[0, 1, 2, 3, 4, 5, 6]
sage: W = ((x,y) for x in ZZ for y in ZZ)
sage: next(W)
(0, 0)
sage: next(W)
(0, 1)
sage: next(W)
(0, -1)
```

```
>>> from sage.all import *
>>> [x for x in GF(Integer(7))]
[0, 1, 2, 3, 4, 5, 6]
>>> W = ((x,y) for x in ZZ for y in ZZ)
>>> next(W)
(0, 0)
>>> next(W)
(0, 1)
>>> next(W)
(0, -1)
```

6.9 Boucles, fonctions, structures de contrôle et comparaisons

Nous avons déjà vu quelques exemples courants d'utilisation des boucles `for`. En Python, les boucles `for` ont la structure suivante, avec une indentation :

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

Notez bien les deux points à la fin de l'instruction for (il n'y a pas de « do » ou « od » comme en Maple ou en GAP) ainsi que l'indentation du corps de la boucle, formé de l'unique instruction `print(i)`. Cette indentation est significative, c'est elle qui délimite le corps de la boucle. Depuis la ligne de commande Sage, les lignes suivantes sont automatiquement

indentées quand vous appuyez sur `entrée` après un signe « : », comme illustré ci-dessous.

```
sage: for i in range(5):
....:     print(i)  # appuyez deux fois sur entrée ici
0
1
2
3
4
```

```
>>> from sage.all import *
>>> for i in range(Integer(5)):
...     print(i)  # appuyez deux fois sur entrée ici
0
1
2
3
4
```

Le signe = représente l'affectation. L'opérateur == est le test d'égalité.

```
sage: for i in range(15):
....:     if gcd(i,15) == 1:
....:         print(i)
1
2
4
7
8
11
13
14
```

```
>>> from sage.all import *
>>> for i in range(Integer(15)):
...     if gcd(i,Integer(15)) == Integer(1):
...         print(i)
1
2
4
7
8
11
13
14
```

Retenez bien que l'indentation détermine la structure en blocs des instructions `if`, `for` et `while` :

```
sage: def legendre(a,p):
....:     is_sqr_modp=-1
....:     for i in range(p):
....:         if a % p == i^2 % p:
....:             is_sqr_modp=1
```

(suite sur la page suivante)

(suite de la page précédente)

```
....:     return is_sqr_modp

sage: legendre(2,7)
1
sage: legendre(3,7)
-1
```

```
>>> from sage.all import *
>>> def legendre(a,p):
...     is_sqr_modp=-Integer(1)
...     for i in range(p):
...         if a % p == i**Integer(2) % p:
...             is_sqr_modp=Integer(1)
...     return is_sqr_modp

>>> legendre(Integer(2),Integer(7))
1
>>> legendre(Integer(3),Integer(7))
-1
```

Naturellement, l'exemple précédent n'est pas une implémentation efficace du symbole de Legendre ! Il est simplement destiné à illustrer différents aspects de la programmation Python/Sage. La fonction {kronecker} fournie avec Sage calcule le symbole de Legendre efficacement, en appelant la bibliothèque C de PARI.

Remarquons aussi que les opérateurs de comparaison numériques comme `==`, `!=`, `<=`, `>=`, `>`, `<` convertissent automatiquement leurs deux membres en des nombres du même type lorsque c'est possible :

```
sage: 2 < 3.1; 3.1 <= 1
True
False
sage: 2/3 < 3/2;   3/2 < 3/1
True
True
```

```
>>> from sage.all import *
>>> Integer(2) < RealNumber('3.1'); RealNumber('3.1') <= Integer(1)
True
False
>>> Integer(2)/Integer(3) < Integer(3)/Integer(2);   Integer(3)/Integer(2) <␣
↪Integer(3)/Integer(1)
True
True
```

Pour évaluer des inégalités symboliques, utilisez `bool` :

```
sage: x < x + 1
x < x + 1
sage: bool(x < x + 1)
True
```

```
>>> from sage.all import *
>>> x < x + Integer(1)
```

(suite sur la page suivante)

(suite de la page précédente)

```
x < x + 1
>>> bool(x < x + Integer(1))
True
```

Lorsque l'on cherche à comparer des objets de types différents, Sage essaie le plus souvent de trouver une coercition canonique des deux objets dans un même parent (voir la section *Parents, conversions, coercitions* pour plus de détails). Si cela réussit, la comparaison est faite entre les objets convertis ; sinon, les objets sont simplement considérés comme différents. Pour tester si deux variables font référence au même objet, on utilise l'opérateur `is`. Ainsi, l'entier Python (`int`) `1` est unique, mais pas l'entier Sage `1` :

```
sage: 1 is 2/2
False
sage: 1 is 1
False
sage: 1 == 2/2
True
```

```
>>> from sage.all import *
>>> Integer(1) is Integer(2)/Integer(2)
False
>>> Integer(1) is Integer(1)
False
>>> Integer(1) == Integer(2)/Integer(2)
True
```

Dans les deux lignes suivantes, la première égalité est fausse parce qu'il n'y a pas de morphisme canonique $\mathbf{Q} \to \mathbf{F}_5$, et donc pas de manière canonique de comparer l'élément 1 de $\mathbf{F}_5$ à $1 \in \mathbf{Q}$. En revanche, il y a une projection canonique $\mathbf{Z} \to \mathbf{F}_5$, de sorte que la deuxième comparaison renvoie « vrai ». Remarquez aussi que l'ordre des membres de l'égalité n'a pas d'importance.

```
sage: GF(5)(1) == QQ(1); QQ(1) == GF(5)(1)
False
False
sage: GF(5)(1) == ZZ(1); ZZ(1) == GF(5)(1)
True
True
sage: ZZ(1) == QQ(1)
True
```

```
>>> from sage.all import *
>>> GF(Integer(5))(Integer(1)) == QQ(Integer(1)); QQ(Integer(1)) ==␣
↪GF(Integer(5))(Integer(1))
False
False
>>> GF(Integer(5))(Integer(1)) == ZZ(Integer(1)); ZZ(Integer(1)) ==␣
↪GF(Integer(5))(Integer(1))
True
True
>>> ZZ(Integer(1)) == QQ(Integer(1))
True
```

ATTENTION : La comparaison est plus restrictive en Sage qu'en Magma, qui considère $1 \in \mathbf{F}_5$ comme égal à $1 \in \mathbf{Q}$.

```
sage: magma('GF(5)!1 eq Rationals()!1')  # optional - magma
true
```

```
>>> from sage.all import *
>>> magma('GF(5)!1 eq Rationals()!1')  # optional - magma
true
```

6.10 Profilage (profiling)

Auteur de la section : Martin Albrecht (malb@informatik.uni-bremen.de)

> « Premature optimization is the root of all evil. » - Donald Knuth (« L'optimisation prématurée est la source de tous les maux. »)

Il est parfois utile de rechercher dans un programme les goulets d'étranglements qui représentent la plus grande partie du temps de calcul : cela peut donner une idée des parties à optimiser. Cette opération s'appelle profiler le code. Python, et donc Sage, offrent un certain nombre de possibilités pour ce faire.

La plus simple consiste à utiliser la commande `prun` du shell interactif. Elle renvoie un rapport qui résume les temps d'exécution des fonctions les plus coûteuses. Pour profiler, par exemple, le produit de matrices à coefficients dans un corps fini (qui, dans Sage 1.0, est lent), on entre :

```
sage: k,a = GF(2**8, 'a').objgen()
sage: A = Matrix(k,10,10,[k.random_element() for _ in range(10*10)])
```

```
>>> from sage.all import *
>>> k,a = GF(Integer(2)**Integer(8), 'a').objgen()
>>> A = Matrix(k,Integer(10),Integer(10),[k.random_element() for _ in␣
↪range(Integer(10)*Integer(10))])
```

```
sage: %prun B = A*A
       32893 function calls in 1.100 CPU seconds

Ordered by: internal time

ncalls tottime percall cumtime percall filename:lineno(function)
 12127  0.160   0.000   0.160  0.000 :0(isinstance)
  2000  0.150   0.000   0.280  0.000 matrix.py:2235(__getitem__)
  1000  0.120   0.000   0.370  0.000 finite_field_element.py:392(__mul__)
  1903  0.120   0.000   0.200  0.000 finite_field_element.py:47(__init__)
  1900  0.090   0.000   0.220  0.000 finite_field_element.py:376(__compat)
   900  0.080   0.000   0.260  0.000 finite_field_element.py:380(__add__)
     1  0.070   0.070   1.100  1.100 matrix.py:864(__mul__)
  2105  0.070   0.000   0.070  0.000 matrix.py:282(ncols)
  ...
```

```
>>> from sage.all import *
>>> %prun B = A*A
       32893 function calls in 1.100 CPU seconds

Ordered by: internal time
```

(suite sur la page suivante)

(suite de la page précédente)

```
ncalls tottime percall cumtime percall filename:lineno(function)
 12127  0.160   0.000   0.160  0.000 :0(isinstance)
  2000  0.150   0.000   0.280  0.000 matrix.py:2235(__getitem__)
  1000  0.120   0.000   0.370  0.000 finite_field_element.py:392(__mul__)
  1903  0.120   0.000   0.200  0.000 finite_field_element.py:47(__init__)
  1900  0.090   0.000   0.220  0.000 finite_field_element.py:376(__compat)
   900  0.080   0.000   0.260  0.000 finite_field_element.py:380(__add__)
     1  0.070   0.070   1.100  1.100 matrix.py:864(__mul__)
  2105  0.070   0.000   0.070  0.000 matrix.py:282(ncols)
  ...
```

Ici, `ncalls` désigne le nombre d'appels, `tottime` le temps total passé dans une fonction (sans compter celui pris par les autres fonctions appelées par la fonction en question), `percall` est le rapport `tottime` divisé par `ncalls`. `cumtime` donne le temps total passé dans la fonction en comptant les appels qu'elle effectue, la deuxième colonne `percall` est le quotient de `cumtime` par le nombre d'appels primitifs, et `filename:lineno(function)` donne pour chaque fonction le nom de fichier et le numéro de la ligne où elle est définie. En règle générale, plus haut la fonction apparaît dans ce tableau, plus elle est coûteuse — et donc intéressante à optimiser.

Comme d'habitude, `prun?` donne plus d'informations sur l'utilisation du profileur et la signification de sa sortie.

Il est possible d'écrire les données de profilage dans un objet pour les étudier de plus près :

```
sage: %prun -r A*A
sage: stats = _
sage: stats?
```

```
>>> from sage.all import *
>>> %prun -r A*A
>>> stats = _
>>> stats?
```

Remarque : entrer `stats = prun -r A\*A` à la place des deux premières lignes ci-dessus provoque une erreur de syntaxe, car prun n'est pas une fonction normale mais une commande du shell IPython.

Pour obtenir une jolie représentation graphique des données de profilage, vous pouvez utiliser le profileur hotshot, un petit script appelé `hotshot2cachetree` et (sous Unix uniquement) le programme `kcachegrind`. Voici le même exemple que ci-dessus avec le profileur hotshot :

```
sage: k,a = GF(2**8, 'a').objgen()
sage: A = Matrix(k,10,10,[k.random_element() for _ in range(10*10)])
sage: import hotshot
sage: filename = "pythongrind.prof"
sage: prof = hotshot.Profile(filename, lineevents=1)
```

```
>>> from sage.all import *
>>> k,a = GF(Integer(2)**Integer(8), 'a').objgen()
>>> A = Matrix(k,Integer(10),Integer(10),[k.random_element() for _ in␣
↪range(Integer(10)*Integer(10))])
>>> import hotshot
>>> filename = "pythongrind.prof"
>>> prof = hotshot.Profile(filename, lineevents=Integer(1))
```

```
sage: prof.run("A*A")
<hotshot.Profile instance at 0x414c11ec>
sage: prof.close()
```

```
>>> from sage.all import *
>>> prof.run("A*A")
<hotshot.Profile instance at 0x414c11ec>
>>> prof.close()
```

À ce stade le résultat est dans un fichier `pythongrind.prof` dans le répertoire de travail courant. Convertissons-le au format cachegrind pour le visualiser.

Dans le shell du système d'exploitation, tapez

```
hotshot2calltree -o cachegrind.out.42 pythongrind.prof
```

Le fichier `cachegrind.out.42` peut maintenant être examiné avec `kcachegrind`. Notez qu'il est important de respecter la convention de nommage `cachegrind.out.XX`.

CHAPITRE 7

Utiliser SageTeX

Le paquet SageTeX permet d'inclure dans un document LaTeX les résultats de calculs effectués avec Sage. Il est fourni avec Sage, mais pour l'utiliser, vous aurez besoin de l'ajouter à votre installation TeX. Cette opération se résume à copier un fichier ; voyez la section *Installation* du présent tutoriel ainsi que « Make SageTeX known to TeX » dans le guide d'installation de Sage (Sage installation guide, ce lien devrait conduire à une copie locale) pour plus de détails.

Voici un bref exemple d'utilisation de SageTeX. La documentation complète se trouve dans `SAGE_ROOT/venv/share/texmf/tex/latex/sagetex`, où `SAGE_ROOT` désigne le répertoire racine de votre installation Sage. Elle est accompagnée d'un fichier exemple et de scripts Python potentiellement utiles.

Pour essayer SageTeX, suivez les instructions d'installation puis copiez le texte suivant dans un fichier `st_example.tex` :

Avertissement

Attention, tel qu'il est affiché dans la version interactive de l'aide de Sage, l'exemple suivant provoque des erreurs lors de la compilation par LaTeX. Consultez la version statique de la documentation pour voir le texte correct.

```
\documentclass{article}
\usepackage[T1]{fontenc}
\usepackage[frenchb]{babel}
\usepackage{sagetex}

\begin{document}

Sage\TeX{} permet de faire des calculs avec Sage en pla\,cant
automatiquement les r\'esultats dans un document \LaTeX.
Par exemple, l'entier $1269$ admet
$\sage{number_of_partitions(1269)}$ partitions. Le r\'esultat de ce
calcul est affich\'e sans que vous ayez \`a le calculer
vous-m\^eme ou m\^eme \`a le copier-coller dans votre document.

Un peu de code Sage :
```

(suite sur la page suivante)

(suite de la page précédente)

```
\begin{sageblock}
    f(x) = exp(x) * sin(2*x)
\end{sageblock}

La d\'eriv\'ee seconde de $f$ est

\[
  \frac{\mathrm{d}^{2}}{\mathrm{d}x^{2}} \sage{f(x)} =
  \sage{diff(f, x, 2)(x)}.
\]

Voici enfin le graphe de $f$ sur $[-1,1]$:

\sageplot{plot(f, -1, 1)}

\end{document}
```

Lancez LaTeX pour compiler `st_example.tex` comme à l'accoutumée. LaTeX va afficher un certain nombre de messages d'avertissement, dont :

```
Package sagetex Warning: Graphics file
sage-plots-for-st_example.tex/plot-0.pdf on page 1 does not exist.
Plot command is on input line 30.

Package sagetex Warning: There were undefined Sage formulas and/or
plots. Run Sage on st_example.sage, and then run LaTeX on
st_example.tex again.
```

En plus des fichiers habituellement produits par LaTeX, le répertoire où vous travaillez contient maintenant un fichier `st_example.sage`. Il s'agit d'un script Sage produit par la compilation du fichier LaTeX. L'avertissement précédent vous demande de lancer Sage sur `st_example.sage`, faites-le. Un nouveau message vous demande de relancer LaTeX sur `st_example.tex`, mais avant de le faire, observez qu'un nouveau fichier `st_example.sout` a été créé. C'est ce fichier qui contient le résultat des calculs effectués par Sage, dans un format que LaTeX est capable de lire pour insérer les résultats dans votre document. Un nouveau répertoire contenant un fichier EPS avec votre graphique est également apparu. Après une nouvelle exécution de LaTeX, les résultats des commandes Sage sont présents dans votre document compilé.

Les macros LaTeX utilisées dans l'exemple ci-dessus ne sont guère compliquées à comprendre. Un environnement `sageblock` compose le code qu'il contient verbatim, et le fait exécuter par Sage. Avec `\sage{toto}`, le résultat placé dans votre document est celui de la commande Sage `latex(toto)`. Les commandes de tracé de graphiques sont un peu plus compliquées. L'exemple ci-dessus utilise la forme la plus simple, `\sageplot{toto}`, qui insère dans le document l'image obtenue sous Sage par `toto.save('fichier.eps')`.

Pour utiliser SageTeX, la procédure est donc la suivante :

— lancez LaTeX sur votre fichier .tex ;
— lancez Sage sur le fichier .sage produit pas LaTeX ;
— lancez LaTeX une seconde fois.

(Il n'est pas nécessaire de lancer Sage si vous recompilez un document LaTeX sans avoir modifié les commandes Sage qu'il contient depuis la compilation précédente.)

SageTeX offre bien d'autres possibilités. Puisque Sage comme LaTeX sont des outils complexes et puissants, le mieux est sans doute de consulter la documentation complète de SageTeX, qui se trouve dans `SAGE_ROOT/venv/share/texmf/tex/latex/sagetex`.

CHAPITRE 8

Postface

8.1 Pourquoi Python ?

8.1.1 Les avantages de Python

Le langage d'implémentation de la base de Sage est le langage Python (voir [Py]), même si le code qui doit s'exécuter rapidement est écrit dans un langage compilé. Python présente plusieurs avantages :

— L'**enregistrement d'objets** est très facile en Python. Il existe en Python un vaste support pour enregistrer (presque) n'importe quel objet dans des fichiers sur le disque ou dans une base de données.

— Python fournit d'excellents outils pour la **documentation** des fonctions et des packages du code source, ce qui comprend l'extraction automatique de documentation et le test automatique de tous les exemples. Ces tests automatiques sont exécutés régulièrement de façon automatique, ce qui garantit que les exemples donnés dans la documentation fonctionnent comme indiqué.

— **Gestion de la mémoire :** Python possède désormais un gestionnaire de mémoire bien pensé et robuste ainsi qu'un ramasse-miettes (*garbage collector*) qui traite correctement les références circulaires et tient compte des variables locales dans les fichiers.

— **Énormément de packages** d'ores et déjà disponibles pour Python pourraient se révéler d'un grand intérêt pour les utilisateurs de Sage : analyse numérique et algèbre linéaire, visualisation 2D et 3D, réseau (pour le calcul distribué, la mise en place de serveurs - par exemple twisted), bases de données, etc.

— **Portabilité :** Python se compile sans difficulté et en quelques minutes sur la plupart des plates-formes.

— **Gestion des exception :** Python possède un système sophistiqué et bien pensé de gestion des exceptions, grâce auquel les programmes peuvent rétablir leur fonctionnement normal même si des erreurs surviennent dans le code qu'ils appellent.

— **Débogueur :** Python comprend un débogueur. Ainsi, quand un programme échoue pour une raison quelconque, l'utilisateur peut consulter la trace complète de la pile d'exécution, inspecter l'état de toutes les variables pertinentes et se déplacer dans la pile.

— **Profileur :** Il existe un profileur Python, qui exécute le code et renvoie un rapport qui détaille combien de fois et pendant combien de temps chaque fonction a été appelée.

— **Un langage :** Au lieu d'écrire un **nouveau langage** pour les mathématiques comme cela a été fait pour Magma, Maple, Mathematica, Matlab, GP/PARI, GAP, Macaulay 2, Simath, etc., nous utilisons le langage Python, qui est un langage de programmation répandu, activement développé et optimisé par des centaines de développeurs qualifiés. Python, avec son processus de développement éprouvé, fait partie des success stories majeures de l'open source (see [PyDev]).

8.1.2 Le préprocesseur Sage et les différences entre Sage et Python

Certains aspects mathématiques de Python peuvent induire des confusions. Aussi, Sage se comporte différemment de Python à plusieurs égards.

— **Notation de l'exponentiation :** `**` au lieu de `^`. En Python, `^` désigne le « xor » (ou exclusif bit à bit) et non l'exponentiation. Ainsi en Python, on a

```
>>> 2^8
10
>>> 3^2
1
>>> 3**2
9
```

Cette utilisation de `^` peut paraître étrange et surtout inefficace pour une utilisation purement mathématique puisque le ou exclusif n'est que rarement utilisé. Par commodité, Sage prétraite chaque ligne de commande avant de la transmettre à Python, en remplaçant par exemple les apparitions de `^` (en dehors des chaînes de caractères) par des `**` :

```
   sage: 2^8
   256
   sage: 3^2
   9
   sage: "3^2"
   '3^2'

Le ou exclusif bit à bit est quant à lui noté ``^^``, et l'opération en
place ``^^=`` fonctionne comme on s'y attend :

::

    sage: 3^^2
    1
    sage: a = 2
    sage: a ^^= 8
    sage: a
    10
```

— **Division entière : L'expression Python `2/3` ne se comporte** pas de la manière à laquelle s'attendraient des mathématiciens. En Python 3, si `m` et `n` sont de type `int`, alors `m/n` est de type `float`, c'est le quotient réel de `m` par `n`. Par exemple, `2/3` renvoie `0.6666...`. Pour obtenir le quotient entier, il faut utiliser `2//3` qui renvoie `0`.

Dans l'interpréteur Sage, nous réglons cela en encapsulant automatiquement les entiers littéraux par `Integer( )` et en faisant de la division un constructeur pour les nombres rationnels. Par exemple :

```
sage: 2/3
2/3
sage: (2/3).parent()
Rational Field
sage: 2//3
0
```

```
>>> from sage.all import *
>>> Integer(2)/Integer(3)
```

(suite sur la page suivante)

(suite de la page précédente)

```
2/3
>>> (Integer(2)/Integer(3)).parent()
Rational Field
>>> Integer(2)//Integer(3)
0
```

— **Entiers longs :** Python possède nativement un support pour les entiers de précision arbitraire, en plus des int du langage C. Les entiers longs Python sont significativement plus lents que ceux que GMP fournit et sont marqués à l'affichage par un `L` qui les distingue des int (il est pas prévu de changer cela à court terme). Sage implémente les entiers en précision arbitraire en utilisant la bibliothèque C GMP. Les entiers longs GMP utilisés par Sage s'affichent sans le `L`.

Plutôt que de modifier l'interpréteur Python (comme l'ont fait certaines personnes pour leurs projets internes), nous utilisons le langage Python exactement comme il est et rajoutons un pré-parseur pour IPython de sorte que la ligne de commande de IPython se comporte comme l'attend un mathématicien. Ceci signifie que tout code Python existant peut être utilisé sous Sage. Toutefois, il faut toujours respecter les règles standards de Python lorsque l'on écrit des packages à importer dans Sage.

(Pour installer une bibliothèque Python, trouvée sur Internet par exemple, suivez les instructions mais exécutez `sage -python` au lieu de `python`. La plupart du temps, ceci signifie concrètement qu'il faut taper `sage -python setup.py install`.)

8.2 Comment puis-je contribuer ?

Si vous souhaitez contribuer au développement de Sage, votre aide sera grandement appréciée ! Cela peut aller de contributions substantielles en code au signalement de bogues en passant par l'enrichissement de la documentation.

Parcourez la page web de Sage pour y trouver les informations pour les développeurs. Entre autres choses, vous trouverez une longue liste de projets en lien avec Sage rangés par priorité et catégorie. Le Guide du développeur Sage (Sage Developer's Guide) contient également des informations utiles. Vous pouvez aussi faire un tour sur le groupe Google `sage-devel`.

8.3 Comment citer Sage ?

Si vous écrivez un article qui utilise Sage, merci d'y préciser les calculs faits avec Sage en citant

```
[Sage] SageMath, the Sage Mathematics Software System (Version 8.7),
       The Sage Developers, 2019, https://www.sagemath.org.
```

dans votre bibliographie (en remplaçant 8.7 par la version de Sage que vous avez utilisée). De plus, pensez à rechercher les composants de Sage que vous avez utilisés pour vos calculs, par exemple PARI, GAP, Singular, Maxima et citez également ces systèmes. Si vous vous demandez quel logiciel votre calcul utilise, n'hésitez pas à poser la question sur le groupe Google `sage-devel`. Voir *Polynômes univariés* pour une discussion plus approfondie de ce point.

Si vous venez de lire d'une traite ce tutoriel et que vous avez une idée du temps qu'il vous a fallu pour le parcourir, merci de nous le faire savoir sur le groupe Google `sage-devel`.

Amusez-vous bien avec Sage !

CHAPITRE 9

Annexe

9.1 Priorité des opérateurs arithmétiques binaires

Combien font `3^2*4 + 2%5` ? Le résultat (38) est déterminé par le « tableau de priorité des opérateurs » suivant. Il est dérivé de celui donné § 5.14 du manuel de référence de Python (*Python Language Reference Manual*, de G. Rossum et F. Drake.) Les opérations sont données par priorités croissantes.

Opérateur	Description
or	ou booléen
and	et booléen
not	négation booléenne
in, not in	appartenance
is, is not	test d'identité
>, <=, >, >=, ==, !=	comparaisons
+, -	addition, soustraction
*, /, %	multiplication, division, reste
**, ^	exponentiation

Ainsi, pour calculer `3^2*4 + 2%5`, Sage « met les parenthèses » comme suit : `((3^2)*4) + (2%5)`. Il calcule donc d'abord `3^2`, ce qui fait `9`, puis `(3^2)*4` et `2%5`, et enfin ajoute les valeurs de ces deux dernières expressions.

CHAPITRE 10

Bibliographie

CHAPITRE 11

Index et tables

— genindex
— modindex
— search

Bibliographie

[Cyt] Cython, http://www.cython.org

[GAP] The GAP Group, GAP - Groups, Algorithms, and Programming, Version 4.4 ; 2005, https://www.gap-system.org

[GAPkg] GAP Packages, https://www.gap-system.org/Packages/packages.html

[GP] PARI/GP, https://pari.math.u-bordeaux.fr/

[Mag] Magma, http://magma.maths.usyd.edu.au/magma/

[Max] Maxima, http://maxima.sf.net/

[NagleEtAl2004] Nagle, Saff, and Snider. *Fundamentals of Differential Equations*. 6th edition, Addison-Wesley, 2004.

[Py] The Python language, http://www.python.org/

[PyB] The Python Beginner's Guide, https://wiki.python.org/moin/BeginnersGuide

[PyDev] Python Developer's Guide, https://docs.python.org/devguide/

[PyLR] Python Library Reference, https://docs.python.org/fr/3/library/index.html

[Pyr] Pyrex, http://www.cosc.canterbury.ac.nz/~greg/python/Pyrex/

[PyT] The Python Tutorial, https://docs.python.org/fr/3/tutorial/

[SA] Sage web site, https://www.sagemath.org/

[Si] G.-M. Greuel, G. Pfister, and H. Schönemann. Singular 3.0. A Computer Algebra System for Polynomial Computations. Center for Computer Algebra, University of Kaiserslautern (2005). https://www.singular.uni-kl.de

[SJ] William Stein, David Joyner, Sage : System for Algebra and Geometry Experimentation, Comm. Computer Algebra {39} (2005) 61-64.

[ThreeJS] three.js, http://threejs.org

Index

E

EDITOR, 95

V

variable d'environnement
    EDITOR, 95